

SCHOOL OF COMPUTER SCIENCE

Towards Spatio-Temporal Superintelligence: Augmenting Spatial Vision-Language Models with Memory

Thomas Bale

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Bachelor of Science in the Faculty of Engineering **worth 40CP**.

Friday 6th November, 2026

Abstract

Vision-language models have demonstrated remarkable capabilities in understanding visual information, yet these models face fundamental limitations when processing videos for spatial reasoning tasks. Without mechanisms for attending/linking previous information against newer information, standard architectures struggle to maintain coherent spatial understanding across extended video sequences, leading to significant performance degradation as video length increases.

This project attempts to addresses this limitation by augmenting the VLM-3R architecture—a state-of-the-art vision-language model with instruction-aligned 3D reconstruction capabilities—with two complementary mechanisms. First, a dual-memory module inspired by VLM^2 maintains both working memory (a FIFO sliding window for recent frames) and episodic memory (similarity-based long-term storage), enabling sustained spatial understanding across video frames. Second, query-based frame selection adapted from VideoStreaming filters frames for relevance to the specific spatial reasoning question, reducing redundancy and improving precision.

The results demonstrate that augmenting VLM-3R with non-trainable dual memory and query-based frame selection preserves baseline performance on VSI-Bench (60.9%) and VSTI-Bench (58.8%), while query-based frame selection additionally improves camera-object relative distance reasoning from 68.6% to 71.8%. However, trainable memory variants and combined architectures consistently underperform the reproduced baseline, with the jointly trained query-selection and memory model falling to 47.3% on VSTI-Bench without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes.

These findings suggest that lightweight retrieval-based mechanisms can maintain baseline spatial reasoning capabilities without degradation, but architectural complexity in the form of trainable memory or combined query-memory pipelines introduces optimisation challenges that lead to worse performance without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes.

Achievements

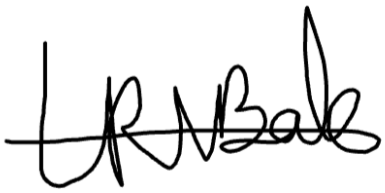
- Successfully reproduced VLM-3R benchmark performance on VSI-Bench (60.9%) and VSTI-Bench (58.7%)
- Implemented and evaluated a non-trainable dual-memory module with working and episodic memory systems
- Implemented and evaluated a trainable dual-memory module with learnable attention and gated fusion
- Implemented and evaluated query-based frame selection for relevance filtering of video frames
- Analysed the augmented model's performance across video length categories to assess long-horizon spatial reasoning
- Conducted ablation studies on memory buffer sizing, alpha blending, the query based number of selected frames, and trainable versus non-trainable configurations
- In depth analysis of where the augmentations harm or improve the model with respect to the VSI and VSTI spatial and spatiotemporal benchmarks.

Dedication and Acknowledgements

I would like to thank my supervisor Dr Xiang Li for introducing me to this domain; his generosity in providing access to appropriate hardware; and for his continuous support and guidance throughout this project.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

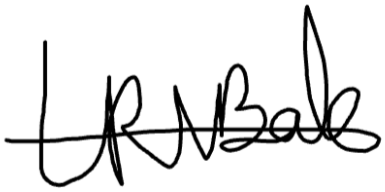


Thomas Bale, Friday 6th November, 2026

AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.



Thomas Bale, Friday 6th November, 2026

Contents

Notation and Acronyms **xii**

1 Introduction **1**
1.1 Background . . . 1
1.2 Previous Works . . . 2
1.3 Current Limitations . . . 3
1.4 Project Aims . . . 4
1.5 Experimental Findings . . . 5

2 Contextual Background **6**
2.1 A Brief History of Artificial Intelligence . . . 6
2.2 Video Understanding with Vision-Language Models . . . 12
2.3 3D Reconstruction . . . 13
2.4 3D Scene Understanding from Monocular Video . . . 14
2.5 Memory Systems in Artificial Intelligence . . . 14
2.6 Hardware in the Age of AI . . . 15

3 Methods **17**
3.1 Method Overview . . . 17
3.2 VLM-3R . . . 17
3.3 Our Architecture . . . 18
3.4 The Dual Memory Module . . . 19
3.5 Non-Trainable Memory . . . 21
3.6 Trainable Memory . . . 22
3.7 Query-based Frame Selection . . . 24
3.8 Combined Architecture . . . 25

4 Experiments and Results **26**
4.1 Hardware . . . 26
4.2 Data Preprocessing . . . 26
4.3 Spatial Reasoning Benchmarks . . . 28
4.4 VLM-3R Performance Reproduction . . . 32
4.5 Non-Trainable Memory . . . 34
4.6 Trainable Memory . . . 35
4.7 Query-based Frame Selection . . . 39
4.8 Combining Approaches . . . 40

5 Results Discussions **42**
5.1 Core VSI and VSTI Bench Results . . . 42
5.2 Long-horizon spatial reasoning performance . . . 46
5.3 Comparison with Human Performance . . . 48
5.4 Extended Task Category Analysis . . . 48
5.5 Uncertainty and Paired Significance . . . 50
5.6 Scene Complexity Split . . . 51
5.7 Bias and Confusion Across Architectures . . . 51
5.8 Cross-Architecture Failure Comparison . . . 52
5.9 Frame Selection Analysis . . . 53

5.10 Failure Mode Analysis 54

6 Conclusion **57**
6.1 Contributions and Findings 57
6.2 Project Status 57
6.3 Future Work 58

A Appendix A: AI Prompts/Tools **66**
A.1 Tools 66
A.2 Example Prompts 66

B Appendix B: Supplementary Material **68**
B.1 Qualitative examples on VSI-Bench 68
B.2 Qualitative examples on VSTI-Bench 68

C Appendix C: Project Hardware **74**
C.1 Hardware Access Timeline 74

List of Figures

1.1 VSI-Bench example QA pairs . . . 2

2.1 Scaled Dot-Product and Multi-Head Attention . . . 8
2.2 Transformer encoder-decoder architecture . . . 8
2.3 Tokenisation example . . . 8
2.4 Vector embeddings in 3D space . . . 9
2.5 LLM training stages flow diagram . . . 11
2.6 Vision-Language Model Architecture . . . 12
2.7 CUT3R method overview . . . 14

3.1 VLM-3R network architecture . . . 18
3.2 Modified VLM-3R with dual-memory module . . . 19
3.3 The dual memory module consists of 3 parts: working memory, episodic memory and fusion. Reprinted from Liu et al. [50]. . . . 20

4.1 VSi-Bench task demonstration and benchmark statistics . . . 29
4.5 VSTI-Bench overview and task categories . . . 30
4.8 Demo inference example . . . 33
4.9 Exploding gradient phenomenon during initial training . . . 38

5.1 VSTI-Bench comparison of best method variants . . . 44
5.2 Examples of reconstructed scenes from the ScanNet dataset (top) [18] and Arkitscenes dataset (bottom) [20]. . . . 49
5.3 Bootstrap confidence intervals for benchmark scores . . . 51
5.4 Scene-complexity split on VSTI-Bench . . . 52
5.5 Mean relative error on numerical tasks . . . 53
5.6 VSTI-Bench multiple-choice option confusion . . . 53
5.7 Per-question-type score delta versus baseline . . . 54
5.8 Query-selection trace for a VSTI-Bench example . . . 55

B.1 **Qualitative examples on VSI-Bench [92].** . . . 69
B.2 **Qualitative examples on VSI-Bench [92].** . . . 70
B.3 **Qualitative examples on VSI-Bench [92].** . . . 71
B.4 **Qualitative examples on VSTI-Bench [26].** . . . 72
B.5 **Qualitative examples on VSTI-Bench [26].** . . . 73

List of Tables

1.1 VLM-3R vs Human Performance on Spatial Benchmarks . . . 3

4.1 VSI-Bench Results: Ours vs Reported . . . 34
4.2 VSTI-Bench Results: Ours vs Reported . . . 34
4.3 VSI-Bench Initial Non-Trainable Memory Results . . . 34
4.4 VSTI-Bench Initial Non-Trainable Memory Results . . . 34
4.5 VSI-Bench Full Non-Trainable Ablation Results . . . 35
4.6 VSTI-Bench Full Non-Trainable Ablation Results . . . 35
4.7 VSTI-Bench Memory Buffer Sizing Ablation Results . . . 35
4.8 Multihead Attention Configuration for Trainable Dual Memory Module . . . 36
4.9 VSTI-Bench Results: Baseline vs Trainable Memory . . . 38
4.10 VSTI-Bench Results: Baseline vs Query-Based Frame Selection . . . 40
4.11 VSTI-Bench Results: Baseline vs Query Selection vs Combined Query + Memory . . . 41

5.1 VSI-Bench evaluation results . . . 43
5.2 VSTI-Bench evaluation results . . . 43
5.3 **Additional long-horizon spatial reasoning results on VSI-Bench [92] and VSTI-Bench [26].** Videos are grouped into Short (< 1 min), Mid (1–2 min), and Long (> 2 min). For Spatial-MLLM [88], the original setting uses 16 input frames for evaluation, while methods marked with * correspond to a 32-frame input setting, used to ensure a consistent frame setting across methods. For VSTI-Bench, the *Avg.* column follows the long-horizon convention from VLM-3R and uses a question-count-weighted average over the five top-level categories, rather than the unweighted five-category average used elsewhere in this report. Our VSTI-Bench Short, Mid, and Long rows are recomputed from the cached benchmark MP4 durations with thresholds < 60 s, 60–120 s, and > 120 s respectively. . . . 47
5.4 VSI-Bench (Tiny) Human Performance vs. Our Full-Benchmark Results . . . 48
5.5 VSTI-Bench (Tiny) Human Performance vs. Our Full-Benchmark Results . . . 48

List of Algorithms

1 Dual-Memory Module (Non-Trainable) 21
2 Dual-Memory Module 23
3 Query-Based Frame Selection 25

Ethics Statement

This project did not require ethical review, as determined by my supervisor, Dr. Xiang Li

Supporting Technologies

Hardware Infrastructure

Training and evaluation were conducted on two HPC systems managed via SLURM (Simple Linux Utility for Resource Management). Jobs loaded CUDA modules (cuda/12.6 and cuda/11.8 available) for GPU support:

- **Isambard-AI** – The UK's fastest AI supercomputer, built by the Bristol Centre for Supercomputing at the University of Bristol. The system uses NVIDIA GH200 Grace Hopper Superchips combining a 72-core CPU with H100 GPUs. Phase 2 deploys 5,280 superchips across 1,362 compute nodes.
- **BluePebble** – A non-homogeneous cluster with GPU-enabled nodes. The MLCNU partition contains two nodes, each with four high-memory NVIDIA A100 GPUs.

Software Framework and Dependencies

The implementation was based on the VLM-3R codebase with the following key dependencies:

- **PyTorch 2.1.2** – Core deep learning framework for tensor computation and automatic differentiation.
- **DeepSpeed 0.12.2** – Distributed training library enabling ZeRO-2 optimization, gradient checkpointing, and mixed precision training.
- **Accelerate 0.29.3** – Hugging Face library for simplified multi-GPU training orchestration via `torchrun`.
- **FlashAttention 2.5.7** – Memory-efficient attention implementation.
- **Weights & Biases (wandb 0.16.5)** – Experiment tracking for training metrics.
- **Hugging Face Transformers 4.40.0.dev0** – Tokenizer, model configuration, and dataset utilities.
- **PEFT 0.4.0** – Parameter-efficient fine-tuning library implementing Low-Rank Adaptation (LoRA).

Notation and Acronyms

General		
AI	:	Artificial Intelligence
LLMs	:	Large Language Models
VLMs	:	Vision-Language Models
NLP	:	Natural Language Processing
Vision & Geometry		
ViT	:	Vision Transformer
CLIP	:	Contrastive Language-Image Pre-training
SigLIP	:	Sigmoid Language-Image Pre-training
SfM	:	Structure from Motion
MVS	:	Multi-View Stereo
NeRFs	:	Neural Radiance Fields
SLAM	:	Simultaneous Localisation and Mapping
3D	:	Three-Dimensional
2D	:	Two-Dimensional
Neural Network Components		
CNN(s)	:	Convolutional Neural Network(s)
RNN(s)	:	Recurrent Neural Network(s)
LSTM	:	Long Short-Term Memory
GRU	:	Gated Recurrent Unit
MLP	:	Multi-Layer Perceptron
GQA	:	Grouped Query Attention
RMSNorm	:	Root Mean Square Layer Normalisation
Training & Adaptation		
RLHF	:	Reinforcement Learning from Human Feedback
PPO	:	Proximal Policy Optimisation
SFT	:	Supervised Fine-Tuning
LoRA	:	Low-Rank Adaptation
PEFT	:	Parameter-efficient fine-tuning
Benchmarks		
VLM-3R	:	Vision-Language Models with Instruction-Aligned 3D Reconstruction
VLM^2	:	Vision-Language Memory for Spatial Reasoning
VSiBench	:	Vision-Spatial Intelligence Benchmark
VSTI-Bench	:	Vision-Spatial-Temporal Intelligence Benchmark
QA	:	Question Answering
Hardware		
GPU	:	Graphics Processing Unit
CPU	:	Central Processing Unit
TPU	:	Tensor Processing Unit
HBM	:	High-Bandwidth Memory
HPC	:	High Performance Computing/Computer
TFLOPS	:	Tera Floating-Point Operations Per Second
FP16/FP32/FP8	:	Floating Point 16/32/8-bit
FMA	:	Fused Multiply-Add
CUDA	:	Compute Unified Device Architecture

NVLink	:	NVIDIA High-Speed GPU Interconnect
InfiniBand	:	High-Speed Cluster Interconnect
	Utilities	
SLURM	:	Simple Linux Utility for Resource Management

Chapter 1

Introduction

1.1 Background

Recent advancements in vision-language models demonstrate significant potential for understanding visual content in context [7]. The release of GPT-4V introduced novel capabilities for image understanding [60], and subsequent models have pushed the boundaries of visual reasoning. However, spatial reasoning over long videos remains challenging [26, 92] — models struggle to maintain coherent understanding as video length increases and often process frames uniformly without filtering for question relevance [63].

The VLM^2 paper [50] identified spatial reasoning over long videos as a critical challenge by analysing VSI-Bench and VSTI-Bench benchmarks by video length [50, 92, 26]. Results showed that model performance degrades significantly as video length increases, suggesting fundamental limitations in existing architectures for maintaining coherent spatial understanding. Parallel work on VideoStreaming [63] approached the same long-video problem through adaptive query-based frame selection to identify question-relevant frames.

Spatial AI, which enables machines to understand and reason about 3D space from visual inputs, has applications across numerous domains where geometric understanding and physical interaction are essential.

Robotics and autonomous manipulation rely fundamentally on spatial understanding for tasks such as grasping, navigation, and object manipulation. Systems like the robotic vaccum cleaners and more advanced surgical robots must perceive the 3D structure of their environments, track object positions, and plan collision-free trajectories. Whilst, dense 3D mapping through SLAM (Simultaneous Localisation and Mapping) enables robots to build geometric representations of unknown environments while localising themselves within them [10], AI with concrete spatial understanding allows systems like these to deployed in a much wider range of applications.

Autonomous vehicles require rich spatial understanding to navigate safely. Self-driving systems combine multiple sensor modalities including cameras, LiDAR, and GPS to perceive the 3D structure of roads, identify pedestrians and obstacles, and plan safe trajectories. The KITTI benchmark established standard evaluation protocols for autonomous driving perception, demonstrating progress in tasks like 3D object detection and odometry estimation from visual data [28].

Medical image analysis involves reasoning about 3D anatomy from volumetric data such as CT and MRI scans, as well as physical examinations assisted by endoscopic cameras. 3D U-Net architectures enable segmentation of anatomical structures in medical volumes, supporting surgical planning and diagnostic analysis [13]. Applications range from tumour delineation in radiation therapy to guiding minimally invasive procedures where spatial awareness is critical [25].

Surveillance and scene understanding systems apply spatial reasoning to video streams for activity recognition, anomaly detection, and person re-identification across cameras. Understanding the 3D layout of monitored spaces enables more accurate tracking and behaviour analysis than purely 2D approaches [29]. Cross-camera tracking requires geometric reasoning about camera viewpoints and spatial consistency to link observations across non-overlapping views [44].



Figure 1.1: Examples of spatial reasoning question-answer pairs from VSI-Bench. The benchmark covers tasks including object counting, relative distance, relative direction, appearance order, object size, absolute distance, room size, and route planning [50, 92]. This shows the sort of tasks identified by the VSI-Benchmarks that are fundamental to spatial reasoning.

1.2 Previous Works

Building on these motivations, several prior works have tackled spatial reasoning in vision-language models.

Video Understanding with VLMs. VideoLLM [89] combined video encoders with LLMs to enable temporal reasoning over video content, establishing foundational approaches for modelling video sequences with language models. Building on this, LLaVA-NeXT-Video [98, 51] advanced video understanding through improved visual encoding and reasoning capabilities, providing the base architecture upon which later spatial reasoning models were built.

VLM-3R. VLM-3R [26] introduced instruction-aligned 3D reconstruction into vision-language models for spatial reasoning, unifying visual encoding with geometric understanding through CUT3R-based spatial tokens.

VSI-Bench. VSI-Bench (Vision-Spatial Intelligence Benchmark) [92] was introduced by the Thinking in Space work, containing approximately 5,000 question-answer pairs derived from 288 real-world indoor videos across eight spatial reasoning task types: object counting, relative distance, relative direction, appearance order, object size, absolute distance, room size, and route planning.

VSTI-Bench. VSTI-Bench (Vision-Spatial-Temporal Intelligence Benchmark) [26] was introduced alongside VLM-3R to address temporal spatial reasoning, comprising approximately 138,600 QA pairs across five task categories assessing how spatial relationships evolve over time during video capture, with tasks including camera displacement, object-camera distance changes, and evolving spatial configurations during navigation.

VLM^2. VLM^2 demonstrated that spatial reasoning performance degrades significantly as video length increases [50, 92], highlighting the need for memory mechanisms in long-video understanding. VLM^2 [50] uses π_3 as its 3D foundation model, differing architecturally from VLM-3R which employs CUT3R for spatial token generation. VLM^2 introduces a dual-memory architecture comprising a working memory (sliding window FIFO over recent frames) and an episodic memory (fixed-capacity similarity-based storage), accumulated across video frames to maintain coherent spatial understanding. The model also incorporates a viewpoint-aware geometry alignment module and adaptive 3D position injection to enforce geometric consistency across frames.

Query-Based Frame Selection. VideoStreaming [63] approached long-video processing through adap-

Table 1.1: Comparison of VLM-3R and human performance on VSI-Bench (Tiny) and VSTI-Bench (Tiny). VLM-3R substantially underperforms humans on numerical estimation tasks (abs. distance, object size, room size) and object counting, while achieving competitive performance on multiple-choice directional and relative reasoning tasks.

	Avg.	*Obj. Count*	*Abs. Dist.*	*Obj. Size*	*Room Size*	*Rel. Dist.*	*Rel. Dir.*	*Route Plan*	*Appr. Order*
	VSI-Bench (Tiny)								
Human	**79.2**	**94.3**	47.0	**69.2**	**67.1**	**94.7**	**95.8**	**95.8**	**100.0**
VLM-3R	60.9	70.2	**49.4**	60.4	45.9	65.4	80.5	45.4	40.1
Gap	-18.3	-24.1	+2.4	+8.8	+21.2	-29.3	-15.3	-50.4	-59.9

tive frame selection, demonstrating that query-based filtering could identify question-relevant clips while discarding redundant information, enabling more efficient and precise video understanding.

More broadly, the spatial AI literature has established depth estimation from monocular images [99], SLAM systems for simultaneous localisation and mapping [10], and feed-forward multi-view reconstruction methods [86, 85]. This project draws from these established bodies of work in an attempt to enhance vision-language models for long-video spatial reasoning tasks.

1.3 Current Limitations

Existing vision-language models face several fundamental limitations when processing videos for spatial reasoning tasks [50, 26]. Standard VLMs lack persistent memory mechanisms, making it difficult to maintain coherent understanding of spatial relationships and object states as video length increases [50]. Furthermore, processing all frames uniformly introduces significant redundancy—not all frames are equally relevant to a given spatial reasoning question [63], and performance degrades substantially as video duration increases [50, 92, 26]. Even spatially enhanced models achieve substantially lower performance than humans on many tasks [92, 26].

Lack of Persistent Memory. Standard architectures process each frame independent of each other, without mechanisms to retain or attend against information from previously seen frames [26]. This prevents models from building cumulative understanding across video sequences.

Performance Degradation with Video Length. The VLM^2 paper analysis on VSI-Bench and VSTI-Bench benchmarks demonstrated that spatial reasoning accuracy decreases significantly as video duration increases [50, 92, 26]. Models struggle to make links between spatial information from earlier frames when answering questions about later frames. This degradation is partly attributable to naive frame selection—processing all frames uniformly without filtering for question relevance [63] as well as the lack of inter frame attention [50].

Sub-Human Performance in Spatial Reasoning Tasks. As illustrated in Figure 1.1, VSI-Bench evaluates eight distinct spatial reasoning categories. The task types range from object counting to absolute distance estimation, revealing distinct difficulty levels. VSI-Bench and VSTI-Bench (Tiny) (the evaluated human subset) highlight that, even spatially enhanced VLMs like VLM-3R achieve substantially lower performance than humans on many tasks, as shown in Table 1.1. VLM-3R achieves 60.9% average on VSI-Bench (Tiny) compared to human performance of 79.2%, and 58.8% on VSTI-Bench (Tiny) compared to 77.0% for humans [92, 26].

Naive Frame Selection from Video Inputs Previous architectures store frames uniformly but retrieves from all stored memories equally, without filtering for relevance to the current question [63]. This means irrelevant frames contribute noise to retrieval, reducing precision in spatial reasoning tasks.

1.4 Project Aims

1.4.1 Approach Overview

To address the limitations of existing VLMs in long-video spatial reasoning, this project augments the VLM-3R architecture with two complementary mechanisms. First, drawing from VLM^2's dual-memory design [50], we implement working memory (a FIFO sliding window capturing recent frames) and episodic memory (similarity-based long-term storage), enabling sustained spatial understanding across video sequences. Second, inspired by VideoStreaming [63], we incorporate query-based frame selection that filters frames for relevance to the specific spatial reasoning question, reducing redundancy and improving precision. Both mechanisms are inserted between VLM-3R's spatial and visual encoders and projectors and the language model, operating on the 3D-aware visual features produced by the Spatial-Visual-View block.

These mechanisms directly address the stated limitations. The dual-memory architecture explicitly tackles the lack of persistent memory by providing complementary short-term and long-term storage, allowing the model to build cumulative understanding rather than processing each frame in isolation. Query-based frame selection complements this by filtering for question-relevance at the input level, reducing the noise from irrelevant frames that uniform storage introduces. Working memory ensures recent frames remain accessible even in long sequences, while episodic memory preserves salient spatial information that would otherwise be lost when the FIFO window advances. The trainable variant additionally allows the fusion mechanism to learn which information is most valuable for spatial reasoning, potentially adapting to different task types. Together, these approaches target the core weaknesses identified in VLM-3R's performance degradation on longer videos and the sub-human accuracy on spatial reasoning tasks.

1.4.2 Objectives

The high-level objective of this project is to enhance long-horizon spatial reasoning in vision-language models through dual-memory architectures and query-based frame selection and subsequently evaluate where the augmentations change performance. Based on the well-known VLM-3R model [26], we aim to enhance the methods with complementary memory systems (a working memory for immediate context and an episodic memory for historical information [50]) and query-based frame selection for relevance filtering [63], enabling more effective spatial reasoning across extended video understanding tasks. More specifically, the concrete aims are:

1. Recreate the performance claimed by VLM-3R model.
2. Implement and evaluate a non-trainable dual-memory module that augments the VLM-3R architecture with working and episodic memory systems.
3. Implement, train and evaluate a trainable dual-memory module that augments the VLM-3R architecture with working and episodic memory systems.
4. Implement, train and evaluate query based frame selection that augments the VLM-3R architecture.
5. Assess the impact of dual-memory on spatial reasoning performance across VSI-Bench and VSTI-Bench benchmarks, particularly for longer video sequences.
6. Assess the impact of query based frame selection on spatial reasoning performance across VSI-Bench and VSTI-Bench benchmarks, particularly for complex scenes.
7. Compare trainable and non-trainable memory approaches to understand the trade-offs between adaptation capability and implementation simplicity.
8. Analyse the frames chosen by the query based frame selection and the information in those frames compared to the query.
9. Analyse the failure modes of vision-language models on spatial reasoning tasks to identify where memory and frame selection augmentation changes performance.

1.5 Experimental Findings

We successfully reproduced the VLM-3R baseline, achieving 60.9% on VSI-Bench and 58.8% on VSTI-Bench, matching the reported performance. Our primary experimental findings are:

Non-trainable memory preserves baseline performance but does not improve it. The non-trainable dual-memory module with working and episodic memory achieved 60.9% on VSI-Bench (matching baseline) and 58.8% on VSTI-Bench (matching baseline), with only minor variations across categories. This confirms that memory retrieval alone, without learned adaptation, cannot improve spatial reasoning beyond the VLM-3R baseline.

Query-based frame selection preserves baseline on VSTI-Bench while improving relative distance reasoning. Query selection (128-to-32 configuration) matched the baseline at 58.8% overall on VSTI-Bench while improving camera-object relative distance from 68.6% to 71.8%. However, on VSI-Bench it significantly degraded performance to 43.9%, indicating that frame selection is not uniformly beneficial across benchmarks.

Trainable memory and combined architectures consistently underperform the baseline. The trainable memory variant achieved only 54.1% on VSTI-Bench (compared to 58.8% baseline), and the combined query-selection plus trainable-memory architecture fell to 47.3%, a substantial degradation of 11.5 percentage points below baseline. This indicates that introducing learnable parameters without sufficient architectural justification or training data leads to optimization difficulties rather than improved capabilities without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes.

Long-horizon spatial reasoning does not benefit from memory augmentation on VSI-Bench. Partitioning videos by duration (Short, Mid, Long) reveals that query selection alone does not solve the long-sequence degradation problem: our best query-selection variant remains below VLM-3R at every duration bucket on VSI-Bench, with the largest gap on mid and long videos. On VSTI-Bench, both non-trainable memory and query selection preserve baseline behavior across all duration buckets, but do not improve long-horizon performance.

The gap between model and human performance remains substantial at 18 percentage points. Compared to human evaluators on VSI-Bench (Tiny, 400 questions), our best model achieves 60.9% versus human 79.2%. The gap is not uniform: humans dramatically outperform our model on temporal reasoning tasks (route planning: 95.8 vs 45.4, appearance order: 100.0 vs 40.0) while our model matches or exceeds human performance on some absolute metric tasks (room size: 67.7 vs 45.9). This suggests that human spatial memory operates fundamentally differently from retrieval-based memory architectures.

Chapter 2

Contextual Background

2.1 A Brief History of Artificial Intelligence

From symbolic reasoning to large language models, AI represents one of the most significant technological transformations in computing history. Understanding the history of AI highlights why modern AI systems possess their remarkable capabilities and limitations.

2.1.1 Artificial Intelligence 1.0

The Dawn of Symbolic AI (1950s–1980s)

Artificial intelligence began with the insight that human reasoning could be formalised and implemented on computers. Alan Turing's 1950 paper proposed the question "Can machines think?" and introduced what became known as the Turing Test as a measure of machine intelligence [83]. Early AI research focused on symbolic approaches: hand-crafted rules and logical inference systems to encode human knowledge explicitly. The Logic Theorist [57] and expert systems such as DENDRAL [46] and MYCIN [77] demonstrated that computers, given some predefined rules could manipulate symbols to solve constrained logical problems. However, these systems suffered from 'the knowledge bottleneck', where manually encoding sufficient knowledge for real-world reasoning was too expensive. [71].

The Rise of Statistical Learning (1980s–2010s)

The recognition that machines could learn patterns from data rather than relying entirely on hand-coded rules was the next turning point. Neural networks, inspired by biological neurons, could learn hierarchical representations from raw data, enabled by the backpropagation algorithm; though early successes were limited to small-scale problems [70]. Support vector machines and ensemble methods like random forests [8] then dominated machine learning competitions throughout the 2000s, with better generalisation and interpretability than neural networks on structured data tasks [17]. This era of AI 1.0 ended under the assumption that progress required careful feature engineering, where humans designed data representations optimised for specific tasks [6].

2.1.2 Artificial Intelligence 2.0

The Deep Learning Revolution (2012–2017)

Convolutional neural networks (CNNs) and their success on the ImageNet challenge marked the next era of AI[21, 40]. AlexNet (2012), a deep CNN trained on GPUs, reduced image classification error rates from 26% to 15%, when progress had been small for years[39]. This effectively demonstrated neural networks could learn their hierarchical features directly from pixels, no longer requiring hand-crafted feature engineering. Next came systems surpassing human performance on narrow tasks: ResNet (2015) achieved 3.6% error through 'skip connections': enabling training of very deep (many hidden layers of neurons) networks [31], and YOLO (You Only Look Once) enabled real-time object detection in images [69].

These same principles (known as deep learning principles) extended to sequence modelling. Recurrent neural networks (RNNs) and their variants (LSTM: long-short term memory, GRUs: gated recurrent

units) could model sequential data for tasks such as speech recognition [33]. However, RNNs struggled with long sequences due to 'vanishing gradients' where gradients increasingly become smaller the 'deeper' (more layers) a network has in the backpropagation algorithm. This limits their effectiveness for tasks over long ranges.

The Transformer Revolution (2017–2020)

The transformer architecture introduced by Vaswani et al. (2017) replaced recurrence with self-attention, which allowed modelling of long-range dependencies and parallel processing of sequences[84]. This architecture became the foundation for nearly all subsequent advances in language and multimodal AI [7]. Language models leveraged the transformer through self-supervised pre-training on massive text corpora, followed by task-specific fine-tuning.

BERT (2018) demonstrated that bidirectional (from both future and past) context could be captured by masking words in the input and training to predict them, achieving state-of-the-art results across a diverse set of natural language processing benchmarks [22]. GPT-2 (2019) showed that language models trained on web text could perform diverse tasks without task-specific training [65].

2.1.3 AI 3.0: The Era of Large Language Models (2020–Present)

GPT-3 (2020) demonstrated that scaling language models to 175 billion parameters unlocked new capabilities, including few-shot learning (see 2.1.4) across hundreds of tasks without model changes [9, 7]. Commercial development lead to ChatGPT (2022) by combining GPT-3.5 with reinforcement learning from human feedback (RLHF) for conversational interaction [59, 62]. Claude, Gemini, and Llama models followed, each advancing capabilities in reasoning, safety, and efficiency [3, 81, 82].

Simultaneously, was the development of vision transformers. ViT (2020) applied the transformer architecture to image patches, achieving performance alongside convolutional networks [24]. CLIP (2021) learned joint vision-language representations by training on 400 million image-text pairs from the internet, enabling zero-shot (without seeing examples beforehand) image classification [64].

Combining these yielded vision-language models that could reason about images and video through natural language. GPT-4 (2023) and open-source efforts like LLaVA combined visual encoders with LLMs to create multimodal systems [60, 48]. Specialist models then emerged for video understanding, 3D scene interpretation, and embodied AI [97, 26].

2.1.4 Large Language Models: Training and Using them

It is clear now that Large Language Models represent a paradigm shift in natural language processing and moreover enable machines to understand and generate human-like text at unprecedented scale. At their core, LLMs are neural networks trained on vast corpora of text using the transformer architecture [84, 9, 7]. The architecture is in a two part encoder-decoder structure: the encoder processes the input sequence and produces a sequence of hidden representations, while the decoder generates output tokens autoregressively (see 2.1.4). Key components include multi-head attention, which allows the model to attend (perform attention on) to different positions simultaneously (see Figure 2.1), and feed-forward layers that apply non-linear transformations to the attention outputs (see Figure 2.2). Residual connections (allowing information to bypass a layer) and layer normalisation (making features have mean 0 and variance 1) stabilise training in deep networks [84].

Let us now cover some key concepts in LLMs related to this project:

Tokens

Tokens form the fundamental units of text processing in LLMs. Before processing, the model segments text into a sequence of tokens: subword units that balance vocabulary size (total number of unique tokens) with representational expressiveness [1] (see Figure 2.3). Common tokenisation schemes include Byte-Pair Encoding (BPE) [76] and WordPiece [75], which split rare words into multiple tokens while representing frequent words as single tokens. For example, the word "hallucination" might become three tokens: "hallu", "cination", and an end-of-word marker. The model's vocabulary typically contains

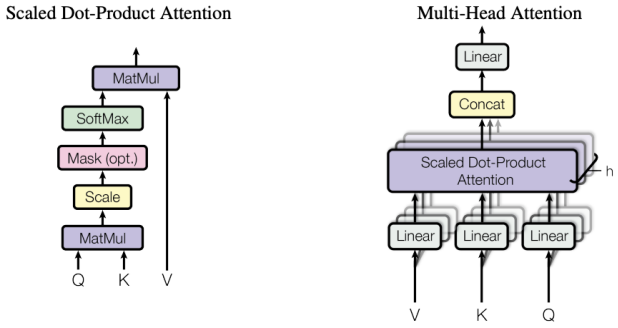


Figure 2.1: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel. Reprinted from Vaswani et al. [84].

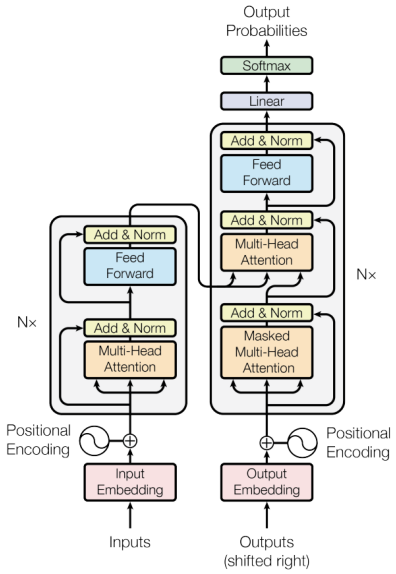


Figure 2.2: Transformer architecture showing encoder-decoder structure with multi-head attention, feed-forward networks, and residual connections. Reprinted from Vaswani et al. [84].

between 30,000 and 100,000 unique tokens, and input or output must be converted into this (sequence of) token representation [9, 22].



Figure 2.3: Tokenisation example showing how text is split into subword tokens. Reprinted from 3Blue1Brown [1].

Autoregressive generation

LLMs work by a process called autoregressive generation where they predict the next token given all previously generated tokens - this typically results in coherent and contextually appropriate text [84, 9]. During inference (when generating text), the model samples from a probability distribution over it's token vocabulary at each step, conditioned on the entire context (all previous tokens). This can be thought of as taking the most likely token to come next based on all previous context. This introduces computational overhead proportional to the sequence length, as each token depends on all preceding tokens [9].

Instruction Tuning

Instruction tuning fine-tunes pre-trained LLMs on instruction-response pairs, training model's behaviour to answer as a human intends [61, 12]. Instruction tuning uses curated datasets where inputs are explicit instructions (e.g., "Explain quantum entanglement") and outputs are desired responses. Retraining a model initially trained for next-token prediction this way turns it into an instruction-following assistant. Techniques such as LoRA [34] enable (parameter-)efficient fine-tuning by learning low-rank adaptations to the original weights, reducing computational requirements while preserving performance.

LoRA (Low-Rank Adaptation)

LoRA is a parameter-efficient fine-tuning technique that reduces the number of trainable parameters by decomposing weight updates into low-rank matrices [34]. Consider a linear layer with weight matrix $W \in \mathbb{R}^{d\times k}$, where d is the output dimension and k is the input dimension. In a standard transformer layer, the forward pass is $h = Wx$, where $x \in \mathbb{R}^k$ is the input vector and $h \in \mathbb{R}^d$ is the output activation. Rather than updating the full weight matrix W during fine-tuning, LoRA introduces a low-rank update using two trainable matrices: $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, where r is the adaptation rank and is chosen such that $r \ll \min(d, k)$. The modified forward pass becomes $h = Wx + BAx$, where the original pretrained weights W remain frozen and only the low-rank matrices A and B are trained. This dramatically reduces memory and compute requirements: for a rank-$r = 64$ adaptation on a layer with $d = 4096$, the number of trainable parameters is reduced by approximately 64× compared to full fine-tuning. LoRA is widely used to adapt large models to downstream tasks without requiring full-parameter updates.

Context Windows

The context window defines the maximum number of tokens a model can process in a single forward pass, representing the "memory" available during generation. Modern large language models typically support context windows ranging from 4,096 tokens (approximately 3,000 words) to over 200,000 tokens [2, 9, 60]. This limit constrains both the input prompt, context and the generated output; text exceeding the context window must be truncated or processed in chunks. Consequently, models may "forget" information from the beginning of long conversations, as earlier tokens fall outside the context [9].

Few Shot Learning

Few-shot learning describes the ability to perform new tasks given only a small number of examples in the prompt, importantly: without any updates to the model [9, 7]. This contrasts traditional machine learning, which requires training on thousands of examples. Few-shot learning works by providing task examples within the model's context window, allowing it to infer the pattern without updating the model. Similarly, zero-shot describes the same above with no examples.

Vector Embeddings

Vector embeddings transform tokens into numerical vectors that capture semantic meaning in a high dimensional continuous space [53, 84, 7, 1] (see Figure 2.4). Each token is mapped to a high-dimensional vector (typically 1,000 to 8,000 dimensions for modern LLMs) such that tokens with similar meanings occupy nearby regions of the embedding space. The layer that performs this embedding is a learned component that converts token IDs (integers) into these continuous representations. These are then processed by the transformer's attention layers. The quality of these embeddings directly influences the model's ability to understand relationships between words and concepts.

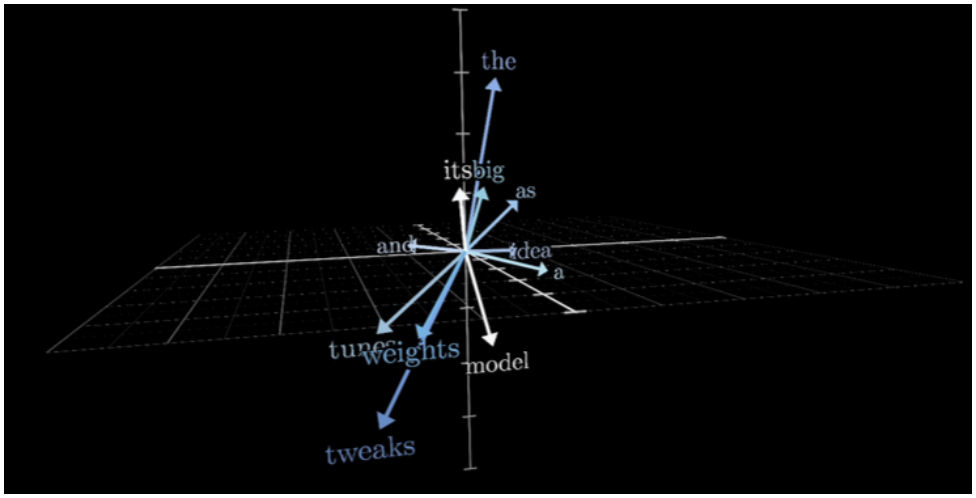


Figure 2.4: Vector embeddings visualising semantic relationships between tokens, simplified to 3D space. Reprinted from 3Blue1Brown (2024).

Prompts and Prompt Engineering

Prompts are the textual inputs that guide LLM behaviour, functioning as instructions or context that shape the model's output [9, 61, 12]. A prompt may consist of a direct question, formatted instructions or specifying the desired response style. Prompt engineering involves crafting these inputs to produce optimal responses, often including examples (few-shot prompting) or detailed task descriptions.

Temperature

Temperature controls the randomness of token selection during generation by scaling the values before the final selection (a softmax operation that gives us a probability distribution over the model's vocabulary given the values of the layer before) [9]. A temperature of 0 produces deterministic outputs by always selecting the highest-probability token (greedy decoding), while higher temperatures introduce greater variability by redistributing probability mass across less likely tokens. A temperature of 1.0 uses the unmodified softmax distribution; values below 1.0 make distributions sharper (more deterministic), while values above 1.0 make them flatter (more random). Temperature allows us to control the trade off between coherence and creativity in outputs.

Hallucination

Hallucination refers to the phenomenon where LLMs generate text that appears confident but contains factual errors, nonsensical statements, or information unrelated to the input or request [36]. This is consistent with how we have described LLMs to work: trained to produce text that looks correct based on statistical patterns in their training data, not to verify factual accuracy. The model has no inherent mechanism to distinguish true information from learned patterns.

Learning Rate controls the how fast a model learns during the backpropagation algorithm during training. A higher learning rate allows faster convergence but risks overshooting optimal weights, while a lower rate provides stability but requires more iterations. Learning rate schedules such as cosine annealing reduce the learning rate gradually over training to enable finer convergence near the optimum.

Batch Size specifies the number of training samples processed before model weights are updated. Larger batches provide more accurate gradient estimates but require more memory. Gradient accumulation enables effective larger batch sizes by processing samples in smaller batches and accumulating gradients before applying updates.

Weight Decay adds a 'regularisation term' to the loss function (how 'wrong' the model was over a pass - which then drives the model's weight changes during backpropagation), discouraging large weight values that can lead to overfitting - where a model does not generalise well to unseen data.

Warmup gradually increases the learning rate from a small value to the target rate over a fixed number of steps, stabilising early training when gradients may be noisy.

Gradient Clipping caps gradient values at a maximum magnitude during backpropagation, preventing exploding gradients that can destabilise training, particularly prevelant in very deep architectures.

BF16 (Brain Floating Point) is a 16-bit floating point format with 8 bits for the exponent and 7 bits for the mantissa, providing greater dynamic range than FP16 while reducing memory and computational requirements compared to FP32.

Gradient Checkpointing trades compute for memory by recomputing intermediate activations during backpropagation rather than storing them during the forward pass, reducing memory footprint at the cost of additional forward pass computations.

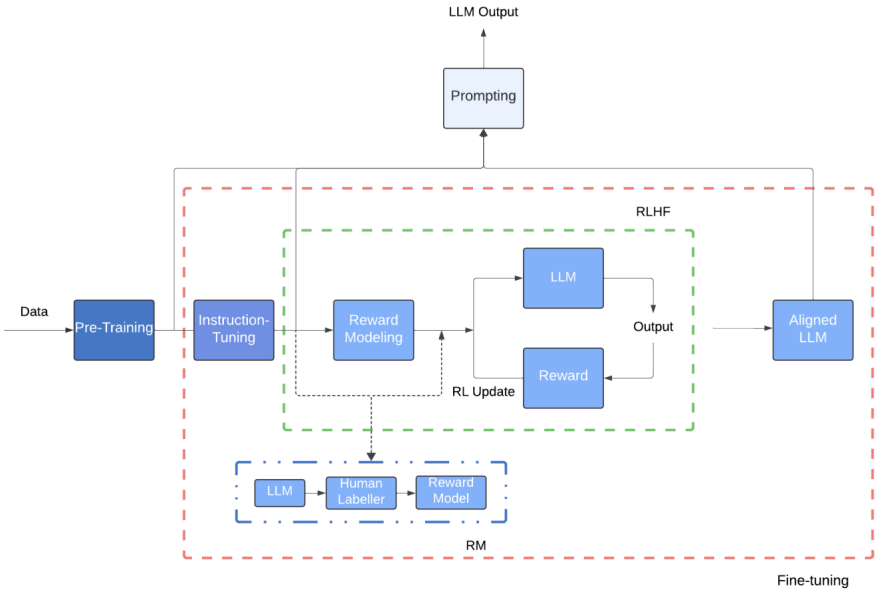


Figure 2.5: A basic flow diagram depicting various stages of LLMs from pre-training to prompting/utilisation. Prompting LLMs to generate responses is possible at different training stages like pre-training, instruction-tuning, or alignment tuning. "RL" stands for reinforcement learning, "RM" represents reward-modelling, and "RLHF" represents reinforcement learning with human feedback. Reprinted from Naveed et al. [56].

2.1.5 Vision-Language Models

Vision Encoders

Vision encoders extract meaningful representations from images and video frames, converting raw pixel data into dense vector embeddings that capture semantics. Modern vision encoders typically fall into two categories: convolutional neural networks (CNNs) or vision transformers (ViTs) as their architecture [24]. CNNs like ResNet process images through convolutional layers (ones that perform mathematical convolution on the pixels) that progressively abstract visual features: edges, textures and objects. Vision transformers instead split images into fixed-size patches, embed them as tokens, and process them with transformer self-attention, often achieving superior performance on large-scale datasets.

CLIP (Contrastive Language-Image Pre-training) trains on 400 million image-text pairs, learning to align visual features with their textual descriptions[64]. SigLIP improves upon CLIP with a sigmoid loss function that enables better scaling and simpler implementation [95]. EVA-CLIP extends this approach with masked image modelling pre-training and larger models, achieving state-of-the-art performance on vision benchmarks [80].

Extending Large Language Models with Vision

Vision-Language Models (VLMs) extend the capabilities of LLMs to process and understand visual information in conjunction with text. This enables tasks such as image captioning, visual question answering, and multimodal reasoning. Pre-trained vision encoders that were trained on massive image datasets provide powerful backbones for vision-language models.

Adding visual encoders to LLMs bridges the gap between visual and textual representations. Modern VLMs typically employ a pre-trained vision encoder—such as CLIP [64], SigLIP [95], or EVA-CLIP [80]—to extract visual features from input images or video frames. These visual features are then projected into the LLM's own embedding space through a vision-language projector, allowing the language model to reason over visual inputs as well as text (see Figure 2.6). This projection must align the distribution of visual features with the LLM's learned representations to enable coherent multimodal generation.

Vision-language alignment ensures that the visual and textual representations correspond meaningfully. Early approaches used simple linear projections or cross-attention mechanisms to fuse visual features into the language model [48]. More recent methods employ sophisticated alignment techniques such as contrastive learning at scale [64], which learns a joint embedding space where corresponding images and texts are pulled together while unrelated pairs are pushed apart. This alignment is critical for VLMs to understand the semantic relationship between visual content and natural language descriptions.

Following the success of instruction tuning for LLMs [62, 12], VLMs are fine-tuned on instruction-response pairs that involve images or videos paired with questions and answers. This process enables the model to perform diverse visual tasks specified through natural language instructions, such as "Describe what is happening in this video" or "Count the number of objects in the image." Datasets such as LLaVA's instruction tuning data [48] provide the supervision signal for this alignment process.



Figure 2.6: Vision-Language Model Architecture

2.2 Video Understanding with Vision-Language Models

Video understanding presents unique challenges beyond static image comprehension, requiring models to reason about motion, temporal dynamics, and causal relationships across frames. While image-based VLMs process individual frames independently, video understanding demands modelling of how scenes evolve over time, tracking objects through occlusions and deformation, and understanding actions that unfold across seconds or minutes.

Temporal modelling challenges constitute a fundamental obstacle in video VLMs. Objects and scenes change continuously across frames: lighting varies, objects move in and out of view, and the same action can be performed at different speeds. Unlike images where spatial relationships are fixed, videos require reasoning about when things happen and how they change. Transformers can theoretically capture long-range dependencies through self-attention, but the computational cost grows quadratically with sequence length, making processing of long videos with hundreds or thousands of frames prohibitively expensive [89]. Temporal information is also often compressed into brief frames—important events may occur between captured frames, requiring models to infer smooth trajectories from discrete samples.

The evolution from image-based to video-based VLMs proceeded through several stages of sophistication. Early approaches simply applied image VLMs to individual video frames and averaged or pooled the resulting representations, losing temporal information entirely [79]. Later methods introduced temporal modules such as 3D convolutions or temporal attention layers that could model frame-to-frame relationships [43]. Video-specific models like VideoBERT and CBT constructed joint embeddings between video clips and text descriptions, enabling zero-shot video classification and captioning. The emergence of large-scale video-text datasets enabled contrastive learning of video-language alignment at scale, similar to how CLIP revolutionised image-language alignment [90, 52].

Current limitations in long-term temporal reasoning persist despite significant progress. Most video VLMs process only short clips (typically 16-32 frames) due to context window and computational constraints, making it difficult to answer questions requiring understanding of hour-long videos or complex multi-step processes. The model must reason about temporal ordering ("did the person sit down before or after turning off the light?"), duration ("which action took longer?"), and causality ("what caused the window to break?"). Without explicit memory mechanisms, models struggle to relate information across distant frames, leading to errors in long-term tracking and narrative understanding. This limitation motivates research into memory-augmented video VLMs that can retain and retrieve information from earlier portions of long videos [50].

2.3 3D Reconstruction

3D reconstruction is the task of recovering the three-dimensional structure of a scene from one or more 2D observations such as images or video frames. Approaches have varied since it's inception from classical geometry-based pipelines to learned, end-to-end methods. They can be grouped into the following families: multi-view stereo and structure-from-motion, volumetric and implicit representations, depth estimation, and, most recently, feed-forward transformer-based methods.

Structure from Motion and Multi-View Stereo

Structure from Motion (SfM) is the classical approach to recovering camera poses and sparse scene geometry from a collection of images [73]. The method works by analysing features from overlapping views and camera parameters together with a sparse point cloud. COLMAP [74] is the dominant open-source SfM pipeline and remains a standard baseline for pose estimation. Multi-View Stereo (MVS) methods [27] build on SfM outputs to produce denser reconstructions by propagating depth estimates across views using photometric consistency. While these pipelines can produce high-quality results, they require significant overlap between views, are computationally expensive, are not suitable for real-time use, and struggle with textureless or reflective surfaces.

Volumetric and Implicit Representations

Neural Radiance Fields (NeRF) [54] represent a scene as a multi-layer perceptron (MLP) that maps a 3D location and viewing direction to colour and volume density, allowing the model to generate images from new viewpoints by simulating how light travels through the scene. NeRF produces photorealistic reconstructions but requires dense, calibrated input views and per-scene optimisation over many hours. Subsequent work has accelerated NeRF through explicit grid representations [55] and Gaussian splatting [38]. Gaussian splatting being a method in itself which represents the scene as a set of anisotropic (direction dependant) 3D Gaussians that can be rendered in real time via differentiable rasterisation (turning a 3D scene into 2D pixels with the use of gradient descent). These approaches can allow real time view but often still require offline processing per scene.

Monocular and Stereo Depth Estimation

Depth estimation methods predict per-pixel depth from a single image or a stereo pair, bypassing the need for multiple unordered views. Classical stereo matching exploits epipolar geometry and photometric consistency [32], while monocular depth estimation is an inherently ill-posed problem that learned methods address by exploiting learned scene priors. DPT [67] and MiDaS [68] demonstrated that vision transformer backbones trained on diverse data produce affine-invariant depth maps that generalise across domains. More recently, Depth Anything [93] scaled monocular depth training with large amounts of unlabelled data to achieve strong zero-shot generalisation. While these methods produce dense depth per frame, they do not inherently produce a globally consistent 3D reconstruction across frames.

Feed-Forward Multi-View Reconstruction

A newer family of methods replaces iterative optimisation with direct feed-forward prediction using transformer architectures trained on large 3D datasets. DUSt3R [86] jointly predicts point maps for pairs of images in a shared coordinate frame using a cross-attention transformer; this approach enables reconstruction without known camera parameters. MASt3R [41] extended this with dense local feature matching, improving downstream pose estimation. These methods reduced the dependence on feature matching, but operated on static image pairs or small sets of unordered images rather than streaming video.

Online and Recurrent Reconstruction

The most recent direction targets online reconstruction from streaming video without global optimisation. CUT3R (Continuous Updating Transformer for 3D Reconstruction) [85] is a representative example of this approach (see Figure 2.7). CUT3R extends the DUSt3R paradigm to the recurrent, online setting: a Vision Transformer (ViT) encoder converts each incoming frame into token representations, which interact with a persistent state via joint state update and readout modules. The state captures accumulated 3D

understanding of the scene, allowing the model to produce pointmaps and camera parameters for each new observation without re-processing previous frames or running bundle adjustment. A notable capability is the ability to query the persistent state from virtual viewpoints to infer unseen structure, effectively hallucinating geometry from positions never directly observed. CUT3R handles variable-length image sequences, both ordered video and unordered collections, and supports static and dynamic scene content.

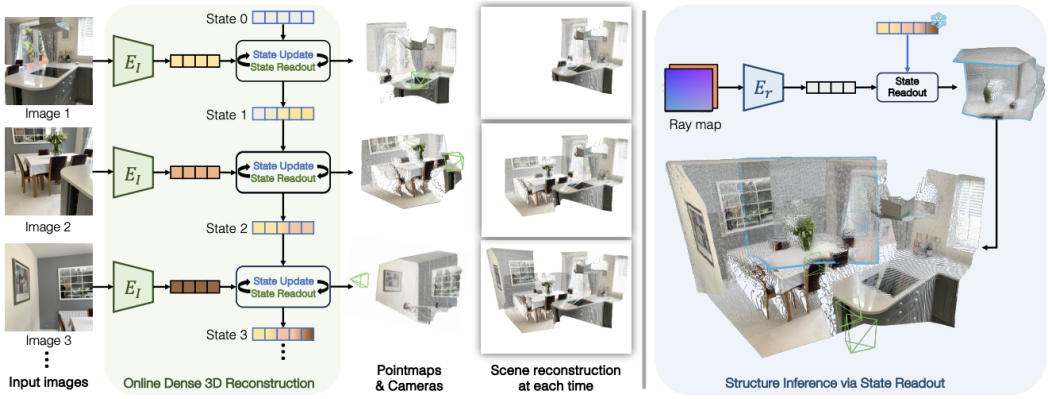


Figure 2.7: CUT3R method overview. The model processes input images via a shared-weight ViT encoder, producing visual tokens that interact with state tokens through simultaneous state update and readout modules. State update integrates current observations into the persistent state, while state readout retrieves past context for generating pointmaps and camera transforms. The method also supports predicting unseen views by querying the state with a raymap. Reprinted from Wang et al. [85].

2.4 3D Scene Understanding from Monocular Video

Understanding 3D space from monocular video (a sequence of 2D images captured by a single moving camera) represents a fundamental challenge in computer vision with critical applications in robotics, autonomous driving, and augmented reality. Unlike systems equipped with LiDAR or stereo cameras, monocular video requires inferring depth and 3D structure from subtle cues such as perspective, scale, object overlap, and camera motion.

Spatial reasoning in video requires more than the geometric reconstruction spoken about previously; models must understand functional relationships between objects and regions in 3D space. "Where is the cup relative to the table? Which drawer was opened first? Can I reach the book on the top shelf?" Spatial reasoning demands explicit or implicit representation of 3D geometry, object dimensions, and spatial relationships that remain consistent across camera movements and occlusions. Recent approaches combine geometric estimation with language grounding, enabling question answering about relative positions, distances, and spatial configurations [26].

Monocular depth estimation challenges arise because 2D images contain inherent scale ambiguity (a small object close to the camera appears identical to a large object far away). Without an external depth signal, models must rely on learned priors about typical object sizes, surface orientations, and scene compositions to resolve this ambiguity. Shadows, specular surfaces, transparent objects, and textureless regions also provide insufficient visual cues for reliable depth estimation. These limitations motivate approaches that combine monocular depth estimates with SLAM (Simultaneous Localisation and Mapping) or depth fusion techniques to maintain consistent scale and accumulate geometric information over time [99].

2.5 Memory Systems in Artificial Intelligence

Memory systems in AI address a fundamental limitation of standard neural networks: their lack of persistent state beyond the current input [30, 87]. While transformers can theoretically attend over long sequences, practical context windows remain finite [84]. Memory systems provide structured mechanisms

for storing, retrieving, and forgetting information [30].

Memory-augmented neural networks combine differentiable memory with neural network controllers that learn when to read, write, and forget [30, 87]. Neural Turing Machines and Memory Networks introduced differentiable access to external memory banks, enabling neural networks to store and retrieve arbitrary information with learned addressing schemes [30, 87]. The VLM^2 architecture introduced a dual-memory system specifically designed for video-based spatial reasoning in vision-language models [50]. Unlike general memory-augmented systems, VLM^2's memory is designed to accumulate 3D-aware representations across video frasmes. The system comprises two complementary memory components:

Working memory maintains buffers that store recent frame representations, enabling retrieval of information recetn context [50]. These buffers implement FIFO (First-In-First-Out) replacement policies, discarding older entries when capacity is reached [50].

Episodic memory stores experiences based on content similarity [50]. Unlike working memory's uniform treatment of entries, episodic memory implements selective storage based on novelty, retaining information that differs substantially from existing entries [50]. Cosine similarity-based replacement allows episodic memories to to cover a wide range relevant experiences[50].

2.6 Hardware in the Age of AI

The remarkable progress in artificial intelligence over the past decade has been enabled not only by algorithmic advances but by unprecedented developments in computing hardware [37]. Training large language models and vision-language models requires computational resources that would have been inconceivable two decades ago [37]. Modern GPUs and specialised accelerators provide the parallel processing capability necessary to train these models on billions of parameters within practical timeframes [15].

2.6.1 The Tensor Core

Tensor cores are specialised hardware units introduced by NVIDIA in their Volta architecture (2017) that dramatically accelerate matrix multiplication operations, which form the computational backbone of deep learning [14]. Unlike traditional CUDA cores that perform scalar operations sequentially, tensor cores operate on 4×4 matrices simultaneously, performing the operation $D = A \times B + C$ where A, B, C, and D are 4×4 matrices. During the training of neural networks, tensor cores accelerate the dense matrix multiplications that occur in both forward passes and backpropagation. For example, a single A100 tensor core can perform 256 FP16 FMA (fused multiply-add) operations per clock cycle, compared to just 2 operations for a standard CUDA core. This 128× throughput advantage per clock cycle translates to orders-of-magnitude speedups for the matrix-heavy workloads characteristic of LLM training [15].

Modern AI accelerators incorporate dedicated tensor core units alongside traditional CUDA cores. The NVIDIA H100 GPU contains 456 tensor cores capable of performing 1,979 TFLOPS (1.979 quadrillion floating point operations per second) on FP16 operations, compared to 67 TFLOPS for standard CUDA cores on FP32. This specialisation enables training runs that would take months on CPU-based systems to complete in hours or days on GPU clusters [16].

2.6.2 High Performance Computers

High Performance Computing (HPC) systems aggregate thousands of compute accelerators into interconnected clusters capable of performing exascale computations (10^{18} floating-point operations per second). Modern AI training facilities represent a specialised subset of HPC optimised for the communication patterns and memory access characteristics of deep learning workloads. [78, 23]

Modern AI training clusters comprise thousands of GPUs organised into hierarchical network topologies. The training of GPT-3 on 10,000 A100 GPUs required approximately 34 days, consuming 1.3 million GPU-hours [9]. Training of larger models such as Gopher required 64 TPUv3 (Google's answer to the NVIDIA GPUs) pods running for 11 days [66]. These resource requirements highlight why access to specialised HPC infrastructure represents a bottleneck for frontier AI research.

Energy consumption represents a critical constraint for large-scale AI training. The H100 GPU consumes 700W under load [16], and large GPU clusters can require tens of megawatts. This has motivated research into more efficient training algorithms, sparse architectures, and specialised low-precision formats that reduce the computational and energy requirements of training.

2.6.3 SLURM

SLURM (Simple Linux Utility for Resource Management) is an open-source, fault-tolerant, and highly scalable cluster management and job scheduling system for Linux clusters [72]. It manages workloads by allocating resources, executing jobs, and monitoring tasks, while providing fair-share scheduling to optimise cluster utilisation.

To utilise the high performance hardware on Isambard AI Phase 2 or Blue Pebble 1, we must submit work as ‘jobs‘ in SLURM. This requires writing SLURM compatible bash scripts outlying the hardware we wish to use ontop of commands needed to run out codes, load environments and where to write logs.

This comes with some notable consequences to consider for this project: we may have to wait in the ‘queue‘ for hours or days whilst requested resources are being consumed by other users or other ‘jobs‘ have higher priority that ours. SLURM is typically also responsible for enforcing user account compute limits - stopping users from submitting jobs that request resources beyond their credit limit.

Chapter 3

Methods

3.1 Method Overview

Our approach builds upon the state-of-the-art VLM-3R architecture [26] and augments it with two complementary mechanisms. First, we adopt the dual-memory design from VLM^2 [50], which maintains working memory for immediate context and episodic memory for historical information, enabling sustained spatial reasoning across video frames. Second, we incorporate query-based frame selection inspired by VideoStreaming [63], which filters frames for relevance to the spatial reasoning question at hand, reducing redundancy and improving precision.

3.2 VLM-3R

VLM-3R (Vision-Language Models Augmented with Instruction-Aligned 3D Reconstruction) is a unified framework for Vision-Language Models that incorporates 3D reconstructive instruction tuning, enabling deep spatial understanding from monocular video without requiring depth sensors or pre-built 3D maps [26]. The architecture extends LLaVA-NeXT-Video [47] by adding a geometry encoder (CUT3R) and a novel Spatial-Visual-View Fusion mechanism that integrates 3D geometric tokens with the LLM's native visual tokens.

3.2.1 Architecture Overview

VLM-3R takes monocular video frames $\{I_t\}_{t=1}^{N}$ and language instructions as input. The model comprises four main components: a visual encoder (SigLIP-SO400M [95]) that extracts 2D appearance features, a spatial encoder (CUT3R [85]) that produces 3D reconstructive tokens, a Spatial-Visual-View Fusion block that integrates these representations [26], and a language model (LLaVA-Video-7B-Qwen2 [47, 5]) that generates text responses (see Figure 3.1). During training, the visual encoder and spatial encoder; only the fusion block and multimodal projector are fine-tuned, with LoRA applied to the language model [26, 34].

3.2.2 3D Reconstructive Tokenization

VLM-3R adopts the pre-trained CUT3R model as its spatial encoder for 3D reconstructive tokenization [85, 26]. CUT3R processes monocular video frame-by-frame using a recurrent architecture. For each frame I_t, the SigLIP [95] visual encoder extracts feature tokens F_t:

$$F_t = \text{SigLIP}(I_t), \tag{3.1}$$

which are then processed together with the recurrent state by the CUT3R spatial encoder [85, 26]. These tokens are then processed together with a learnable pose query token z and the previous recurrent state s_{t-1} by a transformer decoder f_{dec}:

$$[z'_t, F'_t], s_t = f_{\text{dec}}([z, F_t], s_{t-1}). \tag{3.2}$$

Here, s_t is the updated recurrent state, z'_t denotes pose-related camera-view tokens, and F'_t denotes enriched spatial feature tokens. Prediction heads can use these representations to estimate metric-scale

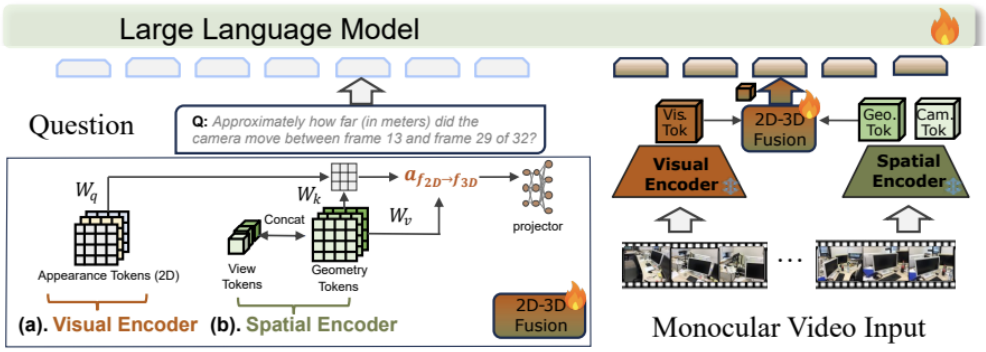


Figure 3.1: Network architecture of VLM-3R. The model takes monocular video and language instructions as input, processing them through a visual encoder (SigLIP) and spatial encoder (CUT3R). A Visual-Geometry Fusion block integrates 2D appearance features with 3D geometric tokens, enabling the language model (Qwen2) to perform spatial and temporal reasoning. Reprinted from Fan et al. (2025) [26].

point maps and relative camera pose, but VLM-3R fuses the implicit latent tokens rather than explicit point clouds [26]. To ensure a consistent token count, input images are resized to 432×432 pixels. During VLM-3R training, the pre-trained visual encoder and CUT3R-based spatial encoder are frozen, while the fusion/alignment components and LoRA adapters are trained [26, 34].

3.2.3 Spatial-Visual-View Fusion

The Spatial-Visual-View Fusion block integrates CUT3R-derived 3D tokens with the VLM's native visual tokens H_v extracted by the pretrained visual encoder [26]. The unified 3D representation is formed by concatenating geometry tokens and camera-view tokens:

$$Z_{\text{3D}} = \text{Concat}(F'_t, z'_t). \tag{3.3}$$

A cross-attention operation then performs 2D-3D feature fusion:

$$H_{\text{attn}} = \text{CrossAttention}(Q = H_v,\ K = Z_{\text{3D}},\ V = Z_{\text{3D}}). \tag{3.4}$$

The fused representation is combined with the original visual features through a residual connection:

$$H'_v = H_v + H_{\text{attn}}. \tag{3.5}$$

tFinally, a two-layer MLP projector maps the fused representations into the LLM embedding space [26]. This fusion mechanism enables VLM-3R to combine semantic visual understanding with implicit 3D spatial awareness derived from CUT3R [26, 85].

3.3 Our Architecture

The VLM-3R architecture (see 3.2) is augmented with a dual-memory module inspired by the VLM^2 architecture [50] and a query based frame selection mechanism inspired by the VideoStreaming architecture [63]. Both of which are positioned between VLM-3R's multimodal projector and the language model (see Figure 3.2), receiving 3D-aware visual features H_t from the Spatial-Visual-View Fusion block and producing memory-enhanced representations M_t for autoregressive generation.

The dual memory implementation comprises three core components. Working memory maintains a FIFO sliding window of recent frame features. Episodic memory stores salient long-term representations using cosine similarity to replace the most similar entry when full. We then orchestrate retrieval from both memories via attention-based queries, then fuse the retrieved memories using a learnable MLP. The fused output is fed back to update memory for the next timestep.

Two versions are implemented: a non-trainable baseline using fixed attention and equal-weight fusion,

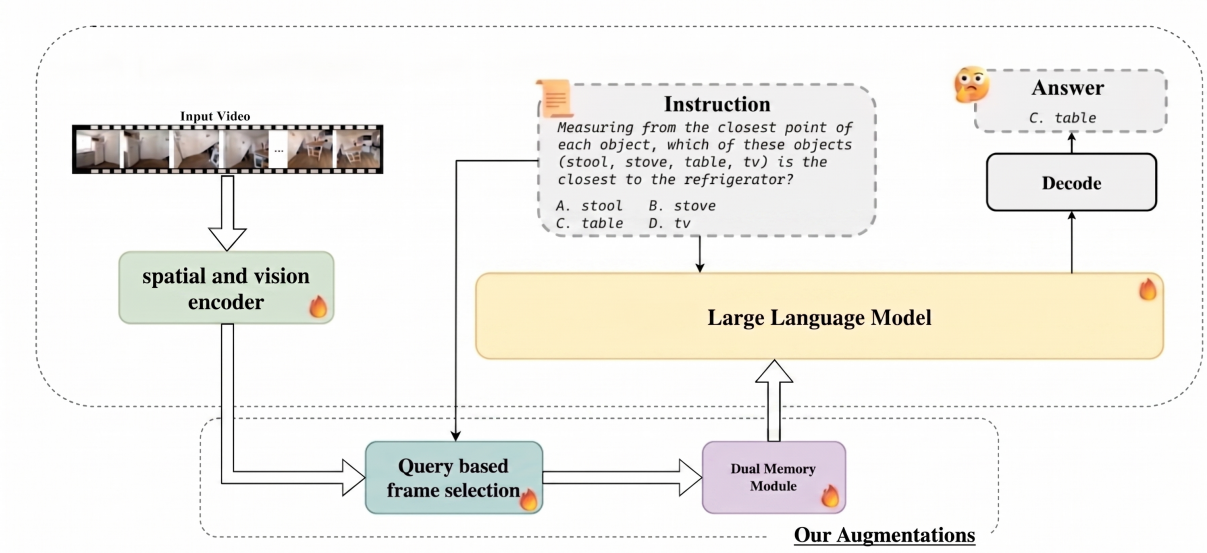


Figure 3.2: Our architecture with query based frame selection and dual-memory module inserted between the multimodal projector and LLM of the VLM-3R model.

and a trainable version where attention projections and the fusion MLP learn from data. Detailed algorithms, equations, and training considerations are presented in the following sections.

Following the VideoStreaming approach [63], we implement query-based frame selection to address the redundancy problem in naive video processing. Rather than processing all frames uniformly, our implementation uses the input question to identify question-relevant frames while discarding redundant information. This approach is motivated by the observation that not all frames are equally relevant to a given spatial reasoning question [63].

The frame selection mechanism operates as follows: for each input question, the system computes relevance scores between the query and all stored frame representations, then selects a constant number of the most relevant frames to pass to the language model for reasoning. This selection is performed by computing cosine similarity between the query embedding and each frame's visual features, followed by a softmax to obtain a probability distribution over frames. The top-k frames with highest probabilities are then selected for processing. By conditioning frame selection on the specific question being asked, the system ensures that only information pertinent to the current query contributes to the response, reducing noise and improving precision in spatial reasoning tasks [63].

This query-based selection is orthogonal to and complements the dual-memory module: frame selection filters which visual information enters the processing pipeline, while the dual-memory module manages persistent state across video frames. Together, they should enable efficient and precise video understanding by both reducing redundancy at the frame level and maintaining coherent temporal understanding at the memory level [63].

3.4 The Dual Memory Module

VLM^2 (Vision-Language Memory for Spatial Reasoning) addresses two fundamental limitations of standard VLMs: the absence of persistent memory to retain information across video frames, and the semantic-geometric misalignment that prevents consistent 3D understanding [50]. The proposed dual-memory module enables efficient spatial reasoning with fixed computational cost by maintaining two complementary memory systems that operate over video sequences [50] (see Figure 3.3). Our model utilises this dual memory idea as per Figure 3.2.

Figure 3.3 shows the structure of the dual memory system as per the VLM^2 model [50]. Where input features are passed to both working and episodic memory before being fused. These fused features are passed to the model and used to update the episodic memory. In the next sections we discuss our

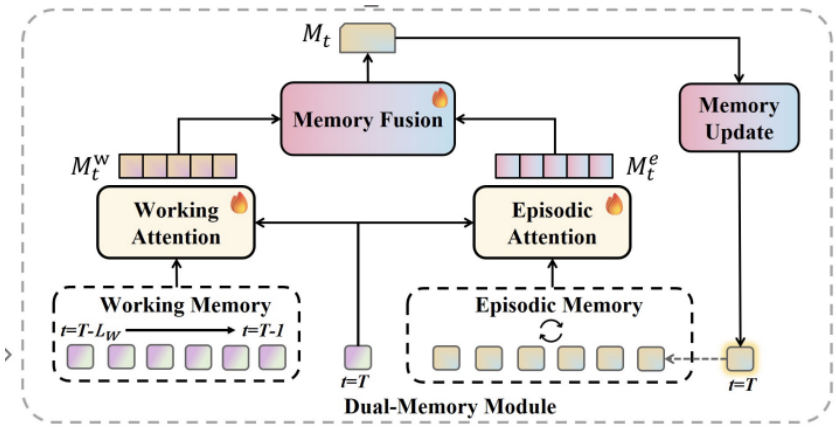


Figure 3.3: The dual memory module consists of 3 parts: working memory, episodic memory and fusion. Reprinted from Liu et al. [50].

approach using the dual memory module ideas from VLM^2 to augment the VLM-3R module with a memory module. We try both a trainable and untrainable approach.

3.4.1 Difference from VLM^2

Our implementation adapts the dual-memory idea from VLM^2 to the VLM-3R architecture.

Feature Dimension. Our memory module operates at `feature_dim=1152`, matching the feature dimension used at the VLM-3R fusion interface in our implementation. This differs from VLM^2, whose memory module is integrated into a different 3D-aware video representation pipeline built on LLaVA-Video-7B and π^3.

Vision and Spatial Tower. Our memory module is designed for the VLM-3R architecture, which differs from VLM^2 primarily in the choice of 3D foundation model and fusion pipeline:

- **Visual Encoder:** First note that both VLM-3R and VLM^2 build on the LLaVA-Video/LLaVA-NeXT-Video family and use a SigLIP-based visual encoder in their reported implementations [26, 50].
- **3D Foundation Model:** VLM-3R adopts CUT3R as its spatial encoder, producing implicit 3D reconstructive tokens consisting of enriched spatial tokens and camera-view tokens [26, 85]. VLM^2 instead uses π^3 as its 3D foundation model and constructs 3D-aware representations through adaptive 3D position injection, viewpoint-aware geometry alignment, and semantic-geometric fusion [50].
- **Fusion Strategy:** VLM-3R concatenates CUT3R-derived spatial and camera-view tokens and fuses them with native visual tokens through Spatial-Visual-View Fusion, using cross-attention followed by a projector into the language-model embedding space [26]. VLM^2 instead first injects predicted 3D coordinates into visual tokens using an adaptive gate, aligns geometry tokens with view tokens, and then applies cross-attention between position-aware visual tokens and viewpoint-aware geometry tokens [50].

Non-Trainable Memory. We also implement a non-trainable memory variant to evaluate whether explicit trainable parameters are necessary for improving temporal and spatial reasoning in this setting.

Further implementation specifics, including the exact attention head configurations, buffer sizes, and trainable component architecture, are detailed in Section 4.6.1.

3.5 Non-Trainable Memory

Algorithm 1 Dual-Memory Module (Non-Trainable)

Require: H_t, $\mathcal{W}_t$, $\mathcal{E}_t$, α
Ensure: M_t^{out}, $\mathcal{W}_{t+1}$, $\mathcal{E}_{t+1}$

▷ Working Retrieval (Non-Learnable)
$M_t^w \leftarrow \text{Softmax}\left(\frac{H_t \mathcal{W}_t^T}{\sqrt{d}}\right)\mathcal{W}_t$
▷ Episodic Retrieval (Non-Learnable)
$M_t^e \leftarrow \text{Softmax}\left(\frac{H_t \mathcal{E}_t^T}{\sqrt{d}}\right)\mathcal{E}_t$
▷ Fixed Memory Fusion
$M_t \leftarrow 0.5 \cdot M_t^w + 0.5 \cdot M_t^e$
▷ Residual Alpha Blending
$M_t^{\text{out}} \leftarrow (1-\alpha)\cdot H_t + \alpha \cdot M_t$
▷ Update Working Memory (FIFO)
if $|\mathcal{W}_t| < L_w$ **then**
  $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup \{H_t\}$
else
  remove oldest element from $\mathcal{W}_t$
  $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup \{H_t\}$
▷ Update Episodic Memory (Similarity-Based Replacement)
if $|\mathcal{E}_t| < L_e$ **then**
  $\mathcal{E}_{t+1} \leftarrow \mathcal{E}_t \cup \{M_t\}$
else
  for $i = 1$ **to** L_e **do**
    $s_i \leftarrow \cos(M_t, E_i)$
  $i_t^\star \leftarrow \arg\max_{i \in \{1,\ldots,L_e\}} s_i$
  del $E_{i_t^\star}$; $\mathcal{E}_{t+1} \leftarrow \mathcal{E}_t \cup \{M_t\}$
return M_t^{out}, $\mathcal{W}_{t+1}$, $\mathcal{E}_{t+1}$

The non-trainable memory implementation provides a baseline evaluation of the dual-memory module's effectiveness without requiring additional learned parameters. This approach follows Algorithm 1, adapted from the VLM^2 paper [50] but substitutes the learnable components with fixed, hand-crafted alternatives and can be seen in Algorithm 1.

3.5.1 Working Memory

Working memory maintains a fixed-capacity FIFO (First-In-First-Out) sliding window of recent frame features [50]. The buffer stores up to L_w frame representations, discarding the oldest element when capacity is reached:

$$\mathcal{W}_{t+1} = \begin{cases} \mathcal{W}_t \cup \{H_t\} & \text{if } |\mathcal{W}_t| < L_w \\ (\mathcal{W}_t \setminus \{\text{oldest}\}) \cup \{H_t\} & \text{if } |\mathcal{W}_t| = L_w \end{cases} \tag{3.6}$$

This implementation employs a simple cross-attention function with fixed, unlearned attention weights. At each timestep t, the current frame features H_t are attended over the working memory buffer W_t using standard scaled dot-product attention:

$$M_t^w = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3.7}$$

where $Q = H_t$, $K = V = \mathcal{W}_t$, and d_k is the feature dimension.. This retrieval mechanism allows the model to condition on recent frames without introducing any trainable parameters.

3.5.2 Episodic Memory

Episodic memory implements a similarity-based replacement mechanism to store latent long-term information [50]. With a capacity of L_e elements, the memory stores frame representations that survive

the FIFO working memory window. When episodic memory is full, incoming features replace the most similar existing element rather than being discarded. This is computed using cosine similarity:

$$\text{sim}(M_t, E_j) = \frac{M_t \cdot E_j}{\|M_t\|\|E_j\|} \tag{3.8}$$

The replacement target is $j^* = \arg\max_j \text{sim}(M_t, E_j)$.

3.5.3 Memory Fusion

Fusion combines the retrieved working memory M_t^w and episodic memory M_t^e outputs into a unified memory representation M_t. Rather than using a learnable gated MLP as specified in the original algorithm, the non-trainable version employs a simple fixed weighting:

$$M_t = 0.5 \cdot M_t^w + 0.5 \cdot M_t^e \tag{3.9}$$

This equal weighting avoids introducing trainable parameters while providing a baseline to compare against the learnable fusion mechanism. The fused memory M_t is then used for downstream reasoning while simultaneously updating both memory buffers for the next timestep.

3.5.4 Residual Connection and Alpha Blending

Motivation

The non-trainable implementation employs a residual connection to blend the memory output with the original input features H_t, controlled by a fixed α parameter. This ensures that even when memory retrieval is uninformative (e.g., empty buffers at initialization), the model can still fall back to the original features.

Implementation

The residual connection is implemented as:

$$M_t^{\text{out}} = (1 - \alpha) \cdot H_t + \alpha \cdot M_t \tag{3.10}$$

For the fusion mode where

$$M_t = 0.5 \cdot M_t^w + 0.5 \cdot M_t^e, \tag{3.11}$$

this becomes:

$$M_t^{\text{out}} = (1 - \alpha) \cdot H_t + \alpha \cdot (0.5 \cdot M_t^w + 0.5 \cdot M_t^e) \tag{3.12}$$

This residual connection prevents information degradation when memory is initially empty or retrieval is noisy, providing a way to better benchmark the memory system without introducing trainable parameters. For single-memory modes (working-only or episodic-only), the formula simplifies to:

$$M_t^{\text{out}} = (1 - \alpha) \cdot H_t + \alpha \cdot M_t^{\text{single}} \tag{3.13}$$

3.6 Trainable Memory

The trainable memory implementation extends the non-trainable baseline by introducing learnable parameters that enable the model to adapt memory retrieval and fusion to the specific requirements of spatial reasoning tasks. Following Algorithm 2 reprinted from the VLM^2 paper [50], the trainable version implements the full architecture with learned attention mechanisms and a fusion module.

Algorithm 2 Dual-Memory Module

Require: H_t, $\mathcal{W}_t$, $\mathcal{E}_t$
Ensure: M_t, $\mathcal{W}_{t+1}$, $\mathcal{E}_{t+1}$
▷ Working Retrieval
$M_t^w \leftarrow$ Working Attention$(Q = H_t, KV = \mathcal{W}_t)$
▷ Episodic Retrieval
$M_t^e \leftarrow$ Episodic Attention$(Q = H_t, KV = \mathcal{E}_t)$
▷ Gated Memory Fusion
$\gamma_t = \sigma(\text{MLP}(\text{Concat}[M_t^w; M_t^e]))$
$M_t = \gamma_t \odot M_t^w + (1 - \gamma_t) \odot M_t^e$
▷ Update Working Memory
if $|\mathcal{W}_t| < L_w$ **then**
  $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup \{H_t\}$
else
  remove oldest element from $\mathcal{W}_t$
  $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup \{H_t\}$
▷ Update Episodic Memory
if $|\mathcal{E}_t| < L_e$ **then**
  $\mathcal{E}_{t+1} \leftarrow \mathcal{E}_t \cup \{M_t\}$
else
  for $i = 1$ **to** L_e **do**
    $s_i \leftarrow \cos(M_t, E_i)$
  $i_t^\star \leftarrow \arg\max_{i \in \{1,\dots,L_e\}} s_i$
  del $E_{i_t^\star}$; $\mathcal{E}_{t+1} \leftarrow \mathcal{E}_t \cup \{M_t\}$
return M_t, $\mathcal{W}_{t+1}$, $\mathcal{E}_{t+1}$

3.6.1 Working Memory

Working memory replacement stay the same as the non trainable variant. Retrieval, however changes from the non trainable variant. Retrieval from working memory employs cross-attention, where the current representation H_t serves as the query attending over all entries in the buffer [50]:

$$M_t^w = \text{Attn}(Q(H_t),\, K(\mathcal{W}_t),\, V(\mathcal{W}_t)) \tag{3.14}$$

The resulting feature M_t^w is an enhanced representation of the current state, enriched with the most relevant context from recent observations [50].

3.6.2 Episodic Memory

Episodic memory only changes from the non trainable version as follows. Retrieval from episodic memory follows the same cross-attention mechanism as working memory, with H_t as the query [50]:

$$M_t^e = \text{Attn}(Q(H_t),\, K(\mathcal{E}_t),\, V(\mathcal{E}_t)) \tag{3.15}$$

3.6.3 Gated Memory Fusion

The fusion mechanism combines outputs from both memory streams through a learnable gate γ_t that controls the relative contribution of working and episodic memory [50]:

$$\gamma_t = \sigma(\text{MLP}(\text{Concat}[M_t^w;\, M_t^e])) \tag{3.16}$$

The fused memory-enhanced representation is then computed as [50]:

$$M_t = \gamma_t \odot M_t^w + (1 - \gamma_t) \odot M_t^e \tag{3.17}$$

where $\odot$ denotes element-wise multiplication. This architecture allows the model to adaptively balance immediate context (via working memory) against accumulated long-term information (via episodic memory) [50]. The memory-enhanced representations M_t are subsequently stored into episodic memory and, together with the instruction embeddings, fed into the language backbone to generate answers [50].

3.6.4 Memory-Augmented Reasoning

The complete pipeline at each timestep t proceeds as follows [50]: (1) current hidden state h_t is computed from the vision-language model; (2) h_t is stored in both working and episodic memory according to their respective update rules; (3) attention-based retrieval extracts relevant information w_t and e_t from both memories; (4) the fusion gate γ_t and MLP produce the final memory-augmented representation h'_t. This design achieves long-horizon reasoning with fixed memory capacity and constant computational overhead per timestep, regardless of video length [50].

3.7 Query-based Frame Selection

Following the VideoStreaming approach [63], we implement query-based frame selection to address both the redundancy problem and the possibility of missing relevant latent information in naive video processing. Rather than processing all frames uniformly, this approach uses the input question to identify question-relevant frames while discarding redundant information. This mechanism is orthogonal to and complements the dual-memory module described previously: frame selection filters which visual information enters the processing pipeline at the input level, while the dual-memory module manages persistent state across video frames at the processing level. Together, they enable efficient video understanding by both reducing redundancy at the frame level and maintaining coherent temporal understanding through memory [63].

Standard video understanding models process all frames equally, regardless of their relevance to the specific question being asked. This leads to redundant information processing and reduced precision in spatial reasoning tasks [63]. The VideoStreaming paper demonstrated that adaptive query-based selection could identify question-relevant information while discarding redundant information, enabling more efficient and precise video understanding. The key insight is that not all frames are equally relevant to a given spatial reasoning question [63], and filtering for relevance improves both efficiency and accuracy.

Our query-based frame selection module is inspired by the VideoStreaming Section 3.3 architecture [63], but is adapted to operate directly on frame-level representations rather than clip memories. The module comprises three learned MLP layers. The streaming encoder MLP projects from the vision encoder output dimension (after the multimodal projector) to an intermediate representation space, producing a single frame indicator $\hat{H}_k$ for each of the T frames. The question encoder MLP projects from the LLM dimension to the same intermediate representation space, producing the query representation H_Q. The LLM input MLP projects the selected frame indicators back to the LLM dimension for processing by the language model. Given T frame features and a question, the system computes cosine similarity between H_Q and each $\hat{H}_k$ to obtain relevance scores across all frames. Selection is performed using Gumbel-Topk [63], a differentiable straight-through estimator that enables gradient flow to the encoders during training while using hard top-k selection during inference. The top-V frames with highest relevance scores are selected, gathered, and passed through the LLM input MLP before being fed to the language model for answer generation. The complete selection process is shown in Algorithm 3.

3.7.1 Difference from VideoStreaming

Our query-based frame selection implementation adapts VideoStreaming's approach for VLM-3R and spatial reasoning tasks. The key differences are:

- **Projection Dimensions.** The VideoStreaming paper uses 4096-dimensional projections for both the streaming encoder and question encoder to match their LLM dimension. Our implementation adapts to VLM-3R's Qwen2 backbone with an LLM dimension of 3584, so the question encoder and LLM input projector project from 3584 rather than 4096; the intermediate dimension of 2560 is retained.
- **Clips vs Frames.** VideoStreaming first segments videos into 16-frame clips sampled at 1 fps, processes each clip with memory propagation from preceding clips, then applies query-based selection on the clip-level representations to select top-4 clips. We instead pass uniformly sampled frames

Algorithm 3 Query-Based Frame Selection

Require: $\{H_k\}_{k=1}^{T}$ (frame features), Q (question tokens), V (number to select), τ (temperature)
Ensure: $\{H_{i_j}\}_{j=1}^{V}$ (selected frame features)

```
▷ Encode Question
H_Q ← MeanPool_masked(Q)
H_Q ← QuestionEncoder(H_Q)
H_Q ← Normalize(H_Q)
▷ Encode Frames
for k = 1 to T do
    Ĥ_k ← StreamEncoder(H_k)
    Ĥ_k ← Normalize(Ĥ_k)
▷ Compute Relevance Scores
for k = 1 to T do
    s_k ← H_Q · Ĥ_k
▷ Select Top-V Frames (Gumbel-Topk)
if use_gumbel then
    for k = 1 to T do
        g_k ← −log(−log(u_k)) where u_k ~ U(0, 1)
        s̃_k ← (s_k + g_k)/τ
    {i_j}_{j=1}^V ← TopK({s̃_k}_{k=1}^T, V)
else
    {i_j}_{j=1}^V ← TopK({s_k}_{k=1}^T, V)
▷ Gather and Project Selected Frames
{H_{i_j}}_{j=1}^V ← Gather({H_k}, {i_j})
{H_{i_j}}_{j=1}^V ← LLMInputProjector({H_{i_j}})
return {H_{i_j}}_{j=1}^V
```

directly to the query-based frame selection mechanism, without clip-level pre-processing or memory propagation between clip segments.

- **Selection Ratio.** VideoStreaming's default 32-to-4 configuration selects 4 clips from a series of 16-frame clips. We adapt this to 128-to-32 frames, selecting 32 frames from up to 128 available frames—a ratio that better suits VLM-3R's performance on spatial reasoning with 32 uniformly sampled frames and we are not working with clips.

- **Integration Point.** VideoStreaming uses memory-propagated streaming encoding where encoded results of preceding clips serve as historical memory for the current clip. Our implementation operates on VLM-3R's spatial tower output, integrating after the Cross-Attention Fusion block rather than in a streaming clip-processing pipeline.

These adaptations reflect our focus on spatial reasoning tasks with VLM-3R's 3D-aware features rather than general video understanding. Further implementation specifics, including the Gumbel-Topk mechanism and training configuration, are detailed in Section 4.7.

3.8 Combined Architecture

The combined model uses query-based frame selection and the dual-memory module in concert. Query-based frame selection first identifies the V most question-relevant frames from the full sampled video sequence, reducing redundant visual information before memory processing. The selected frames are then processed sequentially by the dual-memory module, which maintains working and episodic memory across the reduced frame sequence. This architecture can be seen in Figure 3.2 and specific memory and query based approaches are exactly as spoken about in Section 3.6 and Section 3.7.

Chapter 4

Experiments and Results

4.1 Hardware

Isambard-AI

Isambard-AI is the UK's fastest AI supercomputer and one of Europe's most advanced AI-focused systems, built by the Bristol Centre for Supercomputing at the University of Bristol [58]. The system is built around the NVIDIA GH200 Grace Hopper Superchip, a specialised chip that combines a 72-core CPU (central processing unit) with a H100 GPU (graphics processing unit, the type of chip used for AI training). Phase 2 alone deploys 5280 of these superchips across 1,362 compute nodes [58].

At full capacity, Isambard-AI Phase 2 delivers over 200 PetaFLOP/s (petaflops: one quadrillion floating-point operations per second) at 64-bit precision for traditional scientific computing, scaling to over 21 ExaFLOP/s (exaflops: one quintillion operations per second) when operating at 8-bit precision for AI workloads [58].

Blue Pebble 1

"BluePebble a non-homogenous cluster, meaning that the nodes vary slightly. There are 19 generally available GPU enabled nodes: A mix of NVIDIA RTX2080Ti, NVIDIA RTX309 and NVIDIA V100 (typically each GPU node has 4 GPU cards and 96GB RAM)." These GPUs are not appropriate for AI jobs of the nature spoken about and worked on in this report. We are interested in the two MLCNU nodes which each contain 4 high memory A100s. Due to the low count of GPUs we should expect high wait times (see 2.6.3) to access these if we wish to run AI jobs on the MLCNU parition.

4.2 Data Preprocessing

The VLM-3R model requires preprocessing to convert raw 3D indoor scene datasets into a format suitable for training. This involves geometry extraction, video export, metadata generation, and automated QA pair creation.

4.2.1 ScanNet

ScanNet is a richly-annotated indoor scene dataset containing RGB-D video sequences captured with an Azure Kinect sensor, along with camera poses, 3D meshes, and semantic annotations [18]. Preprocessing for ScanNet is divided into phases. Each phase is executed via dedicated SLURM scripts on the HPC cluster.

Phase 1 (Raw Data Processing) comprises four preprocessing steps. Step 1.1 processes ScanNet200 meshes to extract labelled point clouds: the raw ScanNet scenes contain '_vh_clean_2.ply' mesh files that are converted into structured point clouds with semantic labels (200 categories) and instance IDs using the ScanNet BenchmarkScripts [18]. Step 1.2 exports RGB videos from .sens files: the raw sensor data contains individual frames that must be compiled into video format (24 FPS, 640x480 resolution) that

can then be resampled in the next step (although this seems redundant we setup our SLURM script to process in the exact way as VLM-3R to prevent unforeseen performance changes). Step 1.3 samples 32 frames per scene along with camera poses and intrinsics: VLM-3R processes a fixed number of frames per video (32 frames) to maintain consistent sequence length across the dataset, requiring uniform temporal sampling with corresponding camera parameters. Step 1.4 cleans up .sens files after successful processing to reclaim storage space (approximately 754 GB): a necessary step due to per user space constraints on the HPC clusters.

Phase 2 (Metadata Generation) generates scene-level and frame-level metadata required for QA pair generation. Step 2.1 extracts scene metadata including axis-aligned bounding boxes, object instance IDs, and spatial relationships between objects: this metadata encodes the 3D structure of each scene necessary for generating spatial reasoning questions (e.g., relative distances, object counts). Step 2.2 extracts frame-level metadata including camera pose at each sampled frame, depth maps, and object visibility: this links the 3D scene structure to specific video frames, enabling questions about temporal spatial relationships.

Phase 3 (QA Generation) generates instruction-tuning QA pairs from the processed metadata using spatio-temporal scene graphs. The generation script creates question-answer pairs covering eight spatial reasoning task types: object counting, relative distance, relative direction, appearance order, object size, absolute distance, room size, and route planning. A random subset of 10,000 training samples is used for efficiency while maintaining task diversity. The generated QA pairs are formatted according to the VLM-3R training format [26]. We also download the official datasets and splits for VSi-Bench and VSTI-Bench from the Hugging Face repository to ensure consistency with the original evaluation.

4.2.2 ScanNet++

ScanNet++ is a high-fidelity indoor scene dataset captured using iPhone devices with authentic appearance data [4]. Unlike ScanNet, which uses specialised RGB-D cameras, ScanNet++ captures scenes using consumer iPhones providing RGB videos with accurate camera poses and depth estimates computed using ARKit. The dataset requires more extensive preprocessing due to its iPhone-based capture format.

Phase 1 (Data Extraction) comprises four preprocessing steps. Step 1.1 exports videos from Digital Single-Lens Reflex (DSLR) images: ScanNet++ provides DSLR camera data that must be compiled into MP4 video format (3 FPS, 389x584 resolution) for both train and val splits. This differs from ScanNet which exports from .sens files; ScanNet++ requires compiling individual DSLR frames into video format. Step 1.2 (Preparation) comprises three sub-operations: installing the ScanNet++ toolbox and renderpy, rendering depth maps from the 3D mesh using the ScanNet++ renderpy toolbox, and exporting COLMAP camera parameters (intrinsics and extrinsics) to NPZ format (a file format by numpy that provides storage of array data using gzip compression [35]). Step 1.3 (Export Sampled Frames) samples 32 frames per scene with corresponding camera poses, depth, and intrinsics from the DSLR data: this extracts colour images, depth maps, camera poses (as text files), and camera intrinsics for each sampled frame, organised into separate directories for train and val splits. Step 1.4 (Export iPhone Frames) optionally extracts RGB and depth frames from iPhone video/depth data using COLMAP poses for scenes where iPhone capture is available, producing colour images, depth maps, camera poses, and intrinsics in the same format as Step 1.3.

Phase 2 (Metadata Generation) generates scene-level and frame-level metadata similar to ScanNet. Step 2.1 extracts scene metadata including room size, room center, object counts, and 3D bounding boxes from the semantic mesh files. Step 2.2 extracts frame-level metadata including camera poses, 2D bounding boxes, and camera intrinsics for each sampled frame.

Phase 3 (QA Generation) generates instruction-tuning QA pairs from the processed metadata using spatio-temporal scene graphs, following the same approach as ScanNet. The generated QA pairs are formatted according to the VLM-3R training format. We also download the official datasets and splits for VSi-Bench and VSTI-Bench from the Hugging Face repository to ensure consistency with the original evaluation.

4.2.3 ARKitScenes

ARKitScenes is a diverse real-world dataset for mobile indoor planning captured using ARKit-enabled iOS devices [20]. The dataset includes RGB-D video sequences with camera poses, depth maps, and spatial anchor data from indoor environments.

Phase 1 (Data Preparation) comprises three preprocessing steps. Step 1.1 generates train and validation split files from the metadata CSV: unlike ScanNet which provides official train/val splits, ARKitScenes requires manual split generation based on the dataset structure. This step parses the dataset metadata CSV and creates train.txt and val.txt files listing the video IDs for each split. Step 1.2 generates scene metadata for Training and Validation splits: this step extracts scene-level information including object counts, bounding boxes, room size, and room center from the ARKit captures, processing both splits separately. Step 1.3 converts .mov to .mp4 format for train and val splits: VLM-3R's video processing pipeline expects standardised MP4 video files rather than the raw iOS capture format. This conversion ensures compatibility with the vision encoder's video loading pipeline to sample the 32 frames required.

Phase 2 (Metadata Generation) generates frame-level metadata linking 3D scene structure to specific video frames. This includes camera poses and intrinsics for each sampled frame, enabling questions about temporal spatial relationships.

Phase 3 (QA Generation) generates instruction-tuning QA pairs from the processed metadata using spatio-temporal scene graphs, following the same approach as ScanNet and ScanNet++. The generated QA pairs are formatted according to the VLM-3R training format. We also download the official datasets and splits for VSi-Bench and VSTI-Bench from the Hugging Face repository to ensure consistency with the original evaluation.

4.3 Spatial Reasoning Benchmarks

Evaluating spatial reasoning in vision-language models requires specialised benchmarks that assess a model's ability to understand and reason about 3D space, spatial relationships, and temporal dynamics from visual input.

4.3.1 VSI-Bench

VSiBench (Vision-Spatial Intelligence Benchmark) was introduced by Thinking in Space [92], a work investigating how current VLMs understand spatial relationships from video. VSiBench contains approximately 5,000 question-answer pairs derived from 288 real-world indoor videos, covering eight spatial reasoning task types: object counting, relative distance, relative direction, appearance order, object size, absolute distance, room size, and route planning. These tasks assess both configurational understanding (object arrangements) and measurement estimation (quantitative spatial judgments). The benchmark revealed that even state-of-the-art proprietary models like GPT-4V and Gemini Pro exhibit significant performance gaps in fundamental spatial tasks. Figure 4.1 shows example QA pairs from VSI-Bench and the distribution of tasks across categories.

Understanding The Tasks

Figures B.1 to B.3 show qualitative examples of VSI-Bench [92] spatial reasoning tasks. The benchmark evaluates eight distinct spatial reasoning categories:

Object Counting requires the model to enumerate visible instances of a specified object class (e.g., chairs, tables) within a 3D scene reconstruction. This tests the model's ability to parse visual features for discrete quantity estimation. For example: *"How many chair(s) are in this room?"* with answer *6*.

Absolute Distance Estimation asks for metric distances between objects in meters, requiring the model to translate visual features into depth estimates. This is distinct from relative distance. For example: *"What is the distance between the bed and the sofa (in meters)?"* with answer *3.2*.

Route Planning tests navigation reasoning by asking for sequential turning decisions to traverse from a start position to a goal. The model must reason about allocentric spatial relationships. For example:

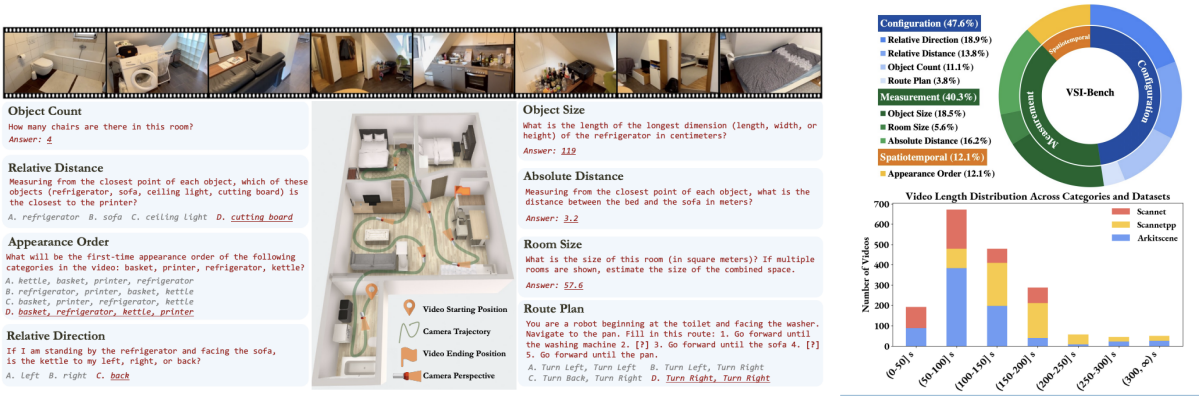


Figure 4.1: VSi-Bench task demonstration showing example QA pairs (left) and benchmark statistics showing task distribution across three main categories and video length statistics (right). Right image shows top: the distribution of tasks across three main categories. Bottom: the video length statistic. Reprinted from Yang et al. (2024) [92].

"You are a robot beginning at the kitchen sink and facing window. You want to navigate to the refrigerator. Options: A. Turn Back B. Turn Right C. Turn Left" with answer *A*.

Room Size Estimation requires metric area estimation of entire rooms in square meters, testing global spatial reasoning about scene-scale properties. For example: *"What is the size of this room (in square meters)?"* with answer *10.5*.

Object Size Estimation asks for metric dimensions of individual objects (e.g., chair length in centimeters), testing fine-grained object-level spatial reasoning. For example: *"What is the length of the longest dimension of the chair, measured in centimeters?"* with answer *69*.

Appearance Order tests temporal sequencing by asking for the order in which specified objects first appear during scene traversal. For example: *"What will be the first-time appearance order of: basket, door, pillow, laptop? Options: A. basket, pillow, door, laptop B. pillow, door, laptop, basket C. basket, door, pillow, laptop D. door, basket, pillow, laptop"* with answer *B*.

Relative Distance asks which of several objects is closest to a reference object, requiring comparative spatial reasoning without metric estimation. For example: *"Which of these objects (chair, table, tv, fireplace) is the closest to the sofa?"* with answer *B*.

Relative Direction tests directional spatial reasoning, with easy variants asking simple left/right/front/back and hard variants asking fine-grained front-left/front-right/back-left/back-right. For example: *"If I am standing by the computer tower and facing the heater, is the door to my left, right, or back?"* with answer *A*; or the harder variant: *"If I am standing by the ceiling light and facing the door, is the kettle to my front-left, front-right, back-left, or back-right?"* with answer *D*.

What We Ask the Model

The VSI-Bench tasks evaluate a model's ability to understand and reason about 3D space from monocular video. The questions require the model to go beyond simple object recognition: it must track objects across frames, maintain consistent spatial relationships, and perform quantitative reasoning about distances, sizes, and counts. For instance, object counting requires parsing visual features to enumerate instances; absolute distance requires translating visual appearance into metric depth estimates; and route planning requires reasoning about allocentric spatial transformations from an egocentric perspective. Successfully answering these questions demonstrates that the model has built a coherent internal representation of the 3D scene structure.

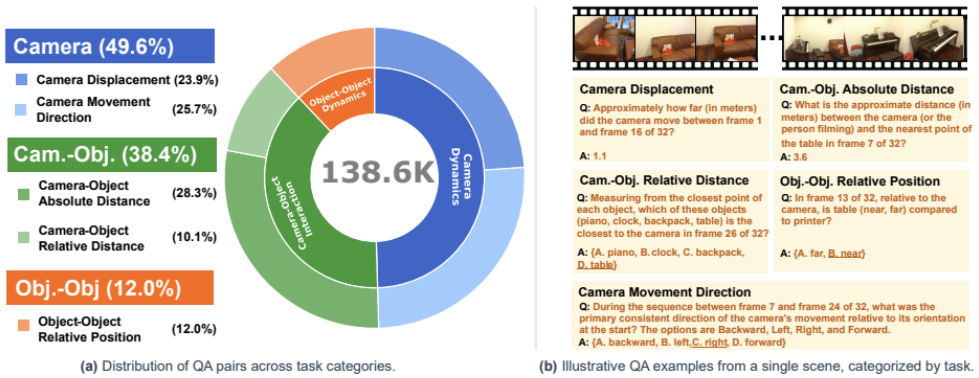


Figure 4.5: VSTI-Bench overview showing the statistical distribution of QA pairs by primary categories and sub-categories, along with example question-answer pairs illustrating different task types within a scene. Reprinted from Fan et al. (2025) [26].

Evaluation Methodology

VSiBench employs different evaluation protocols depending on question type. The benchmark splits tasks into two categories with distinct scoring methods.

Multiple-choice questions (relative distance, relative direction, appearance order, route planning) are evaluated by matching the model's output against the designated option letter (A, B, C, or D). The model outputs a single letter, and accuracy is the fraction of correct letter predictions. This applies to: relative distance (`mc_answer`), relative direction easy variants (`mc_answer`), appearance order (`mc_answer`), and route planning (`mc_answer`). A response is correct only if the predicted letter exactly matches the ground-truth letter label.

Numerical questions (object counting, absolute distance, object size, room size) are evaluated against a floating-point ground truth with an absolute tolerance of ± 0.5 units. For absolute distance and room size, the tolerance reflects the inherent uncertainty in metric estimation from visual features. For object size, the tolerance similarly accounts for minor dimensional errors. Concretely, a prediction p is correct if $|p - \text{ground_truth}| \leq 0.5$. The benchmarks use the `ground_truth` field (a float) as the reference value.

4.3.2 VSTI-Bench

VSTI-Bench (Vision-Spatial-Temporal Intelligence Benchmark) was introduced alongside VLM-3R to address the lack of benchmarks for evaluating temporal spatial reasoning [26]. While VSiBench focuses on static spatial relationships within individual frames, VSTI-Bench assesses understanding of how spatial relationships evolve over time—a capability essential for embodied AI and robotics applications. The benchmark comprises approximately 138,600 QA pairs across five task categories: camera dynamics (49.6%), camera-object interactions (38.4%), and object relative positions (12.0%). Tasks include reasoning about camera displacement between frames, changes in object-camera distance, and evolving spatial configurations during navigation. Figure 4.5 shows VSTI-Bench task categories and example QA pairs.

Understanding The Tasks

Figure B.4 and Figure B.5 show qualitative examples of VSTI-Bench [26] spatial-temporal reasoning tasks. The benchmark evaluates five primary task categories:

Camera-Object Relative Distance asks which of two objects is closer to the camera at a specific frame, requiring comparative spatial reasoning in video. For example: *"Measuring from the closest point of each object, which of these objects (plant, window) is the closest to the camera in frame 4 of 32? Options: A. plant B. window"* with answer *B*.

Camera Movement Direction tests temporal reasoning about camera ego-motion by asking for the

primary direction of camera movement between specified frames. For example: *"During the sequence between frame 4 and frame 23 of 32, what was the primary consistent direction of the camera's movement? Options: A. Forward B. Left C. Right"* with answer *C*.

Camera-Object Absolute Distance requires metric depth estimation from camera to object in video frames. For example: *"What is the approximate distance (in meters) between the camera and the nearest point of the backpack in frame 10 of 32?"* with answer *0.7*.

Camera Displacement asks for the total metric distance the camera traveled between two frames, testing ego-motion estimation. For example: *"Approximately how far (in meters) did the camera move between frame 24 and frame 31 of 32?"* with answer *1.2*.

Object-Object Relative Position tests relative spatial reasoning between pairs of objects from the camera's perspective, with variants for left/right (LR), near/far (NF), and above/below (UD) relationships. For example: *"In frame 13 of 31, relative to monitor, is bed to the Left/Right? Options: A. Right B. Left"* with answer *B*; or *"In frame 18 of 31, relative to chair, is desk to the Above/Below? Options: A. Above B. Below"* with answer *B*.

What We Ask the Model

The VSTI-Bench tasks evaluate a model's ability to reason about both spatial relationships and their evolution over time during video capture. Unlike VSI-Bench, which focuses on static spatial reasoning within individual frames, VSTI-Bench requires understanding how spatial configurations change as the camera moves through the scene. The questions ask about camera-object relationships (how far is the camera from an object?), camera movement (which direction did the camera travel?), and object-object relationships from the camera's perspective at specific moments in time. Answering these questions requires the model to track the camera's trajectory, understand ego-motion effects on spatial perception, and reason about relative object positions at arbitrary video frames. This temporal-spatial reasoning is essential for embodied AI applications such as robot navigation and augmented reality.

Evaluation Methodology

VSTI-Bench employs different evaluation protocols depending on question type, with numerical questions using Mean Relative Accuracy (MRA) and multiple-choice questions using exact letter matching.

Multiple-choice questions (camera-object relative distance v1/v2/v3, camera movement direction, object-object relative position LR/NF/UD) are evaluated via exact match against the `mc_answer` field. The model is instructed to output a single letter (A, B, C, or D) corresponding to its chosen option. The prediction string is first processed through a fuzzy matching function that extracts the first token, strips trailing punctuation, and lowercases the result (e.g., `"A. The answer is left"` becomes `"a"`). This preprocessed prediction is then compared against the `mc_answer` field (which contains the letter, e.g., `"A"`), not the `ground_truth` field (which contains the semantic answer word such as `"Left"`, `"Right"`, or `"Forward"`).

Numerical questions (camera-object absolute distance, camera displacement) use Mean Relative Accuracy (MRA) as the metric. MRA computes accuracy at multiple relative error thresholds rather than a fixed absolute tolerance. For a prediction p and ground truth t, relative error is $e = |p - t|/t$. Accuracy is computed at 11 confidence thresholds from 50% relative error down to 5% relative error:

$$\text{MRA} = \frac{1}{11} \sum_{c \in \{0.50, 0.545, \ldots, 0.95\}} \mathbf{1}\{e \leq 1 - c\} \tag{4.1}$$

This means a prediction is correct at threshold c if its relative error is at most $1 - c$. For example, at the strictest threshold (95% confidence), the prediction must have relative error ≤ 0.05 (i.e., within 5% of the ground truth). The final MRA score is the mean accuracy across all thresholds, ranging from 0 to 1. This metric rewards predictions that are proportionally accurate across different magnitudes, rather than applying a fixed absolute tolerance.

4.3.3 Evaluation Environment Setup

Setting up the evaluation environment on Isambard AI Phase 2 requires careful package management for the the ARM64 (aarch64) architecture. Miniforge3 can provide us with a conda installation compatible with aarch64.

PyTorch is installed from source on a compute node which provides a package with aarch64, CUDA (GPU), and NumPy support. Installation must be performed on a compute node to ensure CUDA is available and a GPU 'visible' during installation. This ensures the correct GPU-enabled version of PyTorch is installed for the GH200 Grace Hopper Superchip.

The environments require PyTorch 2.1.1/2, the lmms-eval library for standardised benchmark evaluation [49], and FlashAttention for memory-efficient inference [19]. Additional dependencies include the transformers library for model loading and the pyav library for video processing on ARM64 architectures. Video loading is performed via pyav instead of decord due to compatibility issues with the ARM64 architecture of Isambard AI Phase 2 nodes; the codebase provides both decord and pyav processing functions and is configured to use pyav on this platform.

4.4 VLM-3R Performance Reproduction

4.4.1 Demo Inference

Before proceeding to benchmark evaluation, the dual-memory augmented VLM-3R model was tested on demo video inputs to verify the implementation was functioning correctly. For video inference, a demo script is used to run the model on an input video. The script takes several positional arguments that control how the video is processed and how the model behaves and allow us to load the VLM-3R released model weights.

The required inputs are:

- **Model checkpoint path:** the trained model weights (e.g., a Hugging Face checkpoint such as `Journey9ni/vlm-3r-llava-qwen2-lora`).
- **Conversation mode:** a template that defines how prompts and responses are formatted (e.g., `qwen1.5`).
- **Number of frames:** how many frames are sampled from the video. These are typically sampled uniformly across the full video to provide temporal coverage (e.g., 32 frames).
- **Spatial pool stride:** controls how much the spatial resolution of visual features is reduced inside the vision encoder. A stride of 2 means the feature map is downsampled by a factor of 2 in each spatial dimension, reducing computation while keeping coarse spatial structure.
- **Pooling mode:** the method used to combine neighbouring visual features during downsampling (e.g., average pooling, which takes the mean of local regions).
- **Video path:** the input video file.
- **Base model path:** the underlying pretrained model used before applying fine-tuning or LoRA weights.

A typical configuration uses a pretrained checkpoint, the `qwen1.5` conversation template, 32 uniformly sampled frames, a spatial pooling stride of 2, and average pooling. Internally, the script loads the video using payav, extracts and samples frames, processes them through the vision encoder (with spatial pooling applied), converts them into visual tokens, and performs inference with the language model to generate a response.

Frame sampling from the input video is performed uniformly: if the video has more frames than the specified number of frames parameter, frames are evenly spaced across the video duration. Each sampled frame is processed through the visual encoder (SigLIP-SO400M) and spatial encoder (CUT3R) before the fused 3D-aware features are passed to the language model. The model generates text responses autoregressively, with the output written to a JSON file in the specified output directory. Figure 4.8 shows the



Figure 4.8: Grid of uniformly sampled frames from the input video that the model processes.

frames of the video we passed to the model for demo inference. The demo script also passes the following prompt to the model: "If I am standing by the stool and facing the stove, is the sofa to my left, right, or back? An object is to my back if I would have to turn at least 135 degrees in order to face it." to which it correctly responds "right".

4.4.2 Benchmark results

Benchmark evaluation of the dual-memory augmented VLM-3R model uses the thinking in space evaluation for VSI (and adapted for VSTI bench) and the lmms evaluation framework, a standardised evaluation library for large multimodal models [49].

Evaluation is submitted as a SLURM job on the HPC cluster. The SLURM submission script specifies: 1 node with 4 GPUs (gres=gpu:4), 32 CPU cores per task and 400GB memory. CUDA 12.6 is loaded to provide the necessary GPU runtime support. The script launches the evaluation via the accelerate library for distributed inference across the 4 GPUs. The evaluation framework handles batching, inference, and metric computation automatically.

Key model arguments passed to the evaluation include: the pretrained model checkpoint, the base model, conversation template, maximum number of frames (32), and dual memory configuration flags (use dual memory, memory mode, memory alpha).

Results are logged with sample-level predictions stored in JSON format in the output directory, enabling detailed analysis of per-category performance and error analysis. Metrics computed include accuracy across all spatial reasoning task categories and breakdown by task type.

Our VSi-Bench evaluation achieved an accuracy of 60.9%, matching the reported result of 60.9% from the original VLM-3R paper [26]. Our VSTI-Bench evaluation achieved an accuracy of 58.7%, matching the reported result of 58.8% from the original VLM-3R paper [26]. These results confirm that we successfully reproduced the reported benchmark performance. Detailed per-category results are shown in Table 4.1 for VSI-Bench and Table 4.2 for VSTI-Bench.

Table 4.1: VSI-Bench Results: Ours vs Reported

	Avg.	*Obj. Count*	*Abs. Dist.*	*Obj. Size*	*Room Size*	*Rel. Dist.*	*Rel. Dir.*	*Route Plan*	*Appr. Order*
		Numerical Answer				Multiple-Choice Answer			
Reported	60.9	70.2	49.4	69.2	67.1	65.4	80.5	45.4	40.1
Ours	60.9	70.0	49.4	69.2	67.5	65.1	79.9	45.4	40.5
Difference	-0.1	-0.2	+0.0	+0.0	+0.4	-0.3	-0.6	-0.0	+0.4

Table 4.2: VSTI-Bench Results: Ours vs Reported

	Avg.	*Cam-Obj Abs. Dist.*	*Cam. Displace.*	*Cam. Mov. Dir.*	*Obj-Obj Rel. Pos.*	*Cam-Obj Rel. Dist.*
		Numerical Answer		Multiple-Choice Answer		
Reported	58.8	39.4	39.6	60.6	86.5	68.6
Ours	58.7	38.9	39.4	60.7	86.1	68.2
Difference	-0.1	-0.5	-0.2	+0.1	-0.4	-0.4

Table 4.3: VSI-Bench Initial Non-Trainable Memory Results

	Overall	**Abs. Dist.**	**Appr. Order**	**Obj. Count**	**Obj. Size**	**Rel. Dir.**	**Rel. Dist.**	**Room Size**	**Route Plan**
No Memory	**60.9**	**49.4**	40.5	70.0	69.2	**79.9**	65.1	**67.5**	**45.4**
Working Memory Only	48.2	34.5	**46.2**	62.1	62.0	65.1	50.2	25.3	42.9
Episodic Memory Only	60.8	49.3	40.3	**70.1**	**69.3**	79.7	**65.5**	**67.5**	44.9
Fusion (Equal Weight)	59.4	48.3	40.3	67.4	68.6	79.5	62.7	64.0	44.3

Table 4.4: VSTI-Bench Initial Non-Trainable Memory Results

	Overall	**Cam-Obj Abs. Dist.**	**Cam. Displace.**	**Cam. Mov. Dir.**	**Obj-Obj Rel. Pos.**	**Cam-Obj Rel. Dist.**
No Memory	**58.7**	39.2	**39.6**	**60.7**	68.1	**86.5**
Working Memory Only	**58.7**	**39.3**	39.4	60.4	**86.2**	68.1
Episodic Memory Only	**58.7**	**39.3**	39.4	60.4	**86.2**	68.1
Fusion (Equal Weight)	**58.7**	39.2	39.4	60.4	**86.2**	68.1

4.5 Non-Trainable Memory

We proceed first by implementing the non trainable dual memory system with no query based frame selection. Then running evaluation in the same way as in Section 4.4.

4.5.1 Initial Non-Trainable Memory Results

The non-trainable baseline was evaluated on VSI-Bench and VSTI-Bench to establish a performance baseline before exploring improved fusion strategies. Results are shown in Tables 4.3 and 4.4. We see that memory has none or negligible effect on VSTI-Bench. Also, that episodic memory only makes none or negligible difference compared to the baseline on VSI-Bench but with working memory only or fused memory results are significantly worse. We analyse this in Section 5.1.1.

Table 4.5: VSI-Bench Full Non-Trainable Ablation Results

	Overall	Abs. Dist.	Appr. Order	Obj. Count	Obj. Size	Rel. Dir.	Rel. Dist.	Room Size	Route Plan
					Working Memory Only				
WM ($\alpha = 0.1$)	60.7	**49.4**	40.3	69.6	69.2	**80.0**	64.7	**67.7**	44.9
WM ($\alpha = 0.3$)	59.2	47.8	42.1	68.1	68.1	78.2	63.8	61.5	44.1
WM ($\alpha = 0.5$)	56.8	44.1	44.5	65.6	66.6	75.3	62.3	52.9	43.3
WM ($\alpha = 0.8$)	50.7	36.9	**45.5**	63.3	63.6	68.3	53.9	30.1	43.8
WM ($\alpha = 1.0$)	48.2	34.5	46.2	62.1	62.0	65.1	50.2	25.3	42.9
					Episodic Memory Only				
EM ($\alpha = 0.1$)	60.8	49.3	40.0	**70.1**	69.3	79.8	**65.6**	**67.7**	44.9
EM ($\alpha = 0.3$)	**60.9**	49.3	40.0	**70.1**	**69.4**	79.8	65.5	**67.7**	**45.4**
EM ($\alpha = 0.5$)	**60.9**	49.3	40.5	**70.1**	69.3	79.7	65.4	**67.7**	**45.4**
EM ($\alpha = 0.8$)	**60.9**	49.3	40.3	**70.1**	69.3	79.7	65.4	67.6	**45.4**
EM ($\alpha = 1.0$)	60.8	49.3	40.3	**70.1**	69.3	79.7	65.5	67.5	44.9
					Fusion (Working + Episodic)				
Fusion ($\alpha = 0.1$)	59.5	48.5	40.3	67.5	68.7	79.5	62.8	64.2	44.3
Fusion ($\alpha = 0.3$)	59.4	48.3	40.3	67.4	68.6	79.5	62.7	64.0	44.3
Fusion ($\alpha = 0.5$)	56.0	43.1	40.3	65.7	66.3	76.3	62.1	50.7	43.8
Fusion ($\alpha = 0.8$)	52.3	39.5	42.1	63.2	64.1	70.2	55.8	38.5	42.5
Fusion ($\alpha = 1.0$)	50.1	37.2	43.8	62.0	62.5	66.8	52.3	32.1	41.8

Table 4.6: VSTI-Bench Full Non-Trainable Ablation Results

	Overall	Cam-Obj Abs. Dist.	Cam. Displace.	Cam. Mov. Dir.	Obj-Obj Rel. Pos.	Cam-Obj Rel. Dist.
			Working Memory Only			
WM ($\alpha = 0.1, 0.3, 0, 5, 0.8, 1.0$)	58.6	**39.3**	39.5	**60.4**	**86.2**	**68.1**
			Episodic Memory Only			
EM ($\alpha = 0.1, 0.3, 0, 5, 0.8, 1.0$)	58.6	39.1	39.4	**60.4**	**86.2**	**68.1**
			Fusion (Working + Episodic)			
Fusion ($\alpha = 0.1, 0.3, 0, 5, 0.8, 1.0$)	**58.7**	39.2	**39.8**	**60.4**	**86.2**	**68.1**

Table 4.7: VSTI-Bench Memory Buffer Sizing Ablation Results

	Overall	Cam-Obj Abs. Dist.	Cam. Displace.	Cam. Mov. Dir.	Obj-Obj Rel. Pos.	Cam-Obj Rel. Dist.
$(L_w, L_e) = (16, 16)$	58.7	39.3	39.6	**60.4**	**86.2**	**68.1**
$(L_w, L_e) = (16, 32)$	58.7	**39.4**	39.6	**60.4**	**86.2**	**68.1**
$(L_w, L_e) = (16, 8)$	**58.8**	**39.4**	39.7	**60.4**	**86.2**	**68.1**
$(L_w, L_e) = (8, 32)$	58.7	39.2	**39.8**	**60.4**	**86.2**	**68.1**

4.5.2 Full Non-Trainable Memory Results

The complete non-trainable configuration with residual connection and alpha blending was evaluated on VSI-Bench and VSTI-Bench. Results are shown in Tables 4.5 and 4.6. Our ablation on alpha blending shows that we are able to reclaim some of the working memory and fused memory performance by blending the features passed through memory with ones not passed. We analyse this in Section 5.1.1.

4.5.3 Memory Buffer Sizing

Working memory buffer size (L_w) and episodic memory buffer size (L_e) were ablated to understand their effect on performance. Table 4.7 shows the results. We see that for untrainable memory, changing the memory buffer sizes does not affect performance meaningfully.

4.6 Trainable Memory

We now modify the existing implementation to match the method for the trainable memory, again, with no query based frame selection.

Table 4.8: Multihead Attention Configuration for Trainable Dual Memory Module

Parameter	Value
Number of attention heads	8
Attention embedding dimension	1152
Dropout rate	0.1
Query projection	$Q = H_t$ (input features)
Key/Value projection	$K = V = W_t/E_t$ (memory buffers)

4.6.1 Implementation Details

The trainable dual memory module extends the non-trainable version by introducing learnable parameters that are optimised via gradient descent. The architecture comprises three learnable components:

Working Memory Retrieval. The working memory maintains a First-In-First-Out (FIFO) buffer of capacity $L_w = 8$, storing the most recent frame features. At each timestep t, retrieval is performed via scaled dot-product attention using PyTorch's `nn.MultiheadAttention` with `batch_first=True`:

$$M_t^w = \text{MultiheadAttention}(Q = H_t, K = W_t, V = W_t) \tag{4.2}$$

The attention mechanism uses 8 heads with `embed_dim=feature_dim`, where `feature_dim=1152` to match the spatial tower output dimension. Dropout of 0.1 is applied to attention weights during training.

Episodic Memory Retrieval. The episodic memory maintains a capacity of $L_e = 32$ elements using similarity-based replacement. Memory is retrieved in the same way as above.

Memory Fusion. Retrieved memories are fused via a learnable gated MLP:

$$\gamma_t = \text{Sigmoid}(\text{MLP}([M_t^w; M_t^e])) \tag{4.3}$$

$$M_t = \gamma_t \odot M_t^w + (1 - \gamma_t) \odot M_t^e \tag{4.4}$$

The fusion MLP architecture is: Linear($2 \times$ feature_dim $\rightarrow$ feature_dim) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.1) $\rightarrow$ Linear(feature_dim $\rightarrow$ feature_dim) $\rightarrow$ Sigmoid, producing gate values in $[0, 1]$.

Attention Configuration. Table 4.8 summarises the attention module parameters.

4.6.2 Training Protocol

Training the trainable memory proceeded in three stages. The first was a bring-up stage used to verify gradient flow and stabilise the new memory path. The second was a memory-only VSTI training configuration that isolated the effect of the memory module while keeping the task LoRA frozen. The third was the later continuation stage in which the task LoRA was unfrozen again and trained over the memory-augmented pathway.

Stage 1: Memory Bring-up

In this bring-up stage, the goal was not yet to obtain the final benchmark checkpoint, but to verify that gradients reached the newly introduced attention modules and fusion MLP, and to diagnose the exploding-gradient behaviour described below.

This prototype configuration used 1 training epoch (although never ran for a full epoch - runs were cancelled after gradient explosion or once learning rate had peaked), learning rate 1×10^{-5}, per-device batch size 1, and gradient accumulation of 128 steps. The base checkpoint was `lmms-lab/LLaVA-NeXT-Video-7B-Qwen2`. The vision tower, spatial tower, fusion block, and multimodal projector were kept frozen, while the memory components were trainable so that gradients could be traced through the new path. BF16 mixed precision, DeepSpeed ZeRO-2, gradient checkpointing, cosine learning-rate decay, and a 3% warmup ratio

were used here as well. This stage provided the debugging environment in which the exploding-gradient issue was isolated and corrected.

Stage 2: Memory-Only Training

The first full trainig stage was run on Isambard-AI Phase 2 using 16 compute nodes, each containing 4 NVIDIA GH200 Grace Hopper Superchips, for a total of 64 GPUs. Jobs were launched through SLURM with 32 CPU cores and 400 GB RAM per task, using CUDA 12.6 and DeepSpeed ZeRO-2.

For this stage, the model was initialised from `lmms-lab/LLaVA-NeXT-Video-7B-Qwen2` and then loaded with the task-specific VSTI LoRA checkpoint `Journey9ni/vlm-3r-llava-qwen2-lora-vstibench` via `lora_weight_path`. Crucially, `lora_enable=False` was used, so the task LoRA and associated non-LoRA trainables were merged into the model and then kept frozen. This isolated the effect of the memory module during optimisation. The base language model, vision tower (encoder), spatial tower (encoder), fusion block, and multimodal projector remained frozen throughout.

The exact hyperparameters for the final VSTI memory-only runs were: 1 epoch, learning rate 1×10^{-5}, per-device batch size 1, gradient accumulation of 2 steps (effective batch size 128 across 64 GPUs), cosine learning-rate schedule, weight decay 0, warmup ratio 3%, and gradient clipping at 0.3. Here, 1 epoch means one complete pass over the selected VSTI training split; under this configuration that corresponded to 1035 optimisation steps. Training used BF16 mixed precision, `frames_upbound=32`, `force_sample=True`, `dataloader_drop_last=True`, `dataloader_num_workers=0`, and gradient checkpointing.

Stage 3: Memory with Unfrozen Task LoRA

After obtaining a stable memory checkpoint, the final stage re-enabled optimisation of the task-specific LoRA so that the language-side task adapter could co-adapt to the memory-augmented visual pathway. In implementation terms, this means restoring the task adapter as trainable PEFT weights with `lora_enable=True` rather than merging it and freezing it as in the memory-only stage. The key methodological point is that memory is no longer being evaluated as an isolated module: it is being refined together with the task adapter that consumes its output.

This final stage matters because it changes the role of trainable memory. In Stage 2, memory is trained against a fixed downstream task interface. In Stage 3, the downstream LoRA can adapt to the representations produced by memory, potentially improving compatibility between retrieval, fusion, and answer generation.

4.6.3 Monitoring Training

Training progress was monitored primarily through per-step console logging, saved trainer state, and SLURM output logs. The final multi-node VSTI memory runs used `logging_steps=1`, saved checkpoints every 100 steps, and retained at most 3 checkpoints. The prototype bring-up stage additionally reported to Weights & Biases (wandb) during debugging, but the final large-scale VSTI memory runs were configured with `report_to none` in order to keep the execution path simpler and reduce external dependencies during distributed training.

The Exploding Gradient Problem

During initial training attempts, we encountered the exploding gradient problem, where gradient magnitudes grew exponentially during backpropagation through the deep network. This caused training instability where loss values would diverge to NaN. Figure 4.9 illustrates this phenomenon, showing loss divergence and gradient magnitude explosion during early training.
Cause

The instability appears to have arisen from the newly introduced trainable memory path, particularly the interaction between memory attention and the feature-injection path used to connect the memory module to the rest of the model. Trying gradient clipping was at 0.3 alone was not sufficient to make optimisation reliable.



Figure 4.9: Exploding gradient phenomenon observed during initial training. Left: Loss values diverging to infinity within the first training steps. Right: Gradient magnitudes growing exponentially, reaching values orders of magnitude larger than stable training regimes.

Table 4.9: VSTI-Bench Results: Baseline vs Trainable Memory

	Avg.	*Cam-Obj Abs. Dist.*	*Cam. Displace.*	*Cam. Mov. Dir.*	*Obj-Obj Rel. Pos.*	*Cam-Obj Rel. Dist.*
		Numerical Answer		Multiple-Choice Answer		
Baseline	**58.7**	**38.9**	**39.4**	**60.7**	**86.1**	68.2
Trainable Memory ($L_w = 8, L_e = 32$)	53.8	34.5	38.8	58.8	68.8	**68.7**
Trainable Memory ($L_w = 4, L_e = 32$)	54.1	35.9	38.2	56.7	71.4	68.2
Difference	-4.6	-3.0	-1.2	-4.0	-14.7	+0.0

Solution

A term: `image_features = image_features + (safety_out - safety_out.detach())` was added in the forward pass in attempt to stabilise gradients. This adds a zero-magnitude term during forward computation while altering the backward gradient path. In subsequent stable runs, gradient norms were far smaller (for example around 0.13–0.37 in the later memory bring-up observations, and around 0.17–0.19 at the end of the final VSTI memory-only run), while losses fell from values near 1.0 early in training to around 0.12 near the end of the epoch. These changes were therefore associated with a substantial improvement in optimisation stability.

4.6.4 Results

The trainable memory checkpoints were evaluated on VSTI-Bench against the reproduced VLM-3R baseline. Results are shown in Table 4.9. Both trainable-memory variants underperformed the baseline overall. The initial configuration with $L_w = 8$ and $L_e = 32$ achieved 53.8 compared to the baseline 58.7, while reducing the working-memory capacity to $L_w = 4$ with $L_e = 32$ produced a small improvement to 54.1.

The $L_w = 4$ continuation recovered some of the degradation seen in the original trainable-memory run. Relative to the $L_w = 8$ model, camera-object absolute distance improved from 34.5 to 35.9 and object-object relative position improved from 68.8 to 71.4, suggesting that a shorter FIFO buffer reduced some harmful short-term interference. However, this came with weaker camera movement direction performance, which fell from 58.8 to 56.7. Camera-object relative distance also lost the small gain seen in the original memory run, returning from 68.7 to the baseline value of 68.2.

Overall, the smaller working-memory buffer was less harmful than the original setting, but neither trainable-memory configuration surpassed the reproduced VLM-3R baseline.

4.7 Query-based Frame Selection

Our query-based frame selection implementation follows the VideoStreaming Section 3.3 architecture [63], adapted for integration with the VLM-3R pipeline. The architecture comprises three learned MLPs: a streaming encoder projecting from the vision encoder output dimension to a 2560-dimensional intermediate space, a question encoder projecting from the LLM dimension to the same intermediate space, and an LLM input MLP projecting from 2560 back to the LLM dimension.

The paper uses 4096-dimensional projections to match their LLM dimension [63], but our base LLaVA-NeXT-Video model uses Qwen2 as its language backbone with an LLM dimension of 3584. The question encoder and LLM input projector therefore project from 3584 rather than 4096; the intermediate dimension of 2560 is retained as per the paper.

The paper's default 32-to-4 configuration (selecting 4 clips from a series of 16 frame clips sampled at 1 fps) is adapted to 128-to-32 frames for our spatial reasoning tasks: up to 128 frames are sampled from each video, and 32 frames are selected via query-based selection. This is a reasonable layout as the VLM-3R pipeline showed good performance on spatial reasoning tasks with 32 uniformly sampled frames.

The selection mechanism itself follows the paper exactly: Gumbel-Topk with straight-through estimator for differentiable training, hard top-k for inference. The straight-through weights are $1.0 + \text{softmax}(\tilde{s})_{i_j} - \text{softmax}(\tilde{s})_{i_j}.\text{detach}()$, allowing gradients to flow through the similarity scores while maintaining hard selection in the forward pass.

4.7.1 Training Protocol

As with trainable memory, query-based frame selection was developed in two stages: first as an isolated component with the VLM-3R task adapter frozen, and then as a continuation run in which the task LoRA itself was unfrozen alongside the query-selection MLPs.

Stage 1: Query Selection in Isolation

The first stage trained only the query-selection module while keeping the rest of the VLM-3R stack fixed. This run was performed on Isambard-AI Phase 2 using 16 nodes with 4 GH200 Grace Hopper Superchips per node (64 accelerators total). The base model was `lmms-lab/LLaVA-NeXT-Video-7B-Qwen2`; the VLM-3R task adapter was loaded via `lora_weight_path`, but `lora_enable=False` was used, so the task LoRA was merged into the model and then frozen.

In practice, the full model was frozen first, then only the query-selection module was unfrozen. The vision tower, spatial tower, fusion block, multimodal projector, base language model, and task LoRA weights remained fixed. Dual memory was disabled in this isolated query-selection stage, so the learned module received features directly from the VLM-3R visual-spatial pipeline and produced a filtered subset for the language model.

The hyperparameters for this isolated query-selection training were: 1 epoch, learning rate 1×10^{-4}, per-device batch size 1, and a target effective batch size of 64 (On the 64-GPU Isambard configuration this means gradient accumulation of 1). The run used BF16 mixed precision, DeepSpeed ZeRO-2, cosine learning-rate decay, weight decay 0, 3% warmup, gradient clipping at 0.3, gradient checkpointing. Query selection then reduced 128 candidate frames to 32 selected frames using Gumbel-Topk.

Stage 2: Query Selection with Unfrozen Task LoRA

The second stage kept the same query-selection architecture and distributed training setup, but switched to `lora_enable=True`: the VLM-3R task LoRA is no longer merged and frozen, but instead is loaded as a trainable adapter through PEFT and updated jointly with the query-selection MLPs. The same core hyperparameters were retained: 1 epoch, learning rate 1×10^{-4}, per-device batch size 1, effective batch size 64, cosine schedule, 3% warmup, weight decay 0, BF16, DeepSpeed ZeRO-2, and gradient clipping at 0.3.

Table 4.10: VSTI-Bench Results: Baseline vs Query-Based Frame Selection

	Avg.	*Cam-Obj Abs. Dist.*	*Cam. Displace.*	*Cam. Mov. Dir.*	*Obj-Obj Rel. Pos.*	*Cam-Obj Rel. Dist.*
		Numerical Answer		Multiple-Choice Answer		
Baseline	58.7	38.9	39.4	**60.7**	**86.1**	68.2
Query Selection + LoRA (128 → 32)	**58.8**	**39.4**	**39.6**	60.5	85.7	**71.8**
Query Selection + LoRA Continue (128 → 64)	56.6	36.5	34.1	56.2	85.0	71.3
Difference (Baseline vs (128 → 32))	+0.1	+0.5	+0.2	-0.2	-0.4	+3.6

4.7.2 Results

The query-based frame selection checkpoints were evaluated on VSTI-Bench against the reproduced VLM-3R baseline. Results are shown in Table 4.10. The initial 128-to-32 query-selection configuration was the stronger of the two query-selection runs, preserving baseline performance overall while improving camera-object relative distance from 68.2 to 71.8.

The later continuation run, which jointly unfroze the task LoRA and expanded selection to 128-to-64, performed worse overall at 56.6. Relative to the earlier query-selection run, it was worse on both numerical categories, falling from 39.4 to 36.5 on camera-object absolute distance and from 39.6 to 34.1 on camera displacement. It also dropped on camera movement direction from 60.5 to 56.2 and slightly on object-object relative position from 85.7 to 85.0. However, camera-object relative distance remained strong at 71.3, showing that the continuation preserved some of the retrieval benefit even though it did not translate into a stronger overall VSTI score.

Overall, query-based selection appears more promising than trainable memory in this study, but the continuation result suggests that increasing the selected frame count and jointly adapting the task LoRA did not improve robustness across categories.

4.8 Combining Approaches

After evaluating trainable memory and query-based selection separately, a joint continuation run was configured to optimise both modules together while also allowing the VSTI task LoRA to adapt. This is the correct training regime for the full combined architecture, because the query-selection module changes the distribution, redundancy, and temporal density of the frame features that the memory module receives. A memory module trained only on uniformly sampled 32-frame inputs is therefore not guaranteed to remain optimal once it is fed 32 query-selected frames from an initial 128-frame candidate pool.

The combined training configuration was launched on Isambard-AI Phase 2 using 16 nodes with 4 GH200 Grace Hopper Superchips per node, again yielding 64 accelerators in total. The run used the same base model, `lmms-lab/LLaVA-NeXT-Video-7B-Qwen2`, but was warm-started from two previously trained components: a query-selection + VSTI LoRA checkpoint, used as the main trainable adapter base, and an overlaid trainable-memory checkpoint. This warm-start strategy preserves the strongest available state for each subsystem before joint optimisation begins.

Our implementation freezes the whole model first, then selectively unfroze three groups of parameters: the task-specific LoRA adapter parameters, the query-selection module parameters, and the dual-memory parameters. The vision tower, spatial tower, fusion block, multimodal projector, and base language-model weights remained frozen.

The hyperparameters for the joint run were intentionally conservative so that optimisation would re-

Table 4.11: VSTI-Bench Results: Baseline vs Query Selection vs Combined Query + Memory

	Avg.	Cam-Obj Abs. Dist.	Cam. Displace.	Cam. Mov. Dir.	Obj-Obj Rel. Pos.	Cam-Obj Rel. Dist.
		Numerical Answer		Multiple-Choice Answer		
Baseline	58.7	38.9	39.4	**60.7**	**86.1**	68.2
Query Selection (128 → 32)	**58.8**	**39.4**	**39.6**	60.5	85.7	**71.8**
Combined Query + Memory	47.3	27.5	29.7	43.5	72.5	63.2
Difference vs Baseline	-11.4	-11.4	-9.7	-17.2	-13.6	-5.0

fine the interaction between already-trained subsystems rather than relearn them from scratch: 1 epoch, learning rate 1×10^{-5}, per-device batch size 1, and gradient accumulation of 2, corresponding to an effective batch size of 128 across 64 GPUs. Up to 128 frames were sampled per video (`frames_upbound=128`, `force_sample=True`), after which query selection retained 32 frames (`query_selection_num_select=32`) before these selected frames were passed through the dual-memory path. The remaining optimisation settings matched the large-scale component runs: BF16 mixed precision, DeepSpeed ZeRO-2, cosine learning-rate decay, 3% warmup, weight decay 0, gradient clipping at 0.3, gradient checkpointing, `dataloader_num_workers=0`, and `dataloader_drop_last=True`.

4.8.1 Results

The jointly trained query-selection and trainable-memory checkpoint was evaluated on VSTI-Bench against the reproduced VLM-3R baseline and the best standalone query-selection run. Results are shown in Table 4.11. The combined model performed substantially worse than both references, reaching only 47.3 on the VSTI-Bench paper-average.

The degradation was broad rather than isolated to a single task. Relative to the baseline, camera-object absolute distance fell from 38.9 to 27.5, camera displacement fell from 39.4 to 29.7, and camera movement direction fell from 60.7 to 43.5. The multiple-choice categories also weakened, with object-object relative position dropping from 86.1 to 72.5 and camera-object relative distance dropping from 68.2 to 63.2.

Compared to the stronger standalone query-selection configuration, the joint run also regressed on every category. This suggests that, in the present implementation, coupling trainable memory with query-based frame selection introduced additional interference rather than complementary benefit. The result therefore does not support the hypothesis that combining the two mechanisms in this training regime improves spatial-temporal reasoning performance, at least without end-to-end training.

Chapter 5

Results Discussions

Our main mode of evaluation will come from two benchmarks: VSI-Bench and VSTI-Bench [26]. VSI-Bench provides a spatial intelligence benchmark comprised QA pairs that make use of the scannet, scannet++ and Arkitscenes datasets. VSTI-Bench provides a spatial and temporal intelligence benchmark comprised of QA pairs that make use of the scannet dataset. Note that due to time and compute constraints not all modules could be trained for and evaluated on VSI-Bench (See Section 6.2 for more detail).

5.1 Core VSI and VSTI Bench Results

Tables 5.1 and 5.2 summarise the main benchmark results against proprietary models, open-source VLMs, and spatially enhanced baselines. On both benchmarks, reproducing VLM-3R was an important first step: our reproduced baseline matched the reported performance closely, giving 60.9 on VSI-Bench and 58.8 on VSTI-Bench.

The best result we obtained on VSI-Bench came from the non-trainable memory family, but this was effectively tied with the reproduced VLM-3R baseline rather than a meaningful improvement. On VSTI-Bench, the strongest result came from query-based frame selection, which matched the reproduced baseline overall while improving camera-object relative distance. By contrast, the trainable-memory and combined query-plus-memory variants both underperformed the reproduced baseline, with the combined model showing the largest degradation without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes. Taken together, these tables show that the most reliable gains in this project came from conservative modifications that preserved the baseline behaviour, while the more aggressively trainable extensions were harder to optimise successfully. Figure 5.1 provides a compact comparison of the strongest VSTI-Bench variant obtained for each method family explored in this project: non-trainable memory, trainable memory, query-based frame selection, and the full combined architecture. This gives a more immediate visual summary of how the different approaches trade off across the five VSTI task categories before the detailed discussion of each method family below.

5.1.1 Non-Trainable Memory

These initial non-trainable results shown in Table 4.3 and Table 4.4 already show a very clear asymmetry between working and episodic memory. On VSI-Bench, working memory alone is strongly harmful, dropping overall performance from 60.9 to 48.2. The largest degradations occur on tasks that depend on stable global scene structure rather than only very recent observations: room size falls from 67.5 to 25.3, absolute distance from 49.4 to 34.5, and relative distance from 65.1 to 50.2. Object counting and object size also degrade substantially. By contrast, appearance order improves from 40.5 to 46.2, which is consistent with working memory being biased toward recent local temporal evidence while being poor at preserving a coherent scene-level representation across the full sequence.

This pattern is plausible given VLM-3R's architecture. CUT3R and the Spatial-Visual-View fusion block already provide a geometry-aware representation that is accumulated recurrently and then aligned with the language model. Injecting a second, non-trainable FIFO memory on top of this pipeline does not add a new source of geometry; it mainly reweights recent token history. That is helpful for sequence-local

Table 5.1: **Evaluations on VSI-Bench [92] for 3D spatial reasoning tasks.** We compare against proprietary models, open-sourced VLMs, and spatial-enhanced models specifically designed for spatial reasoning. Bold indicates the best performance.

Methods	Avg.	Numerical Question				Multiple-Choice Question			
		Obj. Cnt.	Abs. Dist.	Obj. Size	Room Size	Rel. Dist.	Rel. Dir.	Route Plan	Appr. Order
Proprietary Models (API)									
GPT-4o	34.0	46.2	5.3	43.8	38.2	37.0	41.3	31.5	28.5
Gemini-1.5 Flash	42.1	49.8	30.8	53.5	**54.4**	37.7	41.0	31.5	**37.8**
Gemini-1.5 Pro	**45.4**	**56.2**	**30.9**	**64.1**	43.6	**51.3**	**46.3**	**36.0**	34.6
Open-sourced VLMs									
LongVA-7B	29.2	38.0	16.6	38.9	22.2	33.1	**43.3**	25.4	15.7
InternVL2-8B	34.6	23.1	**28.7**	48.2	**39.8**	36.7	30.7	29.9	39.6
InternVL2-40B	36.0	34.9	26.9	46.5	31.8	42.1	32.2	34.0	39.6
LongVILA-8B	21.6	29.1	9.1	16.7	0.0	29.6	30.7	32.5	25.5
VILA-1.5-8B	28.9	17.4	21.8	50.3	18.8	32.1	34.8	31.0	24.8
VILA-1.5-40B	31.2	22.4	24.8	48.7	22.7	40.5	25.7	31.5	32.9
Qwen2.5VL-7B	33.0	40.9	14.8	43.4	10.7	38.6	38.5	33.0	29.8
Qwen2.5VL-72B	37.0	25.1	29.3	54.5	38.8	38.2	37.0	34.0	28.9
LLaVA-OneVision-7B	32.4	47.7	20.2	47.4	12.3	42.5	35.2	29.4	24.4
LLaVA-OneVision-72B	40.2	43.5	23.9	**57.6**	37.5	42.5	39.9	32.5	44.6
LLaVA-NeXT-Video-7B	35.6	48.5	14.0	47.8	24.2	**43.5**	42.4	34.0	30.6
LLaVA-NeXT-Video-72B	**40.9**	**48.9**	22.8	57.4	35.3	42.4	36.7	**35.0**	**48.6**
Spatial-Enhanced Models									
VG-LLM-4B	47.3	66.0	37.8	55.2	59.2	44.6	45.6	33.5	36.4
VG-LLM-8B	50.7	67.9	37.7	58.6	62.0	46.6	40.7	32.4	**59.2**
Spatial-MLLM-4B	48.4	65.3	34.8	63.1	45.1	41.3	46.2	33.5	46.3
VLM-3R-7B	**60.9**	**70.2**	**49.4**	69.2	67.1	65.4	**80.5**	**45.4**	40.1
Ours: Query Selection (best)	43.9	65.9	38.4	58.2	44.5	43.1	49.0	37.1	14.9
Ours: Non-Trainable Memory (best)	**60.9**	70.1	49.3	**69.4**	**67.7**	**65.5**	79.8	**45.4**	40.0
Difference	+0.0	-0.1	-0.1	+0.2	+0.6	+0.1	-0.7	+0.0	-0.1

Table 5.2: **Evaluations on VSTI-Bench [26] for 3D spatial-temporal reasoning tasks.** *Abbr.:* CO-AbsD = Cam-Obj Abs. Dist.; CDisp = Cam. Displace.; CMDir = Cam. Mov. Dir.; OO-RelP = Obj-Obj Rel. Pos.; CO-RelD = Cam-Obj Rel. Dist.

Methods	Avg.	Numerical Question		Multiple-Choice Question		
		CO-AbsD	CDisp	CMDir	OO-RelP	CO-RelD
Proprietary Models (API)						
GPT-4o	**38.2**	**29.5**	**23.4**	**37.3**	**58.1**	**42.5**
Gemini-1.5 Flash	32.1	28.5	20.9	24.4	52.6	33.9
Open-sourced VLMs						
LongVA-7B	32.3	13.5	5.1	43.7	57.9	41.2
InternVL2-8B	43.5	**32.9**	13.5	48.0	68.0	55.0
InternVL2-40B	43.2	11.9	**34.9**	33.3	63.8	**72.2**
LongVILA-8B	30.5	20.0	11.6	35.4	52.3	33.4
VILA-1.5-8B	37.3	30.1	27.3	42.2	50.4	36.7
VILA-1.5-40B	38.2	28.2	15.7	28.8	65.4	53.0
Qwen2.5VL-7B	38.2	22.9	4.9	47.4	65.9	49.9
Qwen2.5VL-72B	40.3	18.0	10.0	41.0	74.2	58.4
LLaVA-OneVision-7B	41.7	29.9	19.3	47.5	62.1	49.8
LLaVA-NeXT-Video-7B	40.0	28.2	1.8	**49.8**	64.7	55.6
LLaVA-NeXT-Video-72B	**44.0**	32.3	10.5	48.1	**78.3**	50.9
Spatial-Enhanced Models						
VLM-3R-7B	**58.8**	**39.4**	39.6	**60.6**	**86.5**	68.6
Ours: Combined Query + Memory	47.3	27.5	29.7	43.5	72.5	63.2
Ours: Trainable Memory (best)	54.1	35.9	38.2	56.7	71.4	68.2
Ours: Non-Trainable Memory (best)	**58.8**	**39.4**	**39.7**	60.4	86.2	68.1
Ours: Query Selection (best)	**58.8**	**39.4**	39.6	60.5	85.7	**71.8**
Difference	+0.0	+0.0	+0.0	-0.1	-0.8	+3.2

tasks such as appearance order, but harmful for tasks such as room size, absolute distance, and route planning, where the model must preserve a globally consistent spatial layout rather than over-trust the most recent views.

Episodic memory behaves very differently. On VSI-Bench it is essentially neutral relative to the baseline, with tiny gains on object count, object size, relative distance, and room size, and only very small losses elsewhere. This suggests that similarity-based long-term storage is not actively damaging the representation, but in its non-trainable form it also does not add enough useful structure to produce a clear benchmark improvement. Equal-weight fusion inherits part of the weakness of working memory:

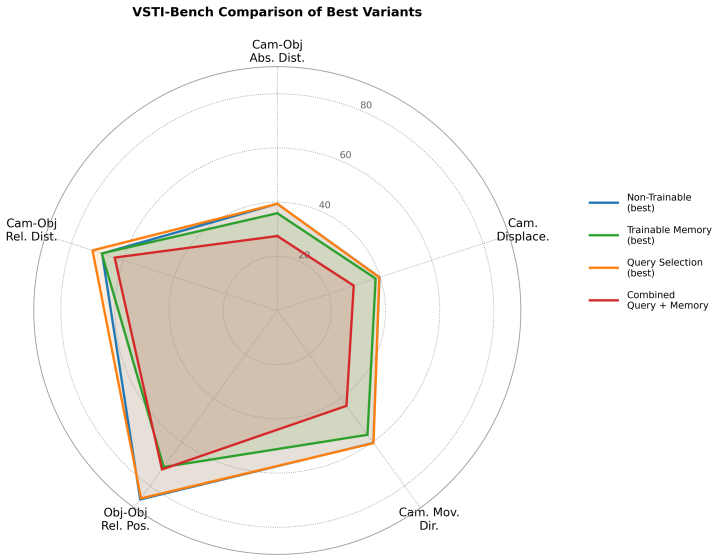


Figure 5.1: Radar-chart comparison of the best-performing VSTI-Bench variant from each method family explored in this project: non-trainable memory, trainable memory, query-based frame selection, and the combined query-selection plus trainable-memory architecture. The chart highlights that the strongest standalone query-selection and non-trainable variants remain close to the reproduced baseline, while the combined architecture degrades broadly across categories.

although it is much less harmful than working memory alone, it still underperforms the baseline overall because the unstable short-term component contaminates retrieval.

On VSTI-Bench, the effect of non-trainable memory is even smaller. All three initial variants remain essentially flat relative to the baseline, indicating that fixed memory retrieval by itself neither meaningfully helps nor catastrophically harms the temporal-spatial benchmark. This is also compatible with the benchmark design: many VSTI questions are frame-indexed or ask about camera motion between specified points, so CUT3R's existing recurrent state and VLM-3R's geometry-aware fusion already provide much of the temporal structure the task demands. Taken together, these initial results suggest that the main risk in the non-trainable design is not long-term storage, but naive short-term retrieval from a FIFO working buffer without any calibration. The alpha-blending ablation reinforces this interpretation. For working memory, reducing the memory contribution restores most of the lost VSI-Bench performance: with $\alpha = 0.1$, overall accuracy recovers to 60.7, almost matching the 60.9 baseline, whereas performance collapses steadily as α increases toward 1.0. This pattern is especially clear on room size, absolute distance, and relative distance, which are the categories most damaged by full working-memory replacement. In other words, the working-memory signal is only tolerable when it acts as a small residual perturbation on top of the original features rather than a full substitute for them.

Episodic memory is almost invariant to α. Across all blending values, VSI-Bench remains effectively unchanged, which supports the view that episodic retrieval in this fixed form is largely passive: it does not inject enough noise to hurt the model, but it also does not contribute enough task-specific information to help substantially. Fusion sits between these two extremes. At low α, it remains close to baseline; at high α, it degrades in much the same direction as working memory alone, again showing that the weak component in the non-trainable design is the short-term buffer rather than the episodic store.

The VSTI-Bench buffer-size ablation tells a similar story. Changing (L_w, L_e) among the tested settings leaves the score almost unchanged. This indicates that, for the non-trainable module, performance is limited primarily by the quality of retrieval and fusion rather than by memory capacity. In practical terms, simply storing more frames does not solve the problem if the model has no learned mechanism for deciding what to read and how strongly to trust it.

5.1.2 Trainable Memory

Looking at Table 4.9, we see that adding learned attention and learned fusion did not improve over the reproduced VLM-3R baseline, and in fact made performance substantially worse on VSTI-Bench without end-to-end fine tuning. The initial trainable-memory configuration drops from 58.7 to 53.8 overall, and the continuation run with a smaller working-memory buffer recovers only slightly to 54.1. This means that introducing trainable memory did not merely fail to provide a gain; it moved the model into a consistently weaker operating regime. It should be noted that while we ablated working memory buffer sizes, we were not able to check a comprehensive set of buffer sizes for episodic memory due to time and compute constraints.

The degradation is broad: the most severe drop is on object-object relative position, which falls from 86.1 in the baseline to 68.8 in the first trainable-memory run and only partially recovers to 71.4 after reducing L_w from 8 to 4. Camera-object absolute distance and camera displacement also decline, while camera movement direction becomes less reliable. The only category that avoids clear regression is camera-object relative distance, where the first trainable-memory run shows a very small gain that disappears in the continuation setting. This pattern suggests that the learned memory path is not helping the model build a better global scene representation; instead, it is disrupting the geometric information already encoded effectively by the baseline VLM-3R pipeline. Architecturally, this makes sense: VLM-3R already combines CUT3R state with visual tokens through a dedicated cross-attention fusion module, so a second trainable retrieval-and-fusion stage may compete with, rather than complement, the model's existing mechanism for aligning appearance with geometry.

5.1.3 The Effect of Trainable Parameters

Comparing the non-trainable and trainable variants clarifies that the problem is not simply "memory versus no memory", but the interaction between additional trainable parameters and the already strong VLM-3R representation. The best non-trainable variants remain close to baseline on both benchmarks, and in some cases match it exactly. By contrast, once memory retrieval and fusion become trainable, performance falls sharply. This means that additional flexibility is not helping the model adapt; instead, it is making optimisation harder and encouraging the network to rely on a memory path that is less stable than the original frozen spatial-reasoning pipeline.

The training supports the same conclusion. Although the exploding-gradient issue was eventually stabilised, stable optimisation alone was not sufficient to produce a better model without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes. In effect, trainability increased capacity without improving inductive bias. The non-trainable designs were limited but safe, whereas the trainable designs were expressive but poorly aligned with the task. For this project, the evidence therefore favours the simpler interpretation: preserving the original VLM-3R representation is more important than learning a richer memory mechanism on top of it. In benchmark terms, both VSI and VSTI reward geometric consistency more than representational flexibility; when the added trainable components disturb that consistency, the cost is larger than the potential benefit without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes.

With more time and compute, training the model end to end with everything unfrozen could have resolved this issue by allowing the spatial and visual tower (encoders) to adapt to the new memory system.

5.1.4 Query Based Frame Selection

Query-based frame selection is the only method family in this project that produces a clear positive result on the main benchmark tables. Table 4.10 show that on VSTI-Bench, the best query-selection run matches the reproduced baseline overall at 58.8 while improving camera-object relative distance from 68.2 to 71.8. Numerical categories remain effectively unchanged, and the small losses on camera movement direction and object-object relative position are minor compared with the gain in relevance-based retrieval. This is a more encouraging outcome than either non-trainable memory, which is largely neutral, or trainable memory, which underperforms without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes.

Table 5.1 shows that VSI-Bench result is weaker. Query selection performs well below the baseline

overall and is worse on most categories, with particularly large losses on appearance order and route planning. This suggests that relevance filtering is more naturally aligned with VSTI-Bench, where the questions often target specific frames or temporally localised camera-object relationships, than with VSI-Bench, where the model frequently needs a more complete global picture of the scene. In other words, question-conditioned filtering helps when the answer depends on isolating the right moments, but it can remove useful context when the task depends on broader scene coverage. This benchmark difference is important: VSI categories such as room size, route planning, and object counting are closer to whole-scene reconstruction problems, whereas VSTI more often asks about localised relations between the camera and a small set of entities at named frames.

The continuation run further sharpens this interpretation. Expanding selection from 128-to-32 to 128-to-64 and jointly adapting the task LoRA reduces overall VSTI-Bench performance to 56.6. The strong camera-object relative-distance result survives, but the numerical categories degrade noticeably. This implies that the benefit of query selection is not simply "more selected frames"; there appears to be a narrow operating point where filtering removes redundancy without reintroducing enough irrelevant content to dilute the gain. From an architectural perspective, this is also plausible: once too many frames are reintroduced, the downstream VLM receives something closer to the original redundant sequence but with an additional learned selection layer inserted in front of it, so the system pays the complexity cost without preserving the sharper relevance signal of the 128-to-32 setting.

This is somewhat surprising given that the VSTI benchmark numbers frames and in its question and our process of query based may disturb this evaluation process. This is expanded on further in Section 5.9.

5.1.5 Full architecture

The combined architecture provides the clearest negative result in the study as shown in Table 4.11. When query selection and trainable memory are optimised together, the model falls to 47.3 on VSTI-Bench, far below both the reproduced baseline and the strongest standalone variants without end-to-end fine tuning and without being able to check a comprehensive set of buffer sizes. Every major category degrades, with especially large losses on camera-object absolute distance, camera displacement, and camera movement direction. Even object-object relative position and camera-object relative distance, which remain comparatively stronger than the numerical categories, are still worse than in the best standalone query-selection configuration.

This matters because the two components looked superficially complementary: query selection should reduce redundancy before memory operates, and memory should then preserve the most useful information over time. The benchmark evidence does not support that intuition. Instead, the joint model appears to amplify the weaknesses of both components. Query selection changes the distribution and density of frame evidence reaching the memory module, while trainable memory then learns on top of a moving target and interferes with the geometry-aware baseline representation. The end result is a model that is more complex, harder to optimise, and less accurate. In effect, the combined model inserts two new decision points between CUT3R-derived spatial features and the language model: one decides which frames survive, and the other decides how those selected frames are reread and fused. Errors made at either stage propagate forward, and the benchmark results suggest that this extra control path is less reliable than the original VLM-3R fusion mechanism.

Taken together with the base VSI and VSTI tables, the combined result strengthens the central empirical conclusion of the project: architectural complexity alone did not improve benchmark spatial reasoning. The strongest outcomes came either from preserving the baseline almost unchanged, as in the non-trainable variants, or from applying a lightweight relevance-filtering mechanism, as in the best query-selection run. Once both mechanisms were made jointly adaptive, performance deteriorated rather than improved.

5.2 Long-horizon spatial reasoning performance

By partitioning the input videos into VSI-Bench [92] and VSTI-Bench [26] according to their duration (Short (less than 1 min), Mid (1-2 min) and Long (greater than 2 min)), we can evaluate long-horizon spatial reasoning directly. Table 5.3 shows the comparisons. The broad pattern is different across the

Table 5.3: **Additional long-horizon spatial reasoning results on VSI-Bench [92] and VSTI-Bench [26].** Videos are grouped into Short (< 1 min), Mid (1–2 min), and Long (> 2 min). For Spatial-MLLM [88], the original setting uses 16 input frames for evaluation, while methods marked with * correspond to a 32-frame input setting, used to ensure a consistent frame setting across methods. For VSTI-Bench, the *Avg.* column follows the long-horizon convention from VLM-3R and uses a question-count-weighted average over the five top-level categories, rather than the unweighted five-category average used elsewhere in this report. Our VSTI-Bench Short, Mid, and Long rows are recomputed from the cached benchmark MP4 durations with thresholds < 60 s, 60–120 s, and > 120 s respectively.

Methods	VSI-Bench				VSTI-Bench			
	Avg.	Short	Mid	Long	Avg.	Short	Mid	Long
Open-sourced VLMs								
LongVA-7B [96]	31.1	33.5	32.2	28.8	38.2	36.9	38.4	40.5
InternVL2-8B [11]	34.6	35.8	33.8	34.9	49.1	46.2	49.9	53.3
InternVL2-40B [11]	36.1	38.5	36.0	35.1	50.3	46.0	51.6	56.2
LongVILA-8B [91]	21.2	20.1	21.0	21.9	34.3	34.6	33.9	34.7
VILA-1.5-8B [45]	30.7	35.1	31.9	27.4	40.9	39.8	42.1	40.6
VILA-1.5-40B [45]	31.6	33.3	32.3	30.1	44.8	41.9	45.1	50.0
Qwen2.5VL-7B [5]	29.5	34.3	28.3	28.4	45.9	43.3	48.4	45.5
Qwen2.5VL-72B [5]	36.3	39.2	36.6	34.7	49.4	45.1	50.8	55.2
LLaVA-OneVision-7B [42]	34.1	38.2	34.3	32.0	46.6	44.1	47.6	49.8
LLaVA-OneVision-72B [42]	**41.2**	**43.0**	**41.5**	40.2	**52.2**	**48.2**	**52.9**	**59.2**
LLaVA-NeXT-Video-7B [98]	36.8	38.5	37.3	35.5	46.4	43.0	47.9	50.1
LLaVA-NeXT-Video-72B [98]	41.1	41.3	40.7	**41.4**	51.3	47.6	51.9	57.6
Spatial-Enhanced Models								
Spatial-MLLM-4B [88]	48.1	49.8	48.3	47.2	-	-	-	-
Spatial-MLLM-4B* [88]	49.0	49.7	49.3	48.5	-	-	-	-
VG-LLM-4B [100]	47.4	49.1	48.7	45.2	-	-	-	-
VG-LLM-8B [100]	50.5	51.9	49.7	50.6	-	-	-	-
VLM-3R-7B [26]	**63.2**	**67.1**	**64.7**	**60.0**	**63.6**	**61.1**	64.7	**66.4**
Ours: Combined Query + Memory	-	-	-	-	52.8	46.2	54.6	60.8
Ours: Trainable Memory (best)	-	-	-	-	58.3	57.0	59.2	60.5
Ours: Non-Trainable Memory (best)	-	-	-	-	**63.6**	60.1	65.1	**65.6**
Ours: Query Selection (best)	43.9	49.1	43.5	41.0	**63.6**	60.1	**65.1**	65.6
Difference	-19.3	-18.0	-21.2	-19.0	+0.0	-1.0	+0.4	-0.8

two benchmarks. On VSI-Bench, performance declines as video duration increases, both for VLM-3R and for our query-selection variant, indicating that static spatial reasoning still becomes more difficult as the model must integrate information across longer trajectories. On VSTI-Bench, the picture is more nuanced: the reproduced VLM-3R baseline is strongest on long videos, and our best non-trainable and query-selection variants closely match that overall long-horizon profile, while trainable memory and the combined query-plus-memory model remain clearly below the baseline across all buckets.

For VSTI-Bench, the *Avg.* column does not use the same unweighted five-category average as the main results table. Instead, it follows the long-horizon reporting convention from VLM^2 [50]: the five top-level categories are first formed, then the final score is weighted by the number of questions in each category. The long-horizon results suggest three main conclusions. First, on VSI-Bench, query-based frame selection does not solve the long-sequence problem by itself: our best query-selection row remains well below VLM-3R at every duration bucket, with the largest gap appearing on mid and long videos. Second, on VSTI-Bench, the simpler non-trainable memory and the standalone query-selection configuration are both much more robust than the trainable alternatives, essentially matching the baseline weighted average of 63.6 and staying close to the baseline in each duration bucket. Third, the fully combined architecture performs worst on short and mid videos and recovers only partially on long videos, which is consistent with the earlier per-category results: in this project, combining memory and query selection introduced interference rather than complementary gains. Taken together, these results do not support the claim that the more complex trainable or combined variants improve long-horizon spatial-temporal reasoning over the reproduced VLM-3R baseline, but they do suggest that lightweight retrieval-style mechanisms can preserve baseline behaviour more effectively than heavier trainable memory modules.

Table 5.4: VSI-Bench (Tiny) Human Performance vs. Our Full-Benchmark Results

	Avg.	Obj. Count	Abs. Dist.	Obj. Size	Room Size	Rel. Dist.	Rel. Dir.	Route Plan	Appr. Order
		Numerical Answer				Multiple-Choice Answer			
Human Performance	**79.2**	**94.3**	**47.0**	60.4	45.9	**94.7**	**95.8**	**95.8**	**100.0**
Ours: Query Selection (best)	43.9	65.9	38.4	58.2	44.5	43.1	49.0	14.9	37.1
Ours: Non-Trainable Memory (best)	60.9	70.1	49.3	**69.4**	**67.7**	65.5	79.8	45.4	40.0

Table 5.5: VSTI-Bench (Tiny) Human Performance vs. Our Full-Benchmark Results

	Avg.	Cam-Obj Abs. Dist.	Cam. Displace.	Cam. Mov. Dir.	Obj-Obj Rel. Pos.	Cam-Obj Rel. Dist.
		Numerical Answer		Multiple-Choice Answer		
Human Performance	**77.0**	**51.4**	**46.8**	**95.1**	**97.5**	**94.3**
Ours: Combined Query + Memory	47.3	27.5	29.7	43.5	72.5	63.2
Ours: Trainable Memory (best)	54.1	35.9	38.2	56.7	71.4	68.2
Ours: Non-Trainable Memory (best)	58.8	39.4	39.7	60.4	86.2	68.1
Ours: Query Selection (best)	58.8	39.4	39.6	60.5	85.7	71.8

5.3 Comparison with Human Performance

VSI-Bench (Tiny) is a sampled subset of 400 questions (50 per category) of the VSI-Bench questions. Thinking in space reports on how human evaluators independently answered each question. Tables 5.4 and 5.5 compare the reported human Tiny scores against our full-benchmark results for the architectures evaluated in this project. These rows are therefore useful for qualitative comparison, but they are not a like-for-like evaluation on the exact same subset. The comparison with human performance on VSI-Bench (Tiny) reveals that our Non-Trainable Memory model achieves 60.9% overall compared to 79.2% for humans, an 18-point gap. However, this gap is not uniform across categories. Our model actually exceeds human performance on Object Size (69.4 vs 60.4) and Room Size (67.7 vs 45.9), suggesting that for these absolute metric tasks, the model's learned representations capture relevant spatial cues effectively. The largest gaps appear in Route Planning (45.4 vs 95.8) and Appearance Order (40.0 vs 100.0), indicating that temporal sequencing and navigation path reasoning remain fundamentally challenging and diverge significantly from human-level performance.

On VSTI-Bench (Tiny), our best model (Query Selection or Non-Trainable Memory, both at 58.8%) sits 18 points below human performance (77.0%). The pattern is more consistent than VSI-Bench: humans outperform our model across all five categories, with the smallest gap on Camera-Object Absolute Distance (39.7 vs 51.4, approximately 12 points) and the largest gap on Camera Movement Direction (60.4 vs 95.1, approximately 35 points). The contrast between our strong Camera-Object Relative Distance performance (68.1-71.8 vs 94.3 human) and weak Camera-Object Absolute Distance (39.7 vs 51.4) mirrors the absolute metric failure pattern observed throughout this project: relative spatial reasoning is substantially more tractable for our model than absolute metric estimation.

5.4 Extended Task Category Analysis

The evaluation results include additional subcategory metrics beyond the main 5 VSTI-Bench and 8 VSI-Bench task categories used in the paper. Analysis of the full per-sample results reveals significant variation in performance across task variants, with some categories achieving near-human-level performance while others fail completely.



Figure 5.2: Examples of reconstructed scenes from the ScanNet dataset (top) [18] and Arkitscenes dataset (bottom) [20].

5.4.1 Dataset-Level Performance Variation

VSI-Bench performance varies across the three source datasets. ScanNet++ achieves 40.7% (593/1458), ScanNet achieves 40.4% (837/2071), while Arkitscenes achieves only 33.5% (537/1601) on sample-level exact match (not using benchmark tolereances). The 7-point gap between Arkitscenes and the ScanNet variants reflects different data distributions, likely due to variations in video quality, scene complexity, or object recognition conditions. Looking at the examples given in Figure 5.2, we see that Arkitscenes scenes tend to hold more information and objects than the furniture dominant ScanNet scenes, supporting our idea. Arkitscenes also shows the lowest performance on room size estimation (1.3% vs 0.0% for ScanNet and ScanNet++), indicating this dataset presents the most challenging absolute metric reasoning conditions.

5.4.2 VSTI-Bench Extended Categories

Beyond the five primary VSTI-Bench categories, the full evaluation results include three camera-object relative distance variants (v1, v2, v3), three object-object relative position variants (near-far, up-down, left-right), and two numerical task types: camera displacement and camera-object absolute distance.

For multiple-choice questions (all except the numerical types), accuracy is measured by comparing the model's letter prediction against the expected letter in `mc_answer`. The object-object relative position variants show strong performance with variation across spatial axes: up-down achieves 94.2% (456/484), near-far achieves 84.0% (467/556), and left-right achieves 80.5% (487/605). This suggests the model reasons about vertical spatial relationships more accurately than horizontal ones, potentially due to floor and ceiling landmarks anchoring vertical orientation.

The camera-object relative distance variants show moderate performance: v1 achieves 61.8% (68/110), v2 achieves 66.3% (391/590), and v3 achieves 76.1% (791/1040), where accuracy here means the predicted letter matches `mc_answer`. Camera movement direction achieves 60.5% (552/913) against `mc_answer`. For numerical questions, accuracy is measured against `ground_truth` (the actual number). Camera displacement achieves only 6.0% (50/839) and camera-object absolute distance achieves 4.2% (38/905), indicating systematic failure on metric estimation tasks. As discussed in Section 5.10, the near-zero performance on these numerical tasks reveals a fundamental limitation of the model architecture for absolute metric reasoning (despite the benchmark allowing for some flexibility).

5.4.3 VSI-Bench Extended Categories

The VSI-Bench results reveal a similar pattern of relative spatial reasoning outperforming absolute measurements, along with significant variation across difficulty levels within task categories.

The relative direction category shows substantial performance variation by difficulty level. Easy variants achieve 83.1-84.8% accuracy, while the hard variant drops to 71.3%. This suggests that multi-hop or fine-grained directional reasoning (e.g., front-left vs. back-right) is considerably more challenging than simple left/right/front/back judgments.

Relative distance achieves 65.2% (463/710), indicating moderate capability for comparative distance reasoning.

Appearance order tasks achieve 40.5% (250/618), consistent with the primary category metric, indicating difficulty with temporal sequencing of object appearances.

Route planning achieves 45.4% (88/194), also consistent with primary metrics.

Four task types achieve near-zero accuracy: absolute distance (5.8%, 48/834), object counting (56.1% but with systematic under-count bias), object size estimation (3.7%, 35/953), and room size estimation (0.7%, 2/288). Object counting shows a pronounced under-count bias: of 248 failure cases, 209 under-counted versus only 39 over-counted. The model predicted "2" in 127 of these failures, suggesting it defaults to conservative counts when uncertain (despite the benchmark allowing for some flexibility).

5.5 Uncertainty and Paired Significance

To test whether the differences between architectures are likely to reflect real behavioural changes rather than evaluation noise, I computed 95% bootstrap confidence intervals from per-sample benchmark results using the same benchmark-weighting scheme as the main tables. I also ran paired randomisation tests against the reproduced baseline.

Figure 5.3 shows that the reproduced baseline, non-trainable memory, and query-selection run on VSTI are statistically indistinguishable overall. The baseline scores 58.77 with a 95% interval of [57.61, 59.91], non-trainable memory scores 58.75 [57.61, 59.92], and query selection scores 58.76 [57.52, 59.92]. The paired deltas relative to baseline are correspondingly tiny: -0.02 points for non-trainable memory (95% CI [-0.12, 0.07], $p = 0.627$) and -0.01 points for query selection (95% CI [-0.10, 0.07], $p = 0.786$). By contrast,

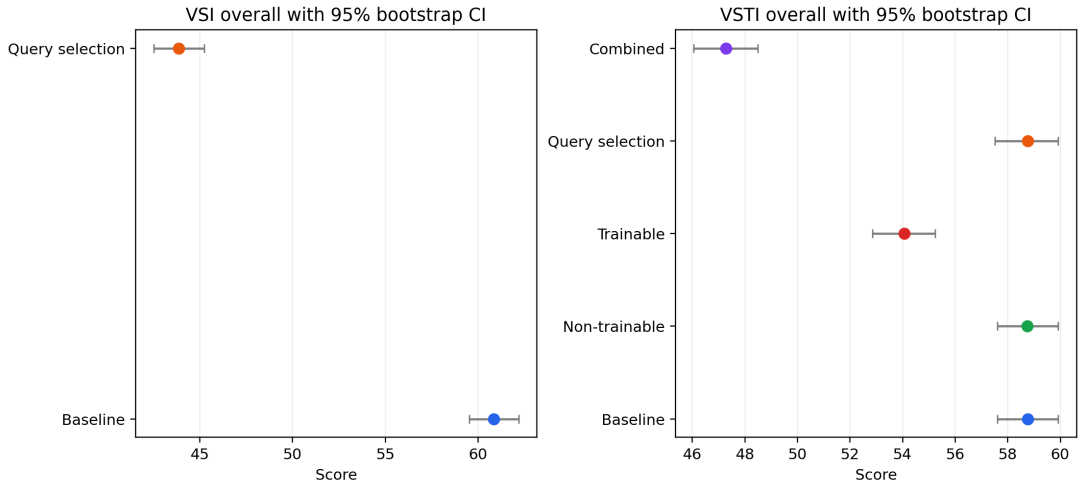


Figure 5.3: Bootstrap 95% confidence intervals for the overall benchmark scores computed from the saved per-sample logs using the same weighting scheme as the main benchmark tables. On VSTI, the reproduced baseline, non-trainable memory, and the final query-selection run overlap heavily, while trainable memory and the combined architecture are clearly separated below them.

the trainable-memory variant is significantly worse at -4.71 points (95% CI [-5.91, -3.46], $p < 0.001$), and the combined query-plus-memory model is worse still at -11.49 points (95% CI [-12.87, -10.11], $p < 0.001$).

The same analysis reinforces how unstable query selection is on VSI-Bench. The reproduced baseline scores 60.84 [59.51, 62.17], while the final query-selection run falls to 43.87 [42.53, 45.24]. The paired delta is -16.97 points with a 95% CI of [-18.65, -15.26] and $p < 0.001$. This is useful because it turns the qualitative interpretation of the core tables into a more defensible claim: on VSTI, the conservative variants preserve baseline behaviour, whereas on VSI the available query-selection run is decisively worse.

5.6 Scene Complexity Split

The saved ScanNet scene metadata makes it possible to test whether architectural differences become larger in more cluttered scenes. I use the total number of annotated object instances in each ScanNet scene as a simple proxy for scene complexity, then split VSTI-Bench scenes into low-, mid-, and high-complexity tertiles. Because the benchmark's question types are not distributed uniformly across scenes, this analysis uses the mean question-level benchmark score within each complexity bin rather than the five-category paper average.

Figure 5.4 shows a consistent pattern. Scores rise from low- to high-complexity bins for all architectures, which suggests that the low-complexity split is not simply the "easy" split in the intuitive sense; rather, question distribution and scene structure interact in a more complicated way. More importantly, the relative ordering of the architectures is stable across the three bins. Baseline, non-trainable memory, and the final query-selection run remain nearly indistinguishable throughout, while trainable memory is consistently below them and the combined architecture is consistently the weakest. The combined model is especially poor in the low-complexity bin at 39.2, but it remains far below the baseline even in the high-complexity bin (59.3 vs. 66.3). This suggests that the regressions introduced by trainable memory and by the combined architecture are not confined to cluttered scenes; they are more plausibly consequences of the augmentation strategy itself.

5.7 Bias and Confusion Across Architectures

The numerical tasks reveal architecture-specific biases that are not obvious from the headline benchmark scores alone. Figure 5.5 plots mean relative error by numerical task. On VSI-Bench, both the reproduced baseline and the query-selection run under-count object numbers on average, but query selection makes this bias substantially worse: its mean relative error on object counting is -0.19 compared with -0.09 for

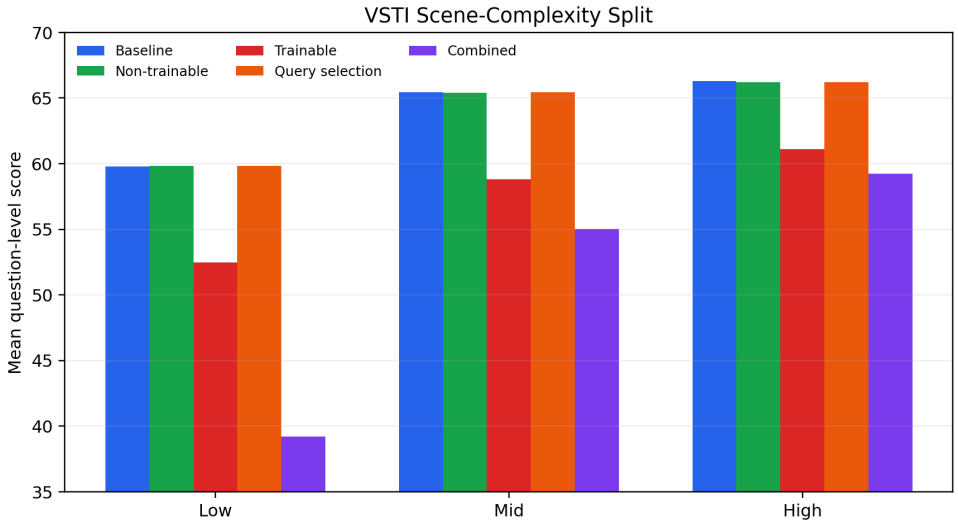


Figure 5.4: VSTI-Bench question-level score split by ScanNet scene complexity, using total object instances as the complexity proxy. Baseline, non-trainable memory, and the final query-selection run remain tightly aligned across all bins, while trainable memory and especially the combined model underperform throughout.

the baseline, and it under-counts 250 samples while over-counting only 4. Query selection also worsens room-size underestimation, with a mean raw error of -5.96 compared with -1.79 for the baseline. On VSTI-Bench, baseline, non-trainable memory, and the final query-selection run are again almost identical on the numerical tasks, whereas the combined architecture shows a strong positive bias, over-estimating both camera displacement and camera-object absolute distance. Its mean relative errors are 0.61 and 0.79 respectively, with over-estimates outnumbering under-estimates in both categories.

The multiple-choice tasks show a different kind of failure: option-position confusion. Figure 5.6 shows that the reproduced baseline, non-trainable memory, and the final query-selection run produce nearly identical VSTI confusion patterns, which is further evidence that these conservative variants preserve the original decision policy. Trainable memory broadens the dominant A/B confusions, reducing true-A accuracy from 0.80 to 0.71 and true-B accuracy from 0.72 to 0.65. The combined architecture is much worse: true-D accuracy falls to 0.30 and true-C accuracy to 0.50, with strong drift into A/B predictions. The available VSI comparison tells the same general story in a more extreme form. The baseline achieves diagonal option accuracies of approximately 0.63, 0.67, 0.58, and 0.63 across A–D, whereas the query-selection run drops these to 0.36, 0.47, 0.31, and 0.35, indicating that its degradation is not localised to one task family but reflects a broad collapse in multiple-choice calibration.

5.8 Cross-Architecture Failure Comparison

Figure 5.7 compares each augmentation directly against the reproduced baseline at the question-type level. This is a useful complement to the broader failure-mode discussion later in the chapter because it highlights not just where the model is weak, but where each augmentation makes it weaker or stronger.

On VSI-Bench, the query-selection comparison is worse than the baseline on every question type. The largest losses occur on relative-direction reasoning, where the easy and medium variants fall by -36.4 and -31.2 points respectively, followed by appearance order, room size, and relative distance. This is a strong signal that the query-selection VSI run is not merely failing on numerical estimation: it is disrupting both spatial comparison and temporal ordering tasks.

On VSTI-Bench, non-trainable memory remains effectively flat relative to baseline, with deltas clustered at or near zero across all nine sub-types. Trainable memory is much more selective in how it fails. Its largest regression is on object-object relative position left/right (-34.2 points), followed by object-object near/far (-7.9), camera movement direction (-3.7), and camera-object absolute distance (-3.5),

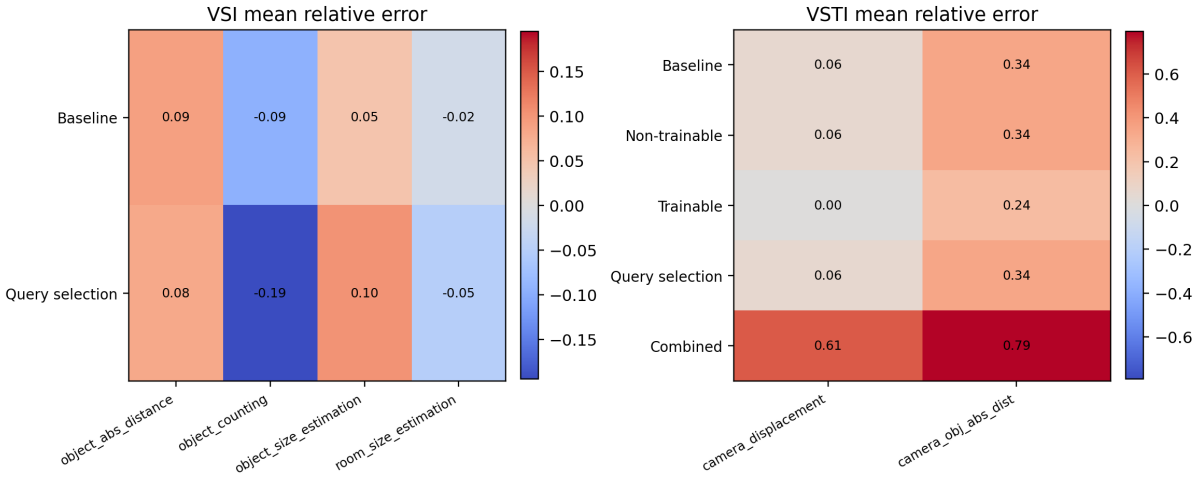


Figure 5.5: Mean relative error on the numerical task families for the benchmark runs. Negative values indicate under-estimation and positive values indicate over-estimation. Query selection worsens VSI under-counting and room-size under-estimation, while the combined VSTI architecture develops a strong over-estimation bias on both numerical categories.

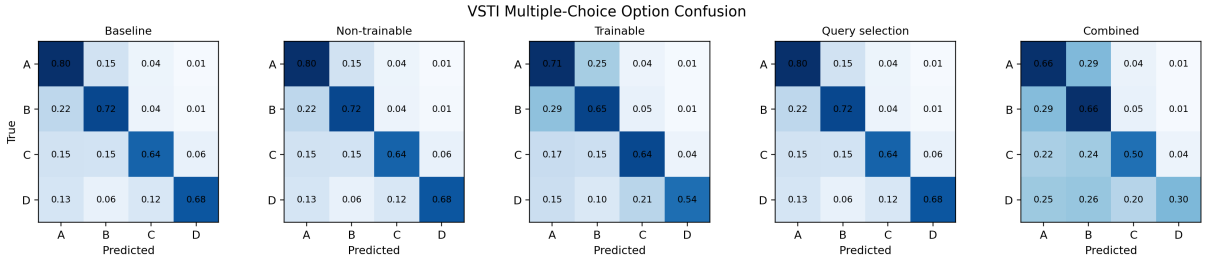


Figure 5.6: Row-normalised true-versus-predicted option confusion for the VSTI-Bench multiple-choice tasks. Baseline, non-trainable memory, and the final query-selection run have nearly identical confusion structure, while trainable memory and especially the combined architecture broaden the dominant A/B confusions and sharply reduce recovery of the rarer answer positions.

while camera-object relative distance v1 is the only subtype with a small positive shift (+4.6). The combined model is broader and more destructive: it loses heavily on object-object relative position left-/right (-31.7), camera movement direction (-17.0), camera-object absolute distance (-11.9), and camera displacement (-10.0), and it is worse than the baseline on every listed VSTI subtype. Taken together, this suggests that the learned augmentations do not mainly fail by weakening numerical estimation alone; they more fundamentally destabilise camera-relative relational reasoning, particularly left/right reasoning between objects.

5.9 Frame Selection Analysis

To inspect what query-based frame selection actually does, I added a diagnostic trace to the evaluation code which records the cosine score assigned to every candidate frame, the top-V selected frame indices, and the corresponding original video frame numbers. This was run on a small VSTI-Bench diagnostic subset using the trained query-selection model with 128 uniformly sampled candidate frames and $V = 32$ selected frames. The trace is qualitative rather than a full benchmark result, but it is useful for understanding the mechanism behind the aggregate scores.

Figure 5.8 shows one representative trace. The question asks about *frame 23 of 32*; when mapped onto the 128-frame candidate sequence used by query selection, this corresponds approximately to candidate frame 90. The dashed vertical line marks this frame. The green markers show the frames selected by the query-selection module according to cosine similarity between the encoded question and each encoded

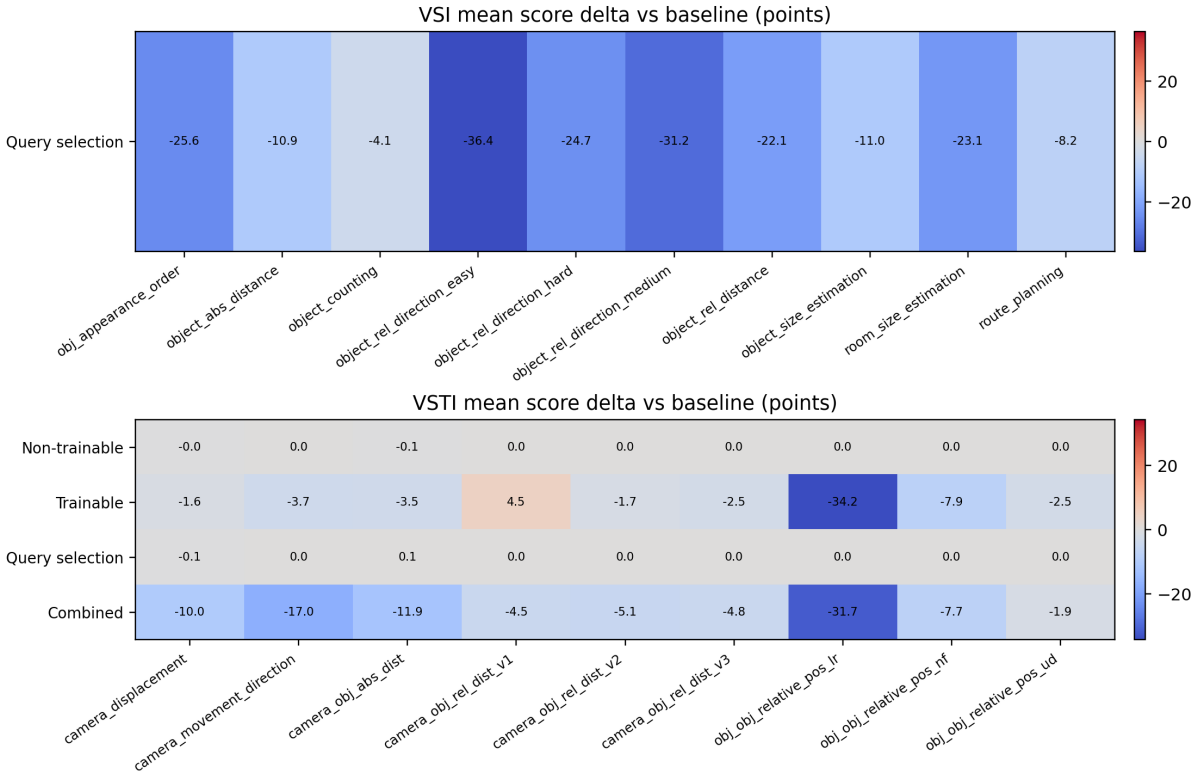


Figure 5.7: Mean score delta, in percentage points, versus the reproduced baseline for each architecture at the question-type level. Non-trainable memory stays essentially flat on VSTI, whereas trainable memory and especially the combined model introduce concentrated losses in object-object relative position and camera-motion reasoning. The VSI query-selection run is worse across every listed question type.

frame indicator. In this example, the exact frame named in the question is not selected. Instead, the selector keeps several high-scoring frames from earlier parts of the video, plus later frames where related objects remain visible.

This illustrates an important tension in applying query-based frame selection to VSTI-Bench. VSTI questions often refer to a specific frame number in a 32-frame sequence, whereas our query-selection configuration samples up to 128 frames and then selects 32 according to question-frame similarity. In the six unique examples traced, the exact mapped frame was selected only once; however, the nearest selected frame was typically close to the target, with a median offset of 4.5 candidate frames. This means the mechanism can preserve nearby visual evidence, but it is not guaranteed to preserve the exact frame referenced by the benchmark question.

This helps explain the mixed quantitative behaviour observed in Section 4.7. Query selection improves camera-object relative distance, suggesting that filtering can remove redundant or irrelevant views while keeping enough object evidence for relative comparison. However, because the selector is driven by semantic similarity rather than explicit frame-index constraints, it can be misaligned with tasks whose correctness depends on the exact requested timestep. For future work, a stronger VSTI-specific selector should condition not only on the text semantics of the question, but also on any explicit temporal reference such as “frame 23 of 32”, ensuring that the named frame and its local temporal neighbourhood are always retained.

5.10 Failure Mode Analysis

The analysis of per-sample predictions reveals distinct failure patterns across spatial reasoning task types. We compare absolute matches between prediction and ground truth unless expressed otherwise. Absolute metric tasks (distance, displacement, size estimation) show systematically poor performance due to the model’s inability to translate visual features into metric predictions. Relative spatial reasoning tasks

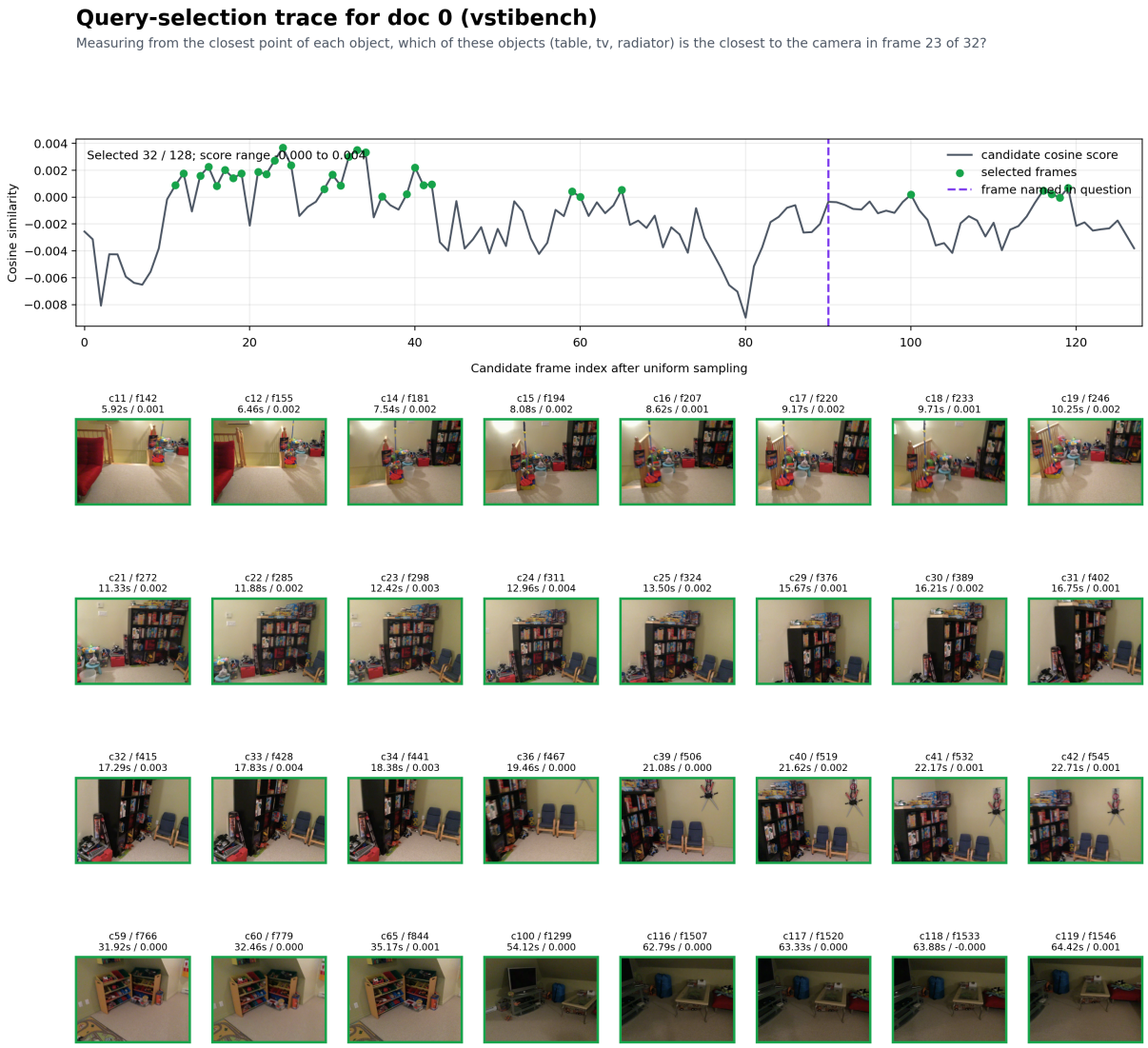


Figure 5.8: Query-selection trace for a VSTI-Bench camera-object relative distance question. The line plot shows cosine similarity scores for 128 candidate frames, green markers show the selected top-32 frames, and the dashed purple line marks the frame explicitly named in the question after mapping from the benchmark's 32-frame numbering to the 128 candidate-frame sequence. The thumbnails below show the selected frames in chronological order.

perform substantially better, with the model successfully identifying spatial relationships in video. These patterns manifest differently across VSI-Bench and VSTI-Bench due to their different question types and evaluation protocols.

5.10.1 Per-Category Performance on VSTI-Bench

The VSTI-Bench results reveal distinct performance patterns across question types. Multiple-choice questions (camera-object relative distance variants, camera movement direction, object-object relative position variants) show strong performance when evaluated against the correct field: the model accurately selects the appropriate option letter. Numerical questions (camera-object absolute distance, camera displacement) perform significantly worse due to the absolute metric reasoning limitations discussed in Section 5.10.2.

5.10.2 Why Absolute Metrics Fail

The contrast between strong relative reasoning (94.2% on up-down object relationships, 83.1% on easy relative direction) and significantly lower performance on absolute tasks points to a representation issue. Relative spatial relationships can be computed from angular or directional cues preserved in video frames, but absolute metric reasoning requires understanding scale and metric depth that visual features do not encode with sufficient precision.

The model stores and retrieves spatial features but does not learn to translate these features into metric predictions. This is most visible in camera displacement: 52% (208/789) of the failed cases default to predicting `1.0`, suggesting the model has no meaningful metric signal and simply outputs a fallback value. Concrete example: for a ground truth displacement of `2.4m`, the model predicts `1.0m`; for `3.1m`, the model again predicts `1.0m`. Camera-object absolute distance shows a similar pattern with common predictions of `1.8`, `2.8`, `1.0` regardless of ground truth values ranging from `0.3m` to `5.2m`.

The MRA (Most Recent Appearance) confidence scores show a clear correlation with accuracy on VSTI-Bench: samples with MRA=0.0 have 0% accuracy, while MRA=1.0 samples achieve 72% accuracy. However, MRA values are bimodally distributed (either 0.0 or 1.0 for most samples), with intermediate values (0.1–0.9) showing 0% accuracy. This indicates that the MRA signal provides useful confidence information but does not guarantee correct predictions.

Object counting shows a different pattern: it is not a complete categorical failure (56.1% accuracy) but exhibits a systematic under-count bias. Concrete example: when asked *"How many chair(s) are in this room?"* with ground truth `6`, the model predicts `2`. Analysis shows the model predicted `2` in 127 of 248 failure cases while ground truth values ranged from `3` to `12`. This suggests the model and memory representation supports approximate counting but has insufficient resolution for precise enumeration, defaulting to conservative lower counts when visual clutter or occlusion obscures objects.

5.10.3 VSI Bench

Spatial reasoning on VSI-Bench fails categorically on four task types: absolute distance, object counting (with bias), object size, and room size. The model can successfully identify which object is closer (relative distance, 65.2%) but cannot determine how far away an object is (absolute distance, 5.8%).

Appearance order (40.5%) and route planning (45.4%) also underperform, suggesting that temporal sequencing and navigation path reasoning remain challenging even with memory augmentation. The arkitscenes dataset is notably more difficult across all task types as spoken about in Section 5.4.1.

5.10.4 VSTI Bench

VSTI-Bench shows a clear split between multiple-choice and numerical question types. Multiple-choice questions (object-object relative position, camera-object relative distance variants, camera movement direction) demonstrate strong performance when correctly evaluated against `mc_answer`, with 80-94% accuracy on object-object position tasks. Numerical questions (camera-object absolute distance, camera displacement) perform substantially worse, consistent with the absolute metric reasoning limitations observed in VSI-Bench and discussed in Section 5.10.2. The model provides sufficient representation for relative spatial reasoning but fundamentally cannot support metric depth estimation required for absolute spatial measurements.

Chapter 6

Conclusion

6.1 Contributions and Findings

The main contributions of this work, in relation to the aims presented in Section 1.4.2, can be summarised as follows:

1. Recreated the performance claimed by VLM-3R model in Section 4.4.
2. Implemented (in Sections 3.5 and 4.5) and evaluated (in Section 5.1.1) a non-trainable dual-memory module that augments the VLM-3R architecture with working and episodic memory systems.
3. Implemented (in Sections 3.6 and 4.6) and evaluated (in Section 5.1.2) a trainable dual-memory module that augments the VLM-3R architecture with working and episodic memory systems.
4. Implemented (in Section 4.7) and evaluated a query-based frame selection module that augments the VLM-3R architecture.
5. Assessed the impact of dual-memory on spatial reasoning performance across VSI-Bench and VSTI-Bench benchmarks in Section 5, particularly for longer video sequences in Section 5.2.
6. Assessed the impact of query-based frame selection on spatial reasoning performance across VSI-Bench and VSTI-Bench benchmarks in Section 5, particularly for complex scenes in Section 5.6.
7. Compared trainable and non-trainable memory approaches to understand the trade-offs between adaptation capability and implementation simplicity in Section 5.1.3.
8. Analysed the frames selected by the query-based frame selection mechanism and the information content in those frames relative to the input query in Section 5.9.
9. Analysed the failure modes of vision-language models on spatial reasoning tasks to identify where memory and frame selection augmentation changes performance in Section 5.10.

6.2 Project Status

I have completed an ambitious research project that addresses all of the aims outlined in Section 1.4.2. The VLM-3R baseline is working and was successfully reproduced, with evidence in Section 4.4. I implemented the non-trainable memory, trainable memory, query-based frame selection, and combined architectures described in Sections 3.5, 3.6, 3.7, and 3.8, and evaluated them across VSI-Bench and VSTI-Bench in Chapter 5. The final system should be understood as a research prototype rather than a deployed model: the conservative non-trainable and query-selection variants preserve or match baseline behaviour in some settings, while the trainable-memory and combined variants reveal clear limitations of late-stage memory insertion. These negative results are still a meaningful project outcome, because they are supported by benchmark evaluation, long-horizon analysis, frame-selection analysis, and failure-mode analysis in Sections 5.2, 5.9, and 5.10. Overall, the project demonstrates substantial self-directed work across literature review, implementation, distributed training, HPC experimentation, benchmark reproduction, ablation, and critical evaluation, with the strongest contribution being a careful empirical assessment of which augmentations to VLM-3R are safe, which are harmful, and why.

6.2.1 Appropriate Hardware Access

This project relied on access to hardware that was capable of training and evaluating the model. Access to appropriate hardware was somewhat sparse, included switching projects which requires re downloading and preprocessing the datasets and included periods of downtime even when available. This created large gaps in time where training, debugging and evaluation could be carried out, potentially restricting me from finding or training optimisations to beyond VLM-3R performance.

See Appendix C.1 for a detailed timeline outlining a detailed timeline of my access to appropriate hardware for this project, including: access to different systems, planned maintenance, unplanned downtime.

6.3 Future Work

6.3.1 Further Training

With more time and compute, there are a few obvious points to continue from:

- Training and evaluating the model end to end on: query based frame selection, trainable memory, combined architecture
- Training and evaluating different parameters in the modules:
  - Different input and output sizes for the query based frame selection
  - Different MLP configurations for the query based frame selection
  - Different memory sizes for working and episodic memory
  - Different MLP architecture for the memory fusion
  - Further hyperparameter tuning over all types of training

6.3.2 New Augmentations

The main limitation of the attempted augmentations is that they operate on frame-level features. Future work should expose more structure before the LLM stage. VideoRefer suggests object-level spatial-temporal tokens: masks from a detector or SAM-style segmenter could pool SigLIP/CUT3R features into object tokens, while a temporal merge step could combine repeated observations of the same object across frames [94]. This would be especially useful for object counting, relative direction, relative distance, and appearance order, where the model currently has to recover object identity from compressed global frame tokens.

Video-3D LLM suggests a complementary geometry-focused extension [101]. Rather than selecting frames uniformly or only by query similarity, future work could use camera poses and depth maps to choose frames by maximum 3D voxel coverage. This would protect global tasks such as room size, route planning, and object counting from losing important context. The same geometry could also be injected into visual-spatial tokens through a gated 3D position encoding, giving the model more direct access to metric structure without letting noisy depth predictions dominate semantic features.

Finally, this project only adapted the query-selection part of VideoStreaming, not its full streaming-memory design [63]. A fuller implementation would encode long videos clip-by-clip, propagate compact historical memory between clips, and use question-conditioned clip indicators to retrieve the most relevant stored memories. A practical hybrid would combine these ideas: first guarantee broad 3D coverage, then re-rank or retrieve frames and memories according to the question.

6.3.3 Different Architecture

A more ambitious direction is to move away from late-stage add-ons. In this project, memory was inserted after VLM-3R's visual-spatial fusion and projection pathway, which may explain why trainable memory degraded performance: it was modifying features already calibrated for the baseline model. VLM^2 instead builds memory on top of view-consistent 3D-aware tokens [50].

One concrete route would be to replace or complement CUT3R with π^3, the 3D foundation model

used by VLM^2 [50]. This would allow VLM^2's adaptive 3D position injection, viewpoint-aware geometry alignment, and cross-attention semantic-geometric fusion to be tested inside the VLM-3R setting. Memory would then store aligned semantic and geometric representations, rather than flat post-projector frame features.

This architecture would also support richer memory entries containing object, view, and approximate 3D position information. For example, a relative-direction question could retrieve memories containing the relevant objects and their camera-relative geometry, while a room-size question could retrieve memories chosen for broad spatial coverage. The key difference is that memory would become part of the 3D representation pipeline itself, rather than a downstream correction applied after fusion.

Bibliography

[1] 3Blue1Brown. But what is a gpt? visual intro to transformers – chapter 5. https://www.youtube.com/watch?v=kCc8FmEb1n, April 2024. Accessed: 28 March 2026.

[2] Meta AI. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[3] Anthropic. Claude model card. *Anthropic Research*, 2023.

[4] Iro Armeni, Yinyu He, Sarthak Joshi, Nicholas Brooks, Adrian Hilton, Rio Kadamba, Jiaping Luo, Greg Mori, Guy Rosman, Kosta Saenko, et al. Scannet++: A high-fidelity indoor scene dataset with authentic appearance. *arXiv preprint arXiv:2212.02316*, 2022.

[5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[7] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[8] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[10] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, and John J Leonard. Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age. In *IEEE Transactions on Robotics*, volume 32, pages 1309–1332, 2016.

[11] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024.

[12] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Mai, Daniel Schuurmans, Erik Mueller, Fei Li, Yao Wu, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.

[13] Ozgun Cicek, Ahmed Abdulkadir, Stephan S Lienkamp, Thomas Brox, and Olaf Ronneberger. 3d u-net: Learning dense volumetric segmentation from sparse annotation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 424–432, 2016.

[14] NVIDIA Corporation. Volta: A hybrid cpu+gpu architecture for massive scale computing. In *SC17*, 2017.

[15] NVIDIA Corporation. Ampere: Nvidia's next generation gpu architecture. In *Hot Chips*, 2020.

[16] NVIDIA Corporation. Nvidia h100 tensor core gpu architecture. NVIDIA Whitepaper, 2022. URL: https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c.

[17] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[18] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: A richly-annotated 3d reconstruction of indoor scenes. pages 5828–5839, 2017.

[19] Tri Dao. Flashattention: Fast and memory-efficient exact attention with io-awareness. 35:16344–16359, 2022.

[20] Riccardo De Lutio, Alessandro D'Antonio, Panos Kalogiannis, Alexandre Locher, Ioannis Havoutis, Diego Marcucci, Ricardo Artmed Borsoi, and Dario Izzo. Arkitscenes: A diverse real-world dataset for mobile indoor planning. *arXiv preprint arXiv:2105.02338*, 2021.

[21] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *CVPR*, 2009.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics*, pages 4171–4186. ACL, 2019.

[23] Jack Dongarra, Ian gate, and Przemyslaw Luszczek. High-performance computing systems: Status and outlook. *Acta Numerica*, 23:1–54, 2014.

[24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[25] Oscar Esteban, Rich Bhatia, et al. Medical image analysis with deep learning: A survey. *Medical Image Analysis*, 80:102–121, 2022.

[26] Zhiwen Fan, Jian Zhang, Renjie Li, Junge Zhang, Runjin Chen, Hezhen Hu, Kevin Wang, Huaizhi Qu, Dilin Wang, Zhicheng Yan, Hongyu Xu, Justin Theiss, Tianlong Chen, Jiachen Li, Zhengzhong Tu, Zhangyang Wang, and Rakesh Ranjan. Vlm-3r: Vision-language models augmented with instruction-aligned 3d reconstruction. *arXiv preprint arXiv:2505.20279*, 2025. URL: https://arxiv.org/abs/2505.20279, doi:10.48550/arXiv.2505.20279.

[27] Yasutaka Furukawa and Carlos Hernández. Multi-view stereo: A tutorial. *Foundations and Trends in Computer Graphics and Vision*, 9(1–2):1–148, 2015.

[28] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 3354–3361, 2012.

[29] Alexander Gorban, Elena Kussul, et al. Face detection in still images. volume 33, pages 65–79, 2015.

[30] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[32] Heiko Hirschmüller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):328–341, 2007.

[33] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[34] Edward J Hu, Yelong Shen, Phil Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Lu Wang. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. arXiv:2106.09685.

[35] ImageIO Contributors. Npz: Numpy's compressed array format. https://imageio.readthedocs.io/en/v2.9.0/format_npz.html, 2020. Accessed: 2026-04-19.

[36] Ziwei Ji, Nayeon Lee, Rita Frye, and Preslav Lee. Survey on hallucination in large language models. *arXiv preprint arXiv:2309.05922*, 2023.

[37] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[38] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[39] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, volume 25, 2012.

[40] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[41] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. MASt3R: Grounding image matching in 3D with MASt3R. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[42] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025.

[43] Linjie Li, Yuan Yuan, et al. Hero: Hierarchical encoder for video+language omni-representation comprehension. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 2046–2060, 2020.

[44] Xi Li, Weiming Hu, Chunhua Shen, Anthony Dick, and Marcel Honderd-Boer. Deep learning in visual tracking: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 31(1):1–19, 2020.

[45] Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. Vila: On pretraining for visual language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[46] Robert K. Lindsay, Bruce G. Buchanan, Edward A. Feigenbaum, and Joshua Lederberg. Heuristic dendral: A program for generating explanatory hypotheses in organic chemistry. *Machine Intelligence*, 3:209–254, 1968.

[47] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. URL: https://llava-vl.github.io/blog/2024-01-30-llava-next/.

[48] Haotian Liu, Chunyuan Li, Qing Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2024.

[49] Yizhou Liu, Hao Peng, Yiran Xiao, Pengchuan Chen, Qiang Liu, Zhekang Zheng, Yalou Zhang, Haoyu Liu, Wen Wang, Renfang Xu, et al. Lmms-eval: A framework for cross-language multimodal model evaluation. *arXiv preprint arXiv:2411.14155*, 2024.

[50] Zuntao Liu, Yi Du, Taimeng Fu, Shaoshu Su, Cherie Ho, and Chen Wang. Vision-language memory for spatial reasoning, 2025. URL: https://arxiv.org/abs/2511.20644, arXiv:2511.20644.

[51] Haotian Luo, Yuhang Li, Youhe Kwon, Yiyuan Zhang, Kaicheng Liu, Songyang Chen, Tao Ge, Wentian Liu, et al. Llava-next: Improved reasoning, ocr, and world knowledge. *arXiv preprint arXiv:2401.10268*, 2024.

[52] Antoine Miech, Ivan Laptev, and Josef Sivic. Howto100m: Learning a text-video embedding by watching hundred million narrated video clips. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2630–2640, 2019.

[53] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26, 2013.

[54] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2022.

[55] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):1–15, 2022.

[56] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*, 2023. arXiv:2307.06435.

[57] Allen Newell, Herbert A. Simon, and J. C. Shaw. The logic theory machine—a complex information processing system. *IRE Transactions on Information Theory*, 2(3):61–79, 1956. doi:10.1109/TIT.1956.1056797.

[58] University of Bristol. Lifting the lid on isambard-ai, the uk's fastest ai supercomputer. *Bristol Centre for Supercomputing Technical Report*, 2025. URL: https://www.bristol.ac.uk/research/centres/bristol-supercomputing/articles/2025/lifting-the-lid-on-isambard-ai.html.

[59] OpenAI. Chatgpt: Optimizing language models for dialogue. *OpenAI Blog*, 2022.

[60] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[61] Alejandro Ouini et al. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023. arXiv:2304.03277.

[62] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[63] Rui Qian, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Shuangrui Ding, Dahua Lin, and Jiaqi Wang. Streaming long video understanding with large language models, 2024. URL: https://arxiv.org/abs/2405.16009, arXiv:2405.16009.

[64] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. *International conference on machine learning*, pages 8748–8763, 2021.

[65] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[66] Jack W Rae, Sebastian Borgeaud, Trevor Cai, et al. Scaling language models: Methods, analysis and insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.

[67] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12179–12188, 2021.

[68] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1623–1637, 2022.

[69] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

[70] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[71] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.

[72] SchedMD. Slurm workload manager. https://slurm.schedmd.com/, 2025. Version 25.11.

[73] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016.

[74] Johannes Lutz Schönberger, Enliang Zheng, Marc Pollefeys, and Jan-Michael Frahm. Pixelwise view selection for unstructured multi-view stereo. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 501–518, 2016.

[75] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152, 2012.

[76] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2015.

[77] Edward H. Shortliffe. Computer-based medical consultations: Mycin. 1976.

[78] Heidi Strohm and Jack Dongarra. Top500 list announcement. In *SC23: The International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

[79] Chen Sun, Fabien Baradel, Kevin Murphy, and Carolina Fermuller. Videobert: A joint model for video and language representation learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7464–7473, 2019.

[80] Quan Sun, Yuxin Fang, Ledell Wu, Xinlong Wang, and Yue Cao. Eva-clip: Improved training techniques for clip at scale. *arXiv preprint arXiv:2303.15389*, 2023. arXiv:2303.15389.

[81] Gemini Team. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[82] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Aude Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[83] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950. doi:10.1093/mind/LIX.236.433.

[84] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[85] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A. Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. *arXiv preprint arXiv:2501.12387*, 2025.

[86] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jérôme Revaud. DUSt3R: Geometric 3D vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[87] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

[88] Diankun Wu, Fangfu Liu, Yi-Hsin Hung, and Yueqi Duan. Spatial-mllm: Boosting mllm capabilities in visual-based spatial intelligence. *arXiv preprint arXiv:2505.23747*, 2025.

[89] Hang Xu, Zhipeng Ma, et al. Videollm: Modeling video sequence with large language models. *arXiv preprint arXiv:2305.13292*, 2023.

[90] Ming Xu, Chengyu Li, et al. More than just skip connections: Hierarchical fusion and cross-attention for video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1561–1570, 2021.

[91] Fuzhao Xue, Yukang Chen, Dacheng Li, Qinghao Hu, Ligeng Zhu, Xiuyu Li, Yunhao Fang, Haotian Tang, Shang Yang, Zhijian Liu, et al. Longvila: Scaling long-context visual language models for long videos. In *International Conference on Learning Representations*, 2025.

[92] Jihan Yang, Shusheng Yang, Anjali Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in Space: How Multimodal Large Language Models See, Remember and Recall Spaces. *arXiv preprint arXiv:2412.14171*, 2024.

[93] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. *arXiv preprint arXiv:2401.10891*, 2024.

[94] Yuqian Yuan, Hang Zhang, Wentong Li, Zesen Cheng, Boqiang Zhang, Long Li, Xin Li, Deli Zhao, Wenqiao Zhang, Yueting Zhuang, Jianke Zhu, and Lidong Bing. VideoRefer suite: Advancing spatial-temporal object understanding with Video LLM, 2025. URL: https://arxiv.org/abs/2501.00599, arXiv:2501.00599.

[95] Xiaohua Zhai, William Wang, Basil Fussner, Evelyn hu, Alexander Kolesnikov, Joel Eble, and Chaithanya Kumar Mummadi. Siglip: Simple visual language model pre-training with sigmoid loss. *arXiv preprint arXiv:2312.12231*, 2023.

[96] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. Long context transfer from language to vision. *arXiv:2406.16852*, 2024.

[97] Peng Zhang, Xi Li, Xin Hu, Yichun Liu, Zhi Jiang, Lin Wang, and Yuan Tian. Large vision-language models: A survey. *arXiv preprint arXiv:2309.05922*, 2023.

[98] Yuanhan Zhang, Bo Li, Haotian Liu, Yong Jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. Llava-next: A strong zero-shot video understanding model, 2024.

[99] Chaoqiang Zhao, Qiyu Sun, Chongzhen Zhang, Yang Tang, and Feng Qian. Monocular depth estimation based on deep learning: An overview. *arXiv preprint arXiv:2003.06620*, 2020.

[100] Duo Zheng, Shijia Huang, Yanyang Li, and Liwei Wang. Learning from videos for 3d world: Enhancing mllms with 3d vision geometry priors. *arXiv preprint arXiv:2505.24625*, 2025.

[101] Duo Zheng, Shijia Huang, and Liwei Wang. Video-3D LLM: Learning position-aware video representation for 3D scene understanding, 2025. URL: https://arxiv.org/abs/2412.00493, arXiv:2412.00493.

Appendix A

Appendix A: AI Prompts/Tools

A.1 Tools

A.1.1 ChatGPT

Used as a general assistant to help with search, code debugging, and ideation.

A.1.2 Claude

Used as a general assistant to help with search, code debugging, and ideation.

A.1.3 Gemini

Used as a general assistant to help with search, code debugging, and ideation.

Google Search AI

Google search includes (Gemini) AI results above the links - I frequently use Google search and read the response which is typically a summary of the most relevant information for top search suggestions.

A.1.4 Minimax Models

Used as a general assistant to help with Linux commands and code debugging. Especially used for long output logs that contain long error messages or traces - it's large context windows and very cheap plans allow it to ingest logs that would quickly consume all my credits with other AI tools. Examples include: "Linux command for finding how much space is left in scratch", "how to create a symlink between /path to x and /path to y", "Extract from the attached log where the core part of the error message is".

Harnessing Minimax Models

Opencode: An open source agent. This enables use of the minimax models.

Claude Code: An agent by Anthropic. This enables use of the minimax models.

A.1.5 Codex Visual Studio Code Extension

Used as a general assistant to help with Linux commands.

A.2 Example Prompts

- "Extract from the attached log where the core part of the error message is"
- "Linux command for finding how much space is left in scratch"
- "How to create a symlink between /path to x and /path to y"

- "What are my current qos limits on this cluster?"
- "See the attached image. How do I centre this algorithm in latex"
- "See the attached image. How do I make one subcategory coloured in the header of this table in latex"
- "What tool/s can I use to visualise point cloud data?"

Appendix B

Appendix B: Supplementary Material

B.1 Qualitative examples on VSI-Bench

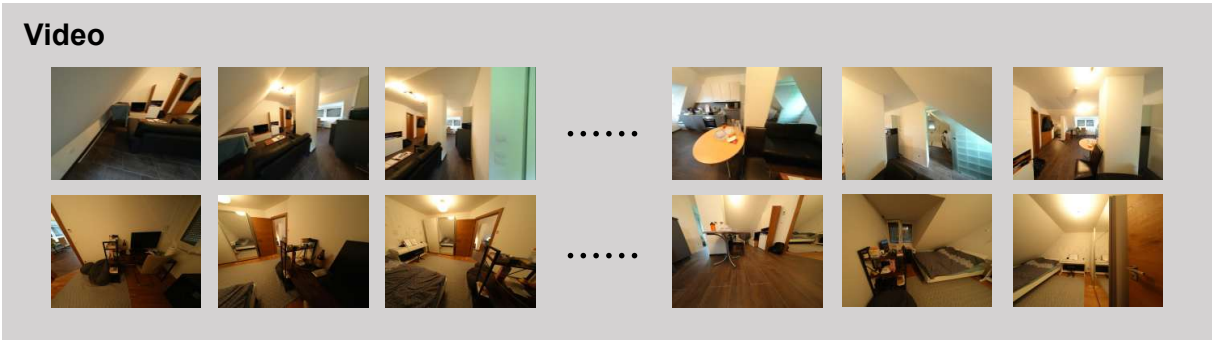
Figures B.1 to B.3 show qualitative examples of VSI-Bench [92] spatial reasoning tasks. Reprinted from Fan et al. [26].

B.2 Qualitative examples on VSTI-Bench

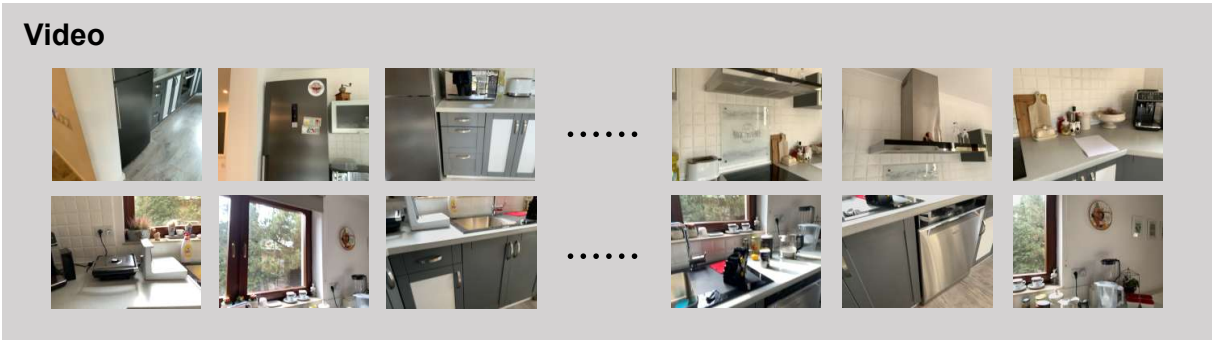
Figure B.4 and Figure B.5 show qualitative examples of VSTI-Bench [26] spatial-temporal reasoning tasks. Reprinted from Fan et al. [26].



Question: *How many chair(s) are in this room?* **Answer**: *6*

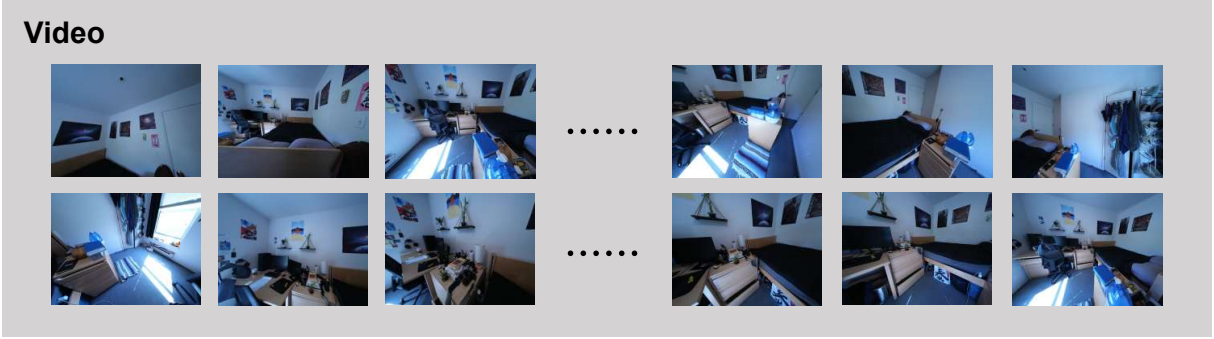


Question: *Measuring from the closest point of each object, what is the distance between the bed and the sofa (in meters)?* **Answer**: *3.2*



Question: *You are a robot beginning at the kitchen sink and facing window. You want to navigate to the refrigerator. Options: A. Turn Back B. Turn Right C. Turn Left* **Answer**: *A*

Figure B.1: **Qualitative examples on VSI-Bench [92].**



Question: *What is the size of this room (in square meters)?* **Answer**: *10.5*

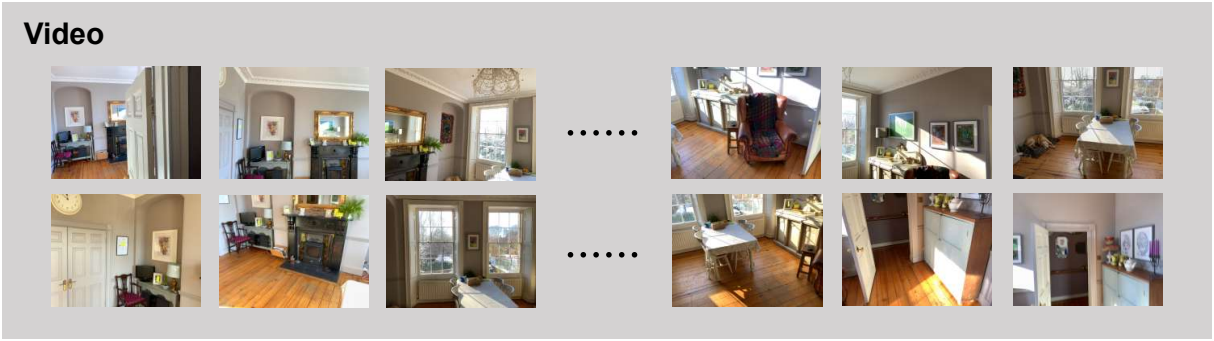


Question: *What is the length of the longest dimension of the chair, measured in centimeters?* **Answer**: *69*

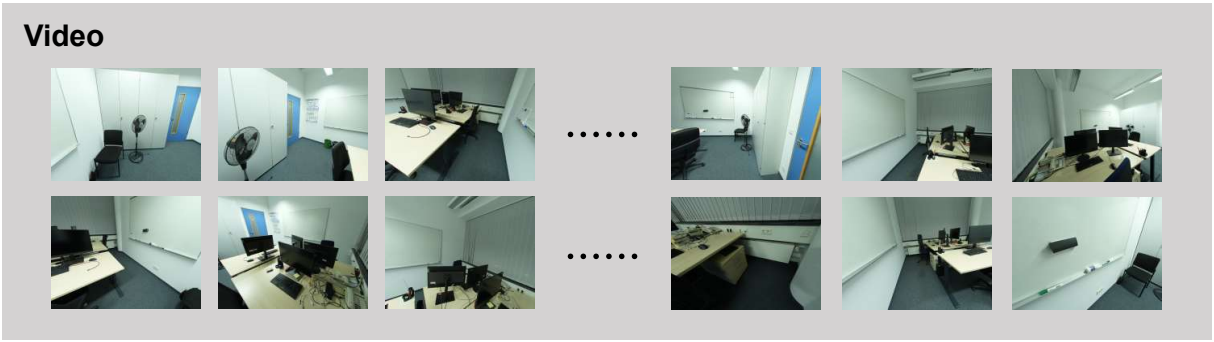


Question: *What will be the first-time appearance order of: basket, door, pillow, laptop? Options: A. basket, pillow, door, laptop B. pillow, door, laptop, basket C. basket, door, pillow, laptop D. door, basket, pillow, laptop* **Answer**: *B*

Figure B.2: **Qualitative examples on VSI-Bench [92].**



Question: *Measuring from the closest point of each object, which of these objects (chair, table, tv, fireplace) is the closest to the sofa? Options: A. chair B. table C. tv D. fireplace* **Answer**: *B*



Question: *If I am standing by the computer tower and facing the heater, is the door to my left, right, or back? Options: A. back B. right C. left* **Answer**: *A*

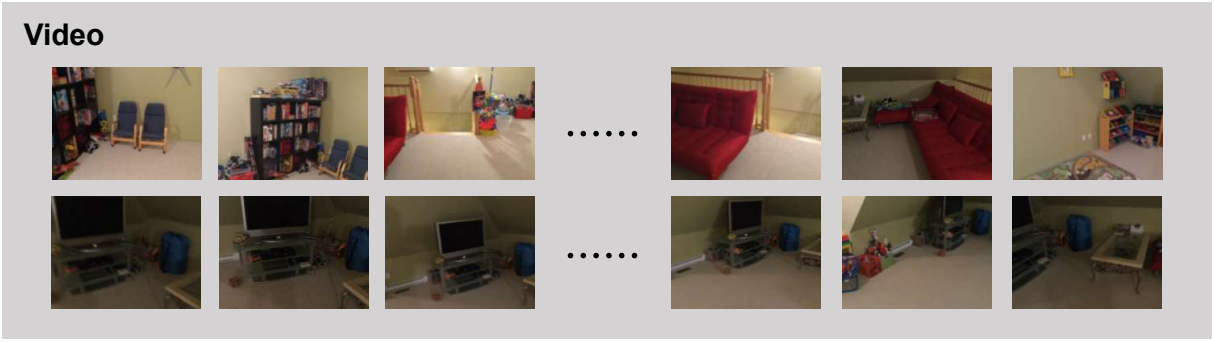


Question: *If I am standing by the ceiling light and facing the door, is the kettle to my front-left, front-right, back-left, or back-right? Options: A. front-left B. back-right C. front-right D. back-left* **Answer**: *D*

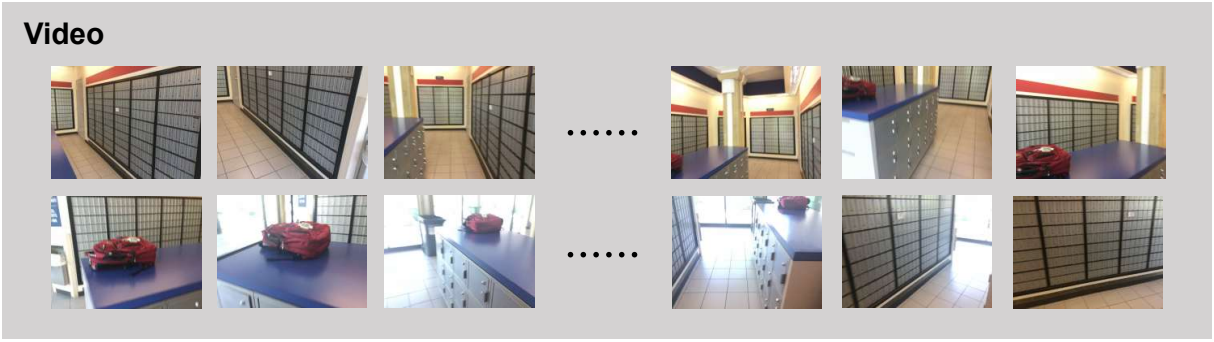
Figure B.3: **Qualitative examples on VSI-Bench [92].**



Question: *Measuring from the closest point of each object, which of these objects (plant, window) is the closest to the camera in frame 4 of 32? Options: A. plant B. window* **Answer**: *B*

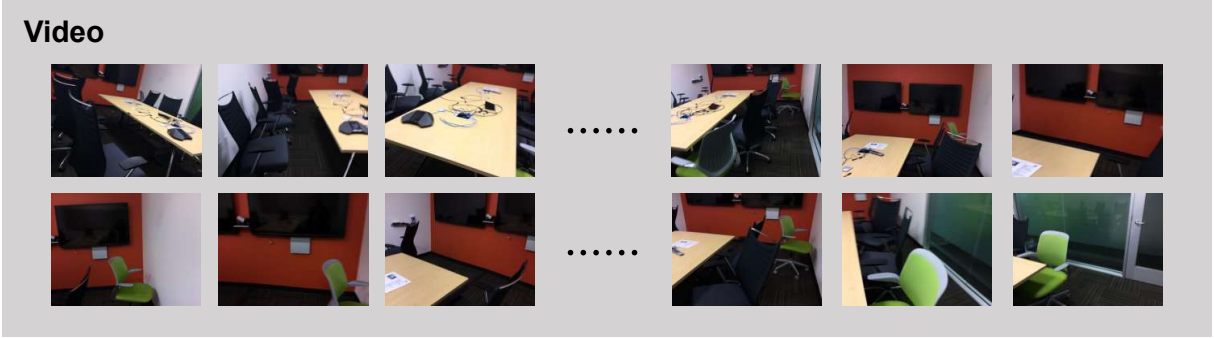


Question: *During the sequence between frame 4 and frame 23 of 32, what was the primary consistent direction of the camera's movement? Options: A. Forward B. Left C. Right* **Answer**: *C*

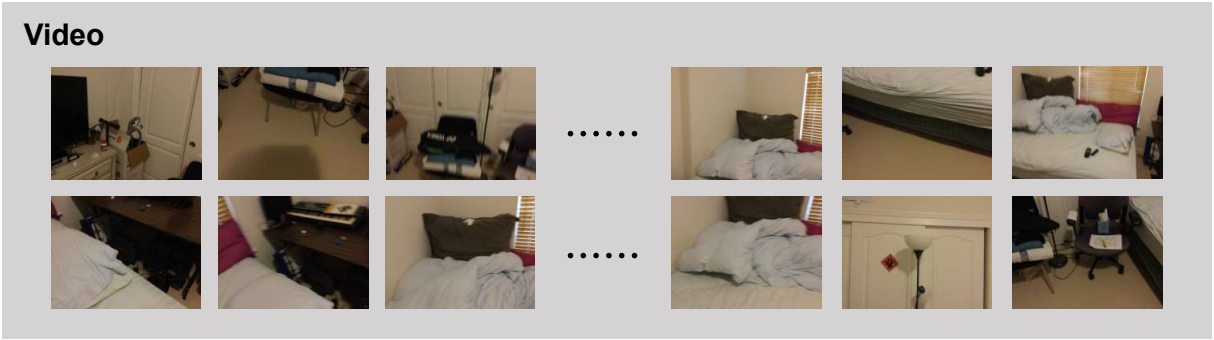


Question: *What is the approximate distance (in meters) between the camera and the nearest point of the backpack in frame 10 of 32?* **Answer**: *0.7*

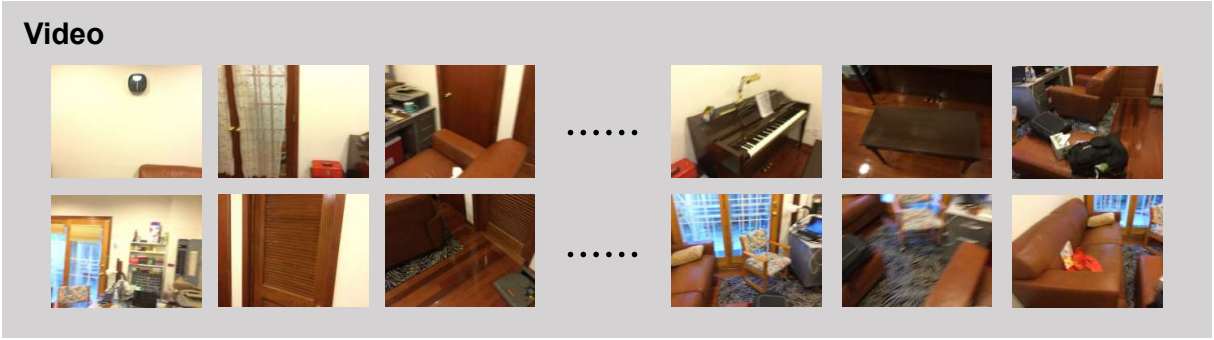
Figure B.4: **Qualitative examples on VSTI-Bench [26].**



Question: *Approximately how far (in meters) did the camera move between frame 24 and frame 31 of 32?* **Answer**: *1.2*



Question: *In frame 13 of 31, relative to monitor, is bed to the Left/Right? Options: A. Right B. Left* **Answer**: *B*



Question: *In frame 1 of 31, relative to clock, is piano to the Up/Down? Options: A. Down B. Up* **Answer**: *A*

Figure B.5: **Qualitative examples on VSTI-Bench [26].**

Appendix C

Appendix C: Project Hardware

C.1 Hardware Access Timeline

- Week 1: No work due to ACL injury
- Week 2: Access to Blue Pebble 1 and ran the demo script. 1 of the 2 available MLCNU nodes was appropriate for this - other partitions on BP1 are unsuitable.
- Week 3:
  - Access is granted to the 3 required datasets but Blue Pebble 1's 1TB space is not enough to hold them.
  - Day 1: Access granted to my supervisor's Isambard AI Phase 2 project and have to build some dependencies from source for new architecture.
  - Day 2: Start downloading scannet again and scannet++ data.
  - Day 3-7: Downloading and preprocessing.
- Week 4: Isambard AI service is down for 2 days. The networking issues so data still downloading during this week. Week finishes with performance recreation of VLM-3R on VSI and VSTI bench. Isambard AI maintenance starts end of this week.
- Week 5: Maintenance for 3 days then nodes temperamental for last 3 days. Some work done on debugging training.
- Week 6: Exploding gradient problem arises then the project runs out of node hours - No access for end of the week .
- Week 7: New Isambard AI access mid week - environment dependencies and datasets are re setup on a new project - continue debugging trainable memory and run non trainable memory experiments.
- Week 8: Isambard AI project runs out of credits due to other's use. Setup on BP1 trying to train with just the scannet dataset but queue times for MLCNU are long and don't get past setup debugging.
- Week 9: Spend 4 days in BP1 queue before nodes go down for the rest of the week.
- Week 10: 1 MLCNU nodes on Blue Pebble is 'planned down' and one 'draining' - waiting to go down.
- Week 11: MLCNU nodes down until Sunday.
- Week 12: New Isambard AI access but only 18 nodes are up and there is network issues for the first half of the week so datasets can't be downloaded. I re setup the environments and then when network is restored I download data and do dataset preprocessing.
- Week 13: Debug working memory training and non trainable memory ablations.
- Week 14: No credits left on the project.

- Week 15: More credits added to the same project. Isambard AI is down early part of the week then there is a 1 job per user limit when brought up at the end of the week. I implement query based frame selection and begin training it.
- Week 16: Query based frame selection is trained, trainable memory is trained: both are evaluated - both trained about separately and jointly then evaluated again. Different parameter choices are trained and evaluated.